

CLOUD PROGRAMMING & OPERATIONS

0) Cloud Native Architectures

Modern sw operates under **permanent stress**: ephemeral infrastructure, unpredictable load, partial and silent failures, changing teams, human errors (from “**control the system**” [*past*] to “**declare intent**” of what the software have to do [*now*]), continuous deployments.

Move to **cloud-based architectures** has been the result of the convergence of **business needs** (agility, efficiency...), **tech pressure** (scale, resilience, complexity management), **user expectation** (always-on, fast, secure services...). **CLOUD COMPUTING** is the **delivery of computing services** (e.g. servers, storage, db, AI...) **over the internet** (*the cloud*) instead of using local resources.

Cloud computing service models:

- **IaaS** (**I**nfrastructure as a Service; **I**nfrastructure platform) → provides computing, storage and network resources (**Azure vm**). Customers manage OS and apps, while provider manages hw. *Cost model* = **pay-per-resource**
- **CaaS** (**C**ontainer as a Service; **C**ontainer platform) → provides container engine, orchestrator and underlying infrastructure (**Kubernetes** [*K8*]). Customers deploy containers (not vm). *Cost model* = **pay for cluster resources**
- **PaaS** (**P**latform as a Service; **A**pplication platform) → provides development and deployment tools, APIs and underlying infrastructure (**Heroku, Azure App Service**). Provider manage scaling, OS and load-balancing. *Cost model* = **pay per application instance** and/or **resource tier**
- **FaaS** (**F**unction as a Service; **S**erverless platform) → provides runtime + infrastructure to run functions and autoscaling (**AWS Lambda, Azure Functions**). Customer just upload functions and don't manage resources. *Cost model* = **pay per execution**
- **SaaS** (**S**oftware as a Service; **S**oftware platform) → provides software + infrastructure to run it; fully managed app accessible via web or API (**GitHub, Slack, Salesforce, Google Workspace**). Customers consume, don't deploy. *Cost model* = **subscription-based**

Benefits of cloud computing are:

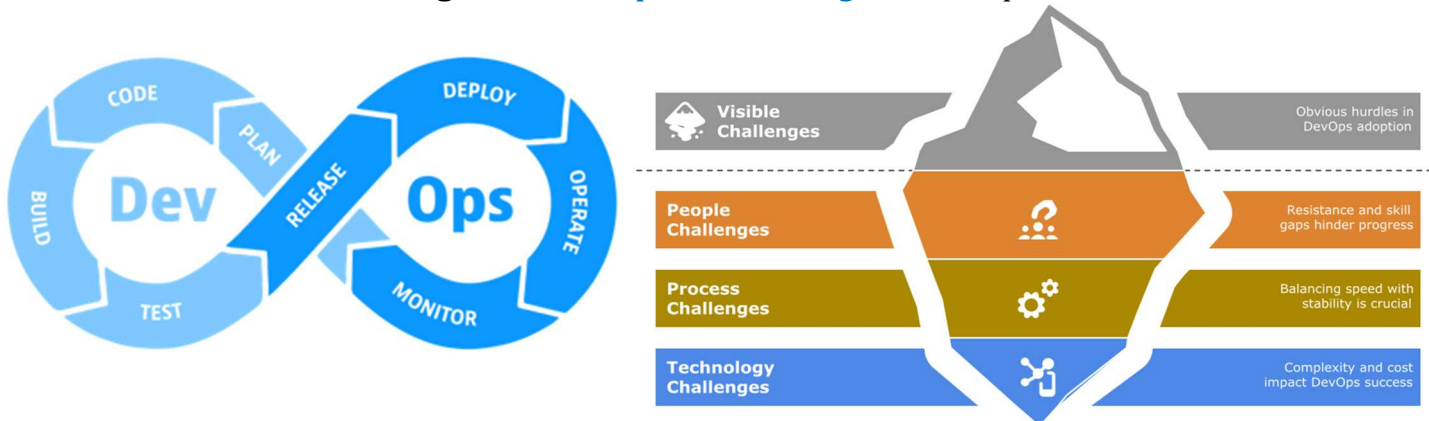
- **Cost efficiency**
- **Elasticity** (requires stateless or loosely coupled services to scale in and out instances)
- **Scalability** (requires modular design, database sharing/replication and asynchronous messaging)
- **Global reach**
- **Security by design**
- **Automation** (only if deployments are scripted, configurations are managed via Infrastructure as Code and app supports automated provisioning)
- **Resilience** (depends on distributed architectures, fault isolation and data replication)
- **Speed** (breaking work in small units and having CI/CD pipelines to push changes quickly and safely)
- **Continuous innovation**

Cloud-native applications are **Horizontally scalable** (*Scale-out* = add more nodes to parallelize the load; *Scale-up* [*Vertical*] = add more power to 1 node), **Observable**, **Portable**, **Interoperable**, **Resilient**.

Cloud-native design is made of **microservices architecture**, **containerization**, **API-driven communication** (often REST or gRPC), **elastic scalability**, **resilience**/fault tolerance, **immutable infrastructure** and **CI/CD** (*Continuous Integration & Continuous Delivery*), **observability** (measurable and transparent), **security by design** (**DevSecOps** ↓ [security from initial phase]), cloud service leverage.

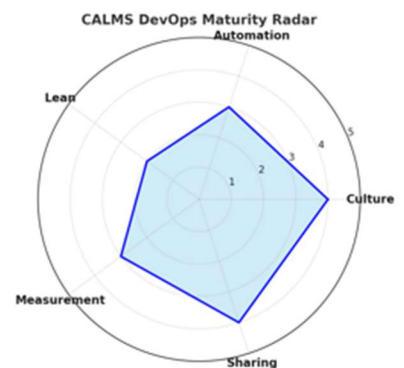
1) DevOps

DevOps is a set of practices, principles and philosophies that aim to unify software development (**Dev**) and IT operations (**Ops**). Purpose is to **shorten development lifecycle**, **increase deployment frequency**, **improved collaboration**, **fast delivery** on market, **resilience** (+ fast recovery from failures [$< MTTR$]) and **automation**. Works well with **Agile**. Under **steps** and **challenges** of DevOps:



DevOps **CALMS** model:

- **Culture** = not only tools, but cultural change to promote collaboration, trust and **shared responsibility** between dev, ops, QA, security and business (*failures are learning opportunities, not personal faults*) [teams must be **multidisciplinary** and must be of 6-10 members: Software engineers, Platform (ops) engineers, QA (quality) engineers, Product representative, Security champion]
- **Automation** = eliminate repetitive manual tasks, speed up delivery and reduce human error
- **Lean** = remove waste, focus on faster delivery and improve processes
- **Measurement** = metrics of performance, quality and business impact (both system and process); **DORA metrics** → measure capability (not effort) and focus on outcomes (not activity); all 4 together:



If you improve only...	You might get...
Deployment frequency	Faster failure
Lead time	Faster wrong changes
Change failure rate	Fear of change
Mean time to recover	Hero culture

- **Sharing** = knowledge sharing, transparency and open communication

⚠ **Psychological safety** is the shared belief that it's safe to speak up, ask questions, admit mistakes and challenge ideas without fear of punishment (mistakes are expected and learning is continuous), but with the aim of improvement (share partial info early, ask for help and be honest)

⚠ **Leader** admits their own mistakes (*hero culture* is toxic)

⚠ **Software Delivery** is a **feedback system**!

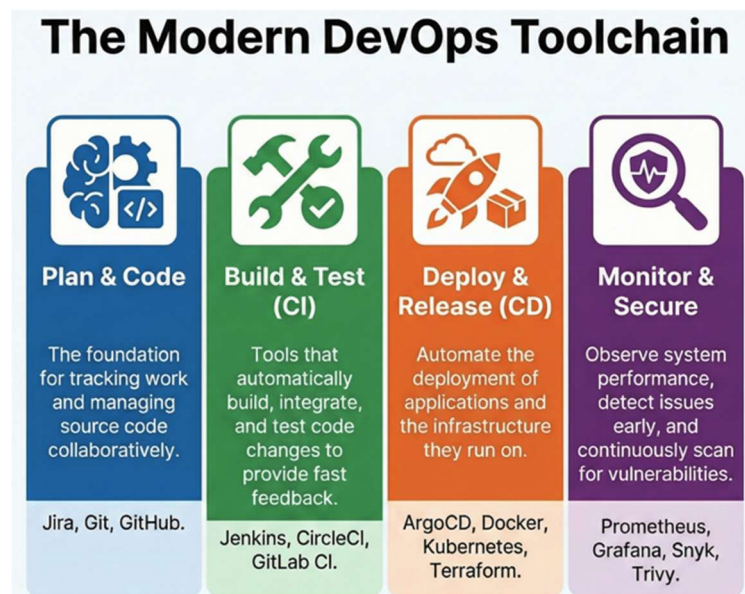
As said before, DevOps guarantees **faster deployments** (+ *fast feedback*); in particular we talk about:

- **Lead Time for changes (LT)** = interval between the commit of a change and its deployment in prod
- **Change Failure Rate (CFR)** = how often a change causes a prod incident (includes outages, rollbacks, hotfixes, defects)
- **Mean Time To Recovery (MTTR)** = time from incident start to service restored

Culture Assessment Tools are **Health checks** (self-assessments that help teams to rate how they feel and how they work [flow of work, quality & testing, collaboration, psychological safety & learning culture, autonomy, tech foundations]) and **Retrospectives** (continuous learning loops [as SoftEng II])

⚠ DevOps is necessary, but *not sufficient* (optimizes speed and feedback, but not for every form of correctness). It's not a universal solution, in fact not for: systems unchanging, systems safety-critical, physical/hw constraints. Also, if trust is low, ownership unclear or incentives misaligned, DevOps accelerates problems! (DevOps amplifies good and bad)

Continuous Integration (CI) = dev practice where **devs frequently commit changes** (at least daily) to the **main** branch (each commit triggers automated build, test and packaging in exe artifacts). Purpose is to **get fast feedback** (after change), **keep the main branch stable and healthy** and **fix errors immediately**. Toolchains and automation are how DevOps ideas are applied:



Based on product type, team size and maturity, CI can range from a **simple local build CI script** (to standardize how devs run “CI locally” before pushing and to test every module to check that everyone runs the same steps) to a fully automated cloud CI

⚠ To reduce the “I forgot to run tests” problem, you can also add a **pre-push hook** (`.git/hooks/pre-push` file) that invokes the local CI to trigger it automatically whenever a **git push** command is invoked [if the hook exits with a non-zero status, the push is aborted]

A Minimal **GitHub Actions CI** (1 job, whole repo) [added as `.github/workflows/ci.yml`] guarantees that every pull request and push on branch **main** is automatically verified (so that CI is centralized and visible, and **main** can be marked as **protected** [require this CI job to pass before merge happens]):

```
name: CI - Full Stack

on:
  pull_request:
    branches: [ "main" ]
```

```

push:
  branches: [ "main" ]

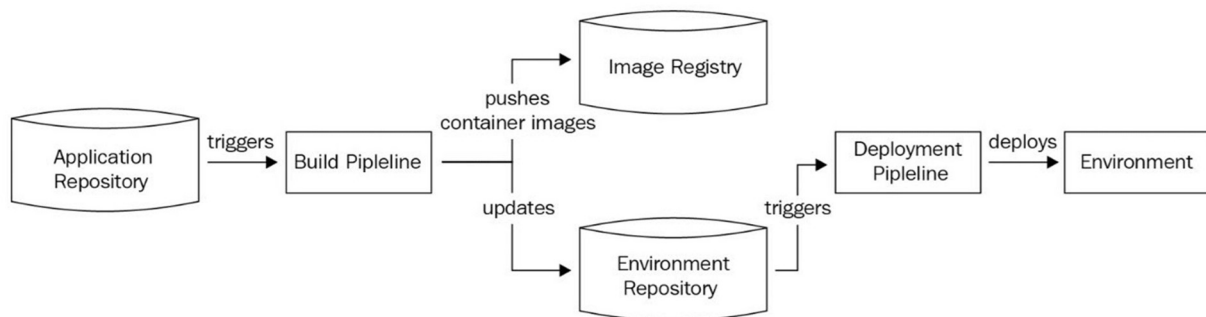
jobs:
  ci:
    runs-on: ubuntu-latest

    steps:
      # implement here suitable code
      # to run tests for each module

```

Obviously, after learning how to use GitHub Actions, you can also move to **Smarter CIs** (separate jobs+ path filters), CIs that build **Docker** images for backend, gateway and frontend, and CI with **integration checks** (to **test/validate the system before code reaches prod**).

Continuous Delivery/Deployment (CD) = ensure software is **always in a releasable state (deployable to prod at any time)** thanks to building on **CI output** (tested artifacts) and to deploy to **“prod-like” environments**. It’s not just tools, but requires also **organizational discipline & cultural change**. Steps:



Infrastructure includes all foundational systems **required for an app to run**: compute resources, networking, storage, platform services and their configurations. When system grow (multiple envs, microservices, cloud services), infrastructure becomes complex and difficult to manage. Common problems include **environment drift**, **manual errors**, **inconsistent deployments** and **scaling challenges**.

Infrastructure as Code (IaC) is the practice of **defining and managing infrastructure using version-controlled code instead of manual processes**. Infrastructure is written declaratively and stored in **Git**, making it **automated**, **reproducible**, **testable**, **reviewable** and **deployed across environments**: **all envs** (dev, staging, prod) are **generated from the same codebase**, **reducing drift and ensuring controlled differences** (in fact, changes are tracked via pull requests [*simple audit* and *simple rollbacks*]).

IaC enables **automated deployments** using tools as **Kubernetes manifests**, **Helm**, **Terraform** and **GitOps** engines. When integrated with CI/CD pipelines, IaC allows **validation**, **preview envs**, **automated deploy** and **security**. IaC also promotes **modularity** and **reusable patterns** (e.g. standard service deployments, ingress rules, autoscaling policies)

Automated Testing (AT) = core pillar of CI; enables **immediate feedback** for devs (every change pushed to GitHub triggers the CI pipeline) and **prevents regressions** (unit, integration, e2e and contract)

Monitoring and Logging = in monolithic systems it’s easy, but in microservice and cloud-native architectures no!

Monitoring collects and analyzes *numerical signals (metrics)* about **system health + performance** (requests/s, latency, error rates, CPU/memory usage, response time, gateway routing success)

Logging records **events**, **context** and **state** from applications and infrastructure (API request logs, error stack traces, gateway routing logs, business events).

Traces show **how a request flows through multiple services** (e.g. UserRequest → gateway → UserService → OrderService → NotificationService) and include service latency, downstream dependency times and location of failures. **Alerts** instead are rules that **detect abnormal behavior and notify** people or systems (e.g. API error rate > 5%).

⚠ So, monitoring and logging help detecting deployment issues early (thanks to metrics, logs, traces and dashboards) and providing safe and continuous deployment → so they lead to **system observability**.

Release Management = discipline that **controls how changes move from “ready” to “available to users”** in a predictable, safe and auditable way. It stays above CI (that builds and verifies changes)/CD (that deploys changes) because it decides **when, how and under what conditions the deployments become releases** (*release* = business-visible delivery of change made **available also to users** [vs] *deploy* = just put sw into an env) [**< risk** while **preserving delivery speed** (**> versioning** → release orchestration), but also **> communication** (everyone should know changes) and **auditability**]

Area	CI	CD	Release Management
<i>Purpose</i>	Verify code	Deploy code	Control exposure
<i>Trigger</i>	Code change	Pipeline event	Intentional decision
<i>Scope</i>	Technical	Technical	Technical + business
<i>Focus</i>	Correctness	Automation	Risk, timing, communication
<i>Owner</i>	Engineering	Engineering	Product + Engineering

Release strategies:

- **Blue-Green** = **2 envs** (Blue = live, Green = new); switch traffic instantly; fast rollback
- **Canary** = **release to small % of users**, monitor metrics, gradually increase traffic
- **Progressive Delivery** = **Canary + automation + SLOs**; release pauses or rollbacks automatically
- **Feature-flag-driven** = code is deployed dark: features are **enabled via config or flags**; rollback = **toggle flag** (not redeploy)

DevSecOps = is an **evolution of DevOps that treats security as a shared+continuous responsibility across entire sw delivery lifecycle** (not a separate phase or separate team), so as a sort of **Security By Design** (**security addressed as early as possible**). Obviously, there is always a security team, but security is a **shared responsibilities** and security is **automated** (static code analysis, vulnerability scanning, container image scanning, IaC validation...). Security rules are **expressed as code** (policies, configurations, access controls...) and processes make security transparent and testable (version control, CI validation...)

⚠ DevSecOps is integrated in development, CI, IaC, CD, release, runtime and operations

⚠ **Chaos Engineering** = proactively testing system resilience by introducing intentional controlled failures in prod env (e.g. simulating network latency)

The **pay-as-you-go cost model provided by cloud computing** allows to move from large upfront *CapEx* (Capital Expenditure) to a flexible *OpEx* (Operation Expenditure)

Governance = decision-making, ownership and control; **Compliance** = verification and evidence.

Together, they define how organizations control, guide and verify sw delivery (safe, auditable...).

Principles are:

- *Policy as Code*
- *Auditability & Traceability*
- *Access governance* (Role-Based Access Control)
- *Standardized Baselines (IaC)*

⚠ CI/CD Tools (**Jenkins**, GitLab CI...), IaC Tools (**Terraform**...), Container & Orchestration Tools (**Docker**, **Kubernetes**, **Helm**), Monitoring Tools (**Prometheus**, **Grafana**...), Security Tools (**Snyk**, **Aqua**...), Collaboration & Communication Tools (**MS Teams**, **Slack**)

3) Execution Environments

Each component of a distributed app needs to be **hosted** somewhere to be executed (runtime env should provide access to all resources needed by the component [CPU, GPU, mem, network...]). Depending on the phase of lifecycle, the resources provided by runtime env can vary; in fact, along its **lifecycle**, a subsystem may be deployed in different **EXECUTION ENVIRONMENTS** (each with different purposes/tools):

- **Development** (write, debug and test code **locally** [on local machines/light servers]) [e.g. *VSCode*]
- **Integration** (**combine and validate with other modules**; integrate with other members of the team and see if components run together) [e.g. *Jenkins, GitHub Actions, GitLab*]
- **Testing** (various **testing scenario** to identify bugs, regressions and bottlenecks) [e.g. Test → *Selenium, Postman*; Load → *Locust, Gatling*; Issue → *YouTrack*]
- **Staging** (**simulate prod** for final validation; used for User Acceptance Testing [UAT]) [e.g. IaC tools → *Terraform, AWS CloudFormation*; Monitoring → *Prometheus, Loki, Tempo, Grafana*; UAT]
- **Production** (**serve live users**: fully optimized for performance, scalability and security) [e.g. App Performance Monitoring (APM) → *New Relic, Datadog*; Logging]
- **Disaster recovery** (**standby for emergencies**)

To guarantee **consistent behavior** across these envs, actual execution should be **shielded** as much as possible from their peculiarities, but at same time **cost** should be kept under control (using **lightweight envs if possible**) and also **data privacy** (**anonymizing or masking sensitive data** in any stage other than production). As said before, the **execution environments COMPONENTS** are:

- Computing resources = physical machine (hw) or vm running in a larger server; each machine has CPUs, RAM and optional dedicated hw. App subsystems may run inside these machines (OS) or inside a container
- Networking = allowing computing resources to communicate with external world (or inside VPN) + eventual firewalls
- Storage (shared disks, AWS S3 [objects storage] or DBMS as a Service)

⚠ As a bare minimum, execution env may consist of a single PC (with its own resources) or a cloud-based env (e.g. *AWS, Azure* or *Google Cloud*) [more you go to cloud, < cost of hw, but > pay-as-you-go]

In **cloud-native envs**, you can manage and provision infrastructure needed using **machine-readable config files** (e.g. *YAML, JSON, Gists*). This configuration code can be **DECLARATIVE** (specifies the **desired state of the infrastructure**, and the **IaC tools** ensures that the actual state matches it [e.g. *Azure Resource Manager, Terraform, AWS CloudFormation*]) or **IMPERATIVE** (specifies how to achieve the desired state, telling the steps to create/modify the infrastructure [e.g. *Ansible, Chef, Puppet*]). These configs are also stored in version control systems (Git) tracking the changes (and possibly roll back).

⚠ **IaC main tasks:** define infrastructure, provision resources, monitor + update, free resources; **IaC benefits:** speed, efficiency, consistency, standardization, scalability, version control, collaboration, disaster recovery

So, as said before, given the availability of computing resources inside an execution env, the sub-systems of a distributed app must be allocated and the execution can happen in different contexts:

- **Locally** → in the OS of a single node; *easy* and *performant*, but *poor isolation* between processes (all processes share same file system, all processes share network stack, shared libraries are problem, OS maintain 1 view of users/permissions) + *expensive* and *< portable* and complex maintenance
- **Docker Container** → a **CONTAINER** is a lightweight, standalone and executable unit of sw that packages a subsystem along with all its dependencies, libraries and configurations required to run reliably across different envs. Containers run in **OS-level isolated envs** (**virtualized** [thanks to **namespaces** [isolation], **cgroups** [resource management], **union filesystems** [fast sharing between containers]; they **share the kernel** [differently from VM]) and are ephemeral (stateless and tmp) [if you want to maintain persistent data, use the option **-v /custom/mount:/var/lib/postgresql/data**].

A **container runtime** (e.g. **Docker**) is the manager who creates, runs and manages containers in the host machine (controlled via **REST API**). Containers are created from **images** (read-only layered files that contains everything to run a sw subsystem), which can be created with **Dockerfile**:

```
# Use a base image
FROM python:3.9-slim

# Set the working directory
WORKDIR /app

# Copy application code into the image
COPY . .

# Install dependencies
RUN pip install -r requirements.txt

# Specify the command to run the application
CMD ["python", "app.py"]
```

Once built, container images can be stored in a **registry** (as a *repo* from which can pull [e.g. Docker Hub]). Each container has its own IP address (by default a private network not accessible from host, but from other containers; in fact, if the host has to access the container, it must expose a port [**docker run -d -p 3128:3128 -e POSTGRES_PASSWORD=mysecretpassword postgres**]) and network stack. If you have to launch more containers at the same time, use **docker-compose.yml** (and use **docker compose up/down -d** to launch and stop the containers). **Example:**

services:

```
# PostgreSQL Database
postgres:
  image: postgres:14
  container_name: postgres-db
  environment:
    POSTGRES_USER: myuser
    POSTGRES_PASSWORD: mypassword
    POSTGRES_DB: mydatabase
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
```

```

networks:
  - app-network
# Spring Boot Application
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: spring-boot-app
  environment:
    SPRING_DATASOURCE_USERNAME: myuser
    SPRING_DATASOURCE_PASSWORD: mypassword
  ports:
    - "8080:8080"
  depends_on:
    - postgres
  networks:
    - app-network

volumes:
  postgres_data:

networks:
  app-network:
    driver: bridge

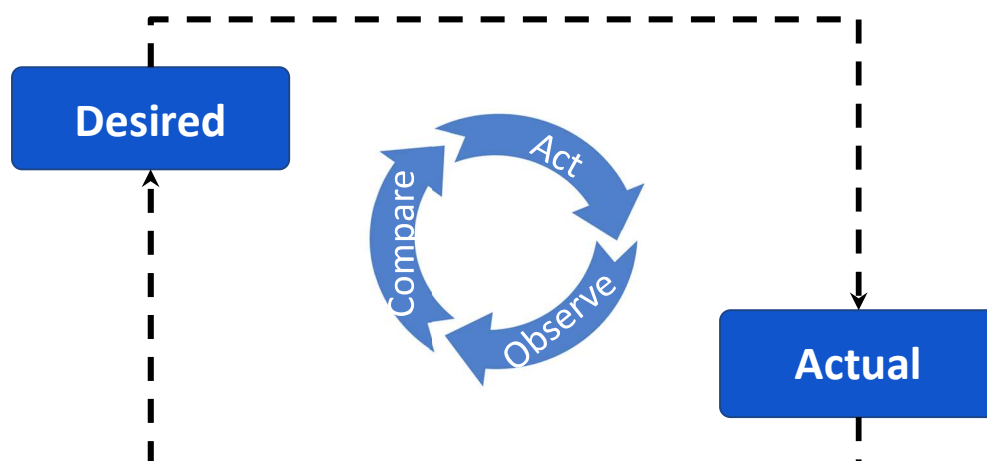
```

⚠ **Container pros:** isolation, reproducibility, scalability, lightweight, ease of deployment; **Container cons:** operational overhead, limited orchestration, resource management, network complexity

⚠ **Docker compose** has limits as systems become distributed, dynamic and prod grade because: **no** built-in **self healing** + **single host** (if host dies, all go down + no scalable), **no auto-scaling**, weak **service discovery** model, **no** built-in config and **secret** management, **min observability**

⚠ Since all containers run inside the local host, resources requested (e.g. # of processes that form a distributed app) may become a problem: **ORCHESTRATION** tools (as K8s) aim to solve this (**container orchestrator** = automatically manages containerized apps across multiple hosts [*deploy, scale, monitor, network, integration*])

- **Kubernetes (K8s) Cluster** → span across several nodes. Nodes in a cluster can play 2 different roles: **master** nodes (**control plane**: manage cluster and decide what should happen [not how fast or where exactly]) and **worker** nodes (machines where **app workloads** [**pods**] run). A Kubernetes cluster is a **horizontally-scalable distributed exec system** that operates in a declarative (**YAML** → **desired state**) **control-loop-driven** way (K8s observes what exists [**current state**] and reconcile it with desired state [**this is the loop**]). Both worker nodes and workloads running on them are continuously probed by the control plane:

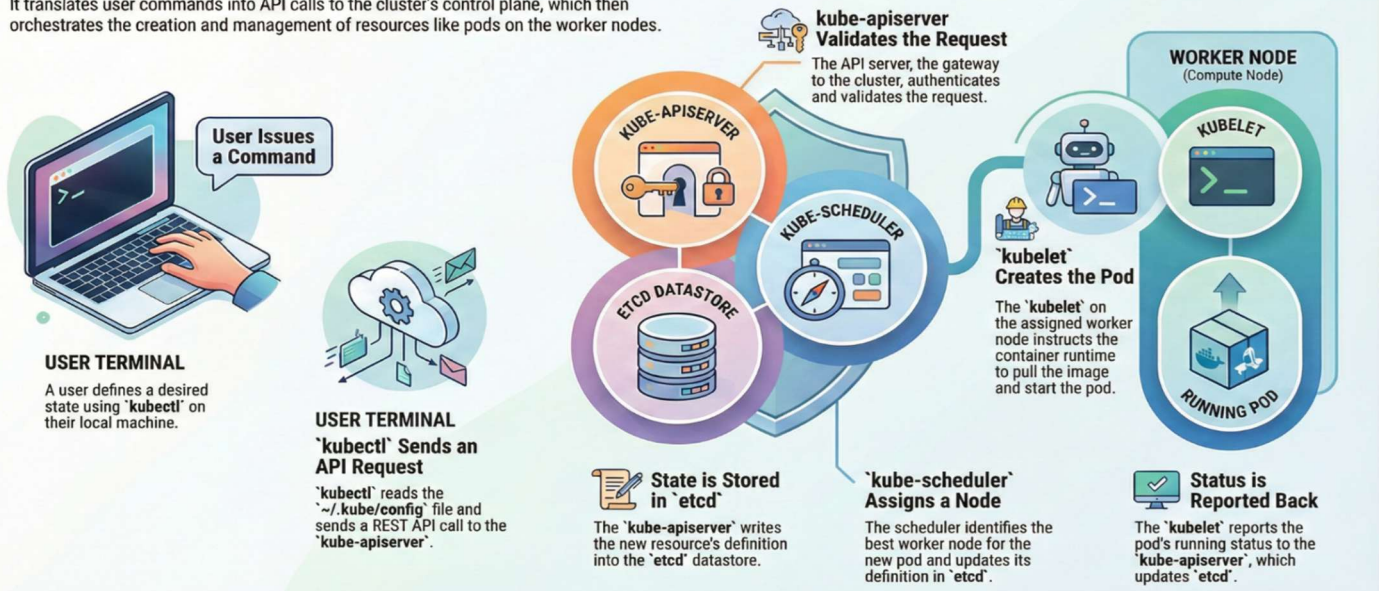


Kubernetes is **API-driven**: desired state is described via a **set of API objects managed via a RESTful interface** provided by master nodes (supporting CRUD operations). There isn't a command "*run this on that node*". Control plane **doesn't care who makes the API call** (human, CI/CD pipeline, controller, operator).

Kubernetes CLI is **kubectl** (a command-line client that talk directly to the API server; it authN with the API server that run inside master nodes [by default credentials are kept in `~/.kube/config` file of the client machine])

The Journey of a `kubectl` Command

`kubectl` is the primary command-line tool for interacting with a Kubernetes cluster. It translates user commands into API calls to the cluster's control plane, which then orchestrates the creation and management of resources like pods on the worker nodes.



As said before, K8s runs **workloads in units** named **pods** (each pod consists of 1 or more containers, executed in 1 of the worker nodes), each with a **UID** and an **ephemeral IP address**. Control plane periodically assess health state of each pod (*unhealthy pods* are killed by sending a **SIGTERM** signal to all its containers [**containers inside a pod live and die together**]) [pod = **temporary replaceable exec unit that groups containers with shared network and storage**]. Describing pods:

```
apiVersion: v1
kind: Pod
metadata:
  name: nodejs-app-pod
  labels:
    app: nodejs-app
spec:
  terminationGracePeriodSeconds: 30
  containers:
    - name: nodejs-app-container
      image: my-node-app:1.2.15
      imagePullPolicy: IfNotPresent

  ports:
    - name: http
      containerPort: 3000
  envFrom:
    - configMapRef:
        name: node-app-config

  command: ["node"]
  args: ["/usr/src/app/app.js"]

# ---- Resource management ----
```

```

resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
# ---- Health checking ----
# give time to boot(avoid early restarts)
startupProbe:
  httpGet:
    path: /healthz
    port: http
  failureThreshold: 30 # 30*2s
  periodSeconds: 2
  timeoutSeconds: 1

# Readiness: can it receive traffic?
readinessProbe:
  httpGet:
    path: /readyz
    port: http
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 1
  failureThreshold: 3
  successThreshold: 1

# Liveness: is the process still healthy,
# or should it be restarted?
livenessProbe:
  httpGet:
    path: /healthz
    port: http
  initialDelaySeconds: 15
  periodSeconds: 10
  timeoutSeconds: 1
  failureThreshold: 3

```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: node-app-config
data:
  NODE_ENV: production
  APP_PORT: "3000"

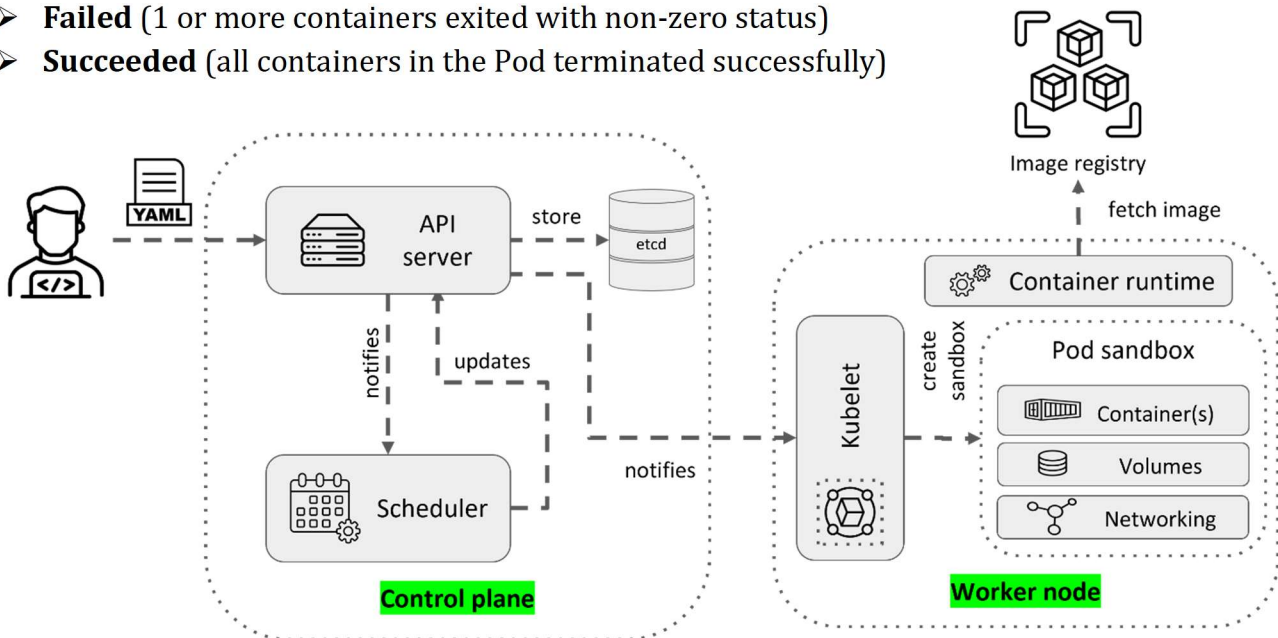
```

To be monitored, apps should expose probe **endpoints** (in the previous example: `GET /healthz` → return 200 when process is alive; `GET /readyz` → return 200 only when it's ready to serve). App resources (requests/limits) must be empirically validated (tuned based on real usage: start small, measure, adjust) [as you can see above, configuration is **split in different YAML** to be more *modular*]

Pod Lifecycle → possible states:

- **Unknown** (e.g. during termination, if worker node is unreachable)
- **Pending** (until its containers started; Pod description is submitted to API server, scheduler chooses a node, container startup ↓)
- **Running** (at least 1 container is running in the Pod)
- **Terminating** (termination request [`SIGTERM`* sent to all containers]; terminationGracePeriod Seconds countdown starts → containers shut down gracefully → `SIGKILL` is sent if time expires → Pod is removed)

- **Failed** (1 or more containers exited with non-zero status)
- **Succeeded** (all containers in the Pod terminated successfully)



Pods (as containers) lack self-healing, horizontal scalability, persistency. So, to this purpose, K8s provides **Controllers**, which are active processes that **implement reconciliation loop**. **Distributed systems are unreliable** (pods crash, node disappear, network partitions, human errors), so controller exists to ensure that failures are expected, repair is automatic, correctness is enforced.

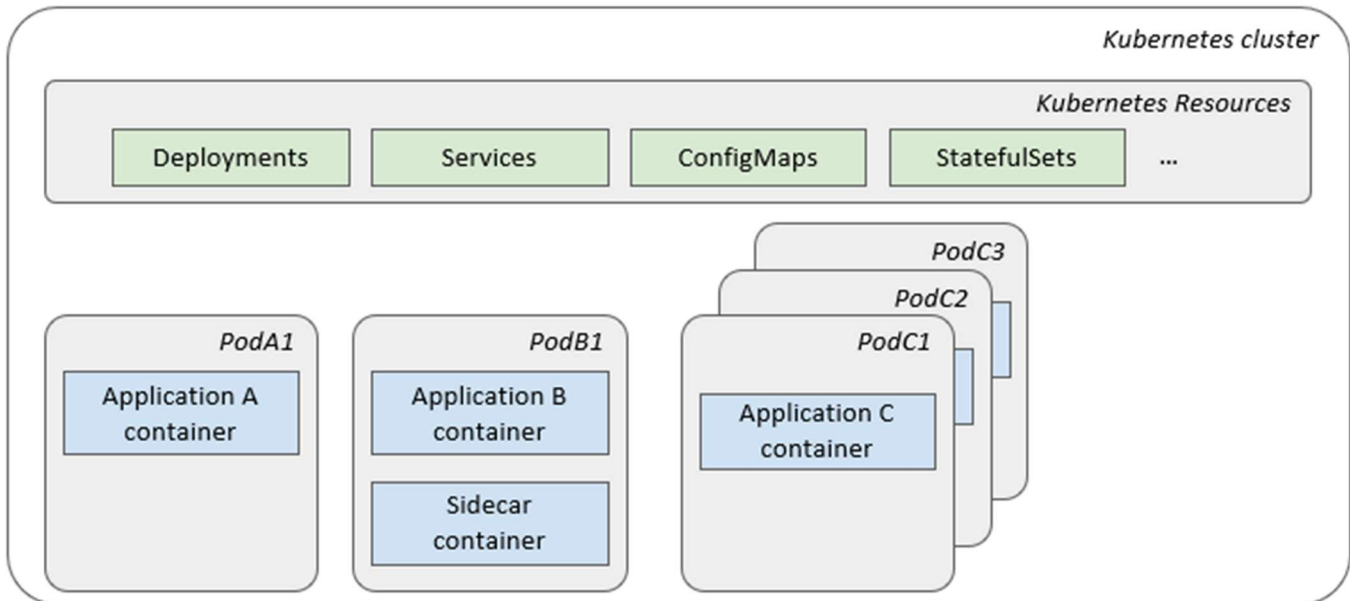
In order to declaratively specify the expected behavior of a given workload, K8s introduces a set of **resources** (versioned structured objects stored in **etcd** database to provide a persistent declarative intent for workloads). Resources can be created, updated, deleted via API calls, but do nothing by themselves. Default resources are **Pods, Deployments, Services, ConfigMaps, Nodes...**, but **custom resources can be defined** (API is extensible). So, **controllers are running processes inside control plane, which watch resources in the etcd database to continuously reconcile desired-current state**.

⚠ Separation of resources and controllers provides resilience, decoupling and declarative safety

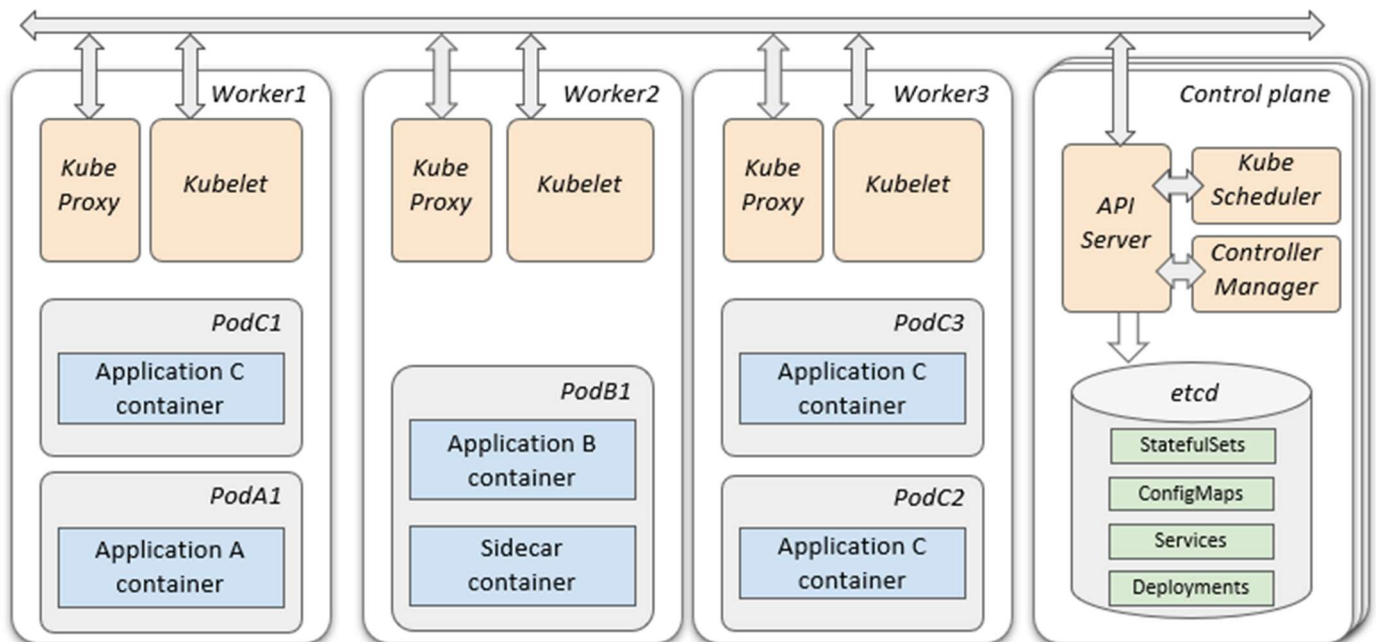
Let's look at the **Kubernetes zoo**:

- **Deployment** = manages the lifecycle of pods and ensures the desired state is maintained
- **Service** = provides a stable network endpoint to access a set of pods (even if they are created, destroyed or replaced). Types of services:
 - **ClusterIP** → exposes the service within the cluster
 - **NodePort** → exposes the service on a static port on each cluster node
 - **LoadBalancer** → integrates with cloud providers to expose the service via an external load balancer
 - **ExternalName** → maps a service to an external DNS name
- **StatefulSet** = used to manage stateful applications where each pod has a unique identity (pods belonging to a stateful set are named *sequentially* [e.g. my-app-0, my-app-1]) and persistent storage (through **PersistentVolumeClaims** [PVCs])
- **ResourceMap** = describes operational requirements of containers inside pods (e.g. CPU, mem)
- **DaemonSet** = ensures that a copy of a specific pod runs on all nodes of a cluster
- **ConfigMap** = object used to **store non-sensitive config data in key-value pairs** (it decouples app configuration from app code, enabling apps to be more flexible and environment-agnostic)
- **Secret** = object designed to store and manage sensitive info (as pwd, API keys, SSH keys, TLS certificates) and make it available to apps running in Pods (injected as env variables, files or accessed programmatically)

Kubernetes logical view



Kubernetes physical view



⚠ **K8s pros:** scalability, availability, orchestration, multi-cloud, compatibility, declarative config, resource efficiency, extensibility; **K8s cons:** complexity, operational overhead, resource usage, overkill for small apps

L01) GitHub Actions

A **Workflow** is the overarching automated process defined in a **YAML** file and is **triggered by an Event** (e.g. a code commit). A **Job** is a **set of Steps** executed on same machine (*Runner*); jobs run in parallel by default. A **Step** is a **single task**: it can run a shell command or use a pre-built **Action**. An **Action** is a **reusable standalone command** (e.g. checkout-code).

Each **YAML** is a **single workflow** and you can have many workflows (in `.github/workflows`, otherwise Github's servers will ignore it). To specify the Event trigger, every workflow starts with **on: event** (event can be **push**, **pull_request** or **workflow_dispatch** [which allows you to trigger the pipeline manually from Github UI]):

```
name: Node.js CI

# Trigger on push OR pull request to main
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

# Bonus: allow manual triggering from UI
workflow_dispatch:
```

Jobs define the **env**: since CI/CD is declarative, you define what env you need, not how to provision it (note that multiple jobs will run in parallel unless you specify dependencies using **needs**):

```
jobs:
  build-and-test:
    # The OS of the runner machine
    runs-on: ubuntu-latest

    steps:
      # Step definitions go here ...
      - name: Checkout code
        uses: actions/checkout@v4
```

Inside a **Job**, **Steps** are executed sequentially from top to bottom. **uses** is used to call a **pre-existing Action** (a packaged script); **run** is used to execute a custom **Shell Command** on runner. Always use **name** to give step a readable title in logs:

```
steps:
  # 1. Action: Download your repo code
  - name: Checkout code
    uses: actions/checkout@v4

  # 2. Action: Set up Node.js
  - name: Setup Node
    uses: actions/setup-node@v4
    with:
      node-version: '20'

  # 3. Shell: Run custom commands
  - name: Install dependencies and Test
    run: |
      npm ci
      npm test
```

⚠ **Don't reinvent the wheel**: use GitHub Marketplace to search for already-built Actions (e.g. Docker build, AWS deploy, linters...), use starter workflows (GitHub suggests templates based on your codebase in the "Actions" tab) or read already-built Actions in `.github/workflows/folder` of popular repositories

Every Job runs on a fresh isolated vm; if Job A compiles your code, Job B can't see the compiled files → solution is to use **Artifacts** (Job A uploads output files to GitHub storage and Job B downloads them to use for deployment or test):

```
steps:
  # ... (build your app into a ./dist folder)

  - name: Save Build Output
    uses: actions/upload-artifact@v4
    with:
      name: my-compiled-app
      path: ./dist

  # In a DIFFERENT job:
  - name: Download Build Output
    uses: actions/download-artifact@v4
    with:
      name: my-compiled-app
      path: ./deploy-folder
```

Pipelines have access to our code and infrastructure, so need **Security**: never hardcode secrets (pwd and keys belong in **GitHub Secrets**, injected via the **env** context) and use **least privilege principle** (by default workflows have broad write access; you need to restrict what the workflow token can do using the **permissions** block):

```
# Restrict token permissions explicitly
permissions:
  contents: read # Can read code, cannot push

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh
      env:
        # Inject secrets safely at runtime
        API_KEY: ${ secrets.PROD_API_KEY
    }}
```

4) Distributed Systems

A **Distributed System** is composed of 2 or more interconnected subsystems that cooperate to achieve a common goal (*interconnection* = **network**; *communication* = **msg passing**). Driving forces of DS are:

- **Composability** (complex behavior can be achieved splitting in smaller and loosely connected parts)
- **Scalability** (adding more machines allows to deal with larger problems and # of users)
- **Availability** (redundancy to support partial failure)
- **Resource sharing**
- **Reusability**
- **Cost efficiency**
- **Compliance** and data localization

So, a DS is characterized by a set of attributes interdependent (↑ one = ↓ another); **3 theorems** affect the relationships between these properties:

1. **CAP theorem** = a DS can't simultaneously provide Consistency, Availability and Partition tolerance
2. **PACELC theorem** = in presence of a network partition, we must choose between Consistency and Availability; without a network partition, a trade-off emerges between Latency and Consistency
3. **Universal Scalability Law** (*) = how a system's throughput changes as the # of resources (e.g. servers, CPUs or users) increases

Let's see these properties in detail:

- **CONSISTENCY** = guarantee that all nodes in the system have a consistent view of the data, even in the presence of concurrent updates, failures, or delays. A system has:
 - o **strong** consistency → all nodes in DS see the same data at the same time
 - o **causal** consistency → order of operations that are causally related is preserved
 - o **eventual** consistency → all nodes in DS will converge to the same state, if no new updates are made (weaker) [see *Distributed Systems* course to see more details on consistency]
- **AVAILABILITY** = ability of a system to respond to requests from users or other systems in a timely manner, even in the presence of failures. Key aspects of availability are:
 - o **uptime** = % of time when DS is operational
 - o **timely response** = in order to be considered available, DS must respond to user requests within an acceptable time frame, regardless of failures and workload
 - o **fault tolerance** = tolerate and recover from sw/hw failures without big service interruption
 - o **graceful degradation** = in cases of partial failure or resource constraints, the system may reduce functionality but remain available in a limited capacity
- **PARTITION TOLERANCE** = continue operating correctly even when communication between some of its components is disrupted due to a network partition (in presence of a partition, a system must choose to prioritize Consistency or Availability)
- **LATENCY** = time for a request to travel from the source to its destination and for a corresponding response to be received (**DELAY** = propagation delay + transmission time + processing time + queuing time) [inversely related to throughput]
- **SCALABILITY** = handle an increasing amount of work or accommodate growth in demand by adding resources (elasticity, performance consistency, resource efficiency, predictable behavior)
- **RELIABILITY** = perform its intended functions correctly and consistently over time, even in the presence of failures (correctness, fault tolerance, error recovery)
- **FAULT TOLERANCE** = continue operating correctly and providing its intended services even in the presence of faults or failures in one or more of its components (resilience, redundancy, error detection + recovery, graceful degradation)

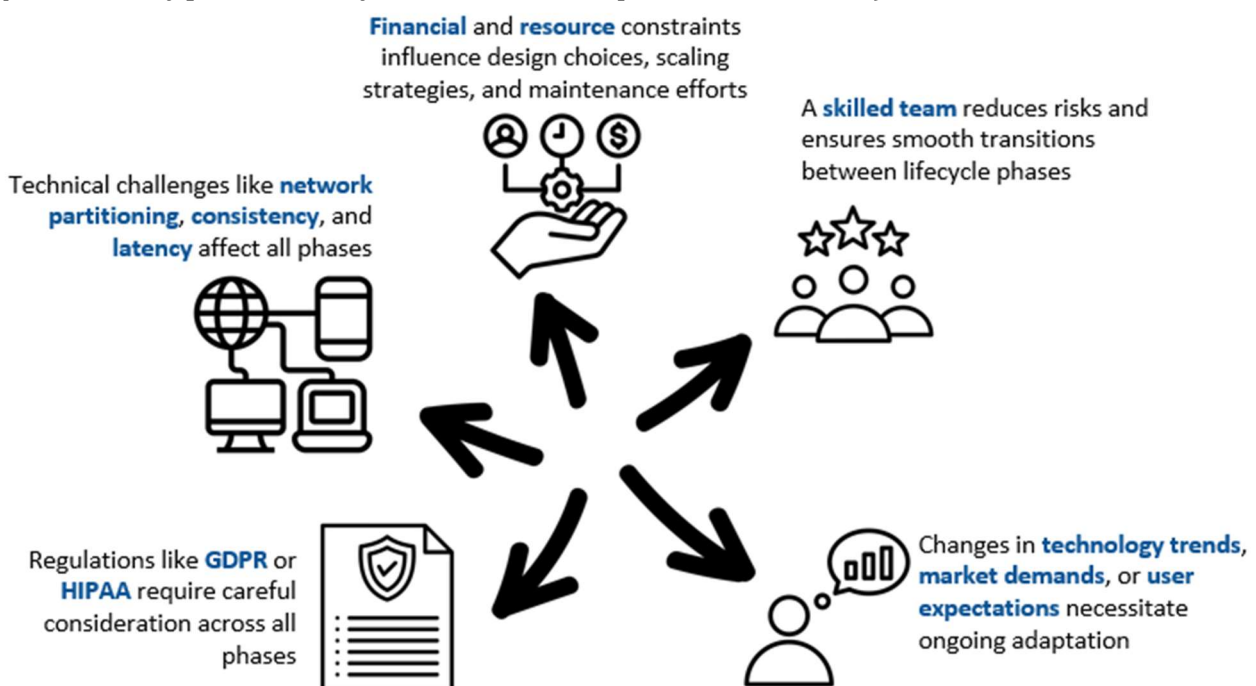
A DS can be understood as an **emergent behavior** arising from the interactions and coordination of its individual components: **each component** interacts with a limited set of neighbors/resources and operates with local knowledge and specific responsibilities, but the overall behavior and functionality of the system result from their **collective interactions** (mechanisms that drive emergent behavior are *decentralized coordination, replication and redundancy, feedback loops, dynamic adaptation, partitioning and independence*).

⚠ Maintaining emergent behavior in a DS, while allowing subsystems to evolve, **requires careful design and governance** (emphasizing modularity + loose coupling + stable interfaces to isolate subsystems changes; use observability + testing + incremental updates to validate system-wide effects; design + redundancy to ensure resilience)

To preserve the overall design goal of the system, a clear **GOVERNANCE for subsystems evolution** must be established: individual sw artifact codebases (e.g. components) must be managed by a **Version Control System** (e.g. **Git**) and you need to enforce a structured branching model to manages changes and releases (adopting workflows like **GitFlow** or **Trunk-Based development**, enforce code reviews before merging, automate testing and validation as part of the CI/CD pipeline) [*contract testing* = validating that client and server are developed with I/O understandable between each other]

You need to:

- use **semantic versioning** to communicate the impact of changes (**MAJOR** = breaking changes, **MINOR** = backward-compatible new features, **PATCH** = backward-compatible bug fixes);
- maintain a **version compatibility table** for interacting subsystems (to **document** which versions of services are **compatible** and to **automate compatibility testing**)
- enforce **well-defined API** (e.g. **Swagger/OpenAPI**) contracts to decouple subsystems (using **versioned APIs** [e.g. v1/api, v2/api...], using **API gateways** [to enforce access rules and manage traffic across multiple API versions] and **deprecate APIs gracefully** [by supporting older versions for a transition period])
- **minimize** and **explicitly manage dependencies** between subsystems (using **dependency injection** to isolate and dynamically manage dependencies; recording **approved libraries**)
- **manage database and data model changes** (using **schema migration tools** [e.g. *Flyway, Liquibase*] to apply incremental changes; maintaining **versioned schemas**)
- adopt well defined **deployment strategies**:
 - o **blue/green** deployment → new versions are deployed alongside old ones (parallel)
 - o **canary** releases → gradually roll out changes to a small subset of users (monitoring issues)
 - o **rolling** updates → incrementally update instances to the new version without downtime
- ensure that **changes don't break existing subsystems** (use **feature flags** to test and roll out new features incrementally; implement **fallbacks** to handle unsupported features gracefully)
- **validate changes in isolation** and in the context of the entire system (contract testing, integration tests and e2e tests)
- **monitor subsystem behavior** to detect issues early (integrate distributed tracing [e.g. *OpenTelemetry*], monitor key metrics and set up automated alerts)

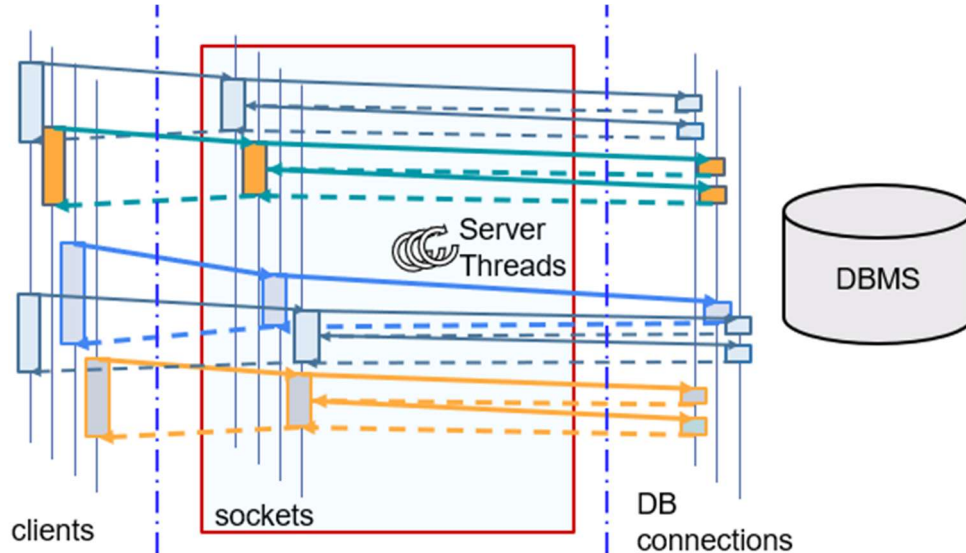


So, **CHALLENGES** of a DS are of different kinds:

- **tech** → **complexity, partial failures, lack of a global clock, security**
- **design phase** → domain **partitioning, consistency vs availability, fault tolerance + redundancy, scalability** planning
- **development phase** → **state** management, **concurrency** control, **testing** for failures
- **deployment phase** → deployment **strategies, configuration** management, **service discovery**, distributed **logging** and **monitoring**
- **runtime phase** → **scaling, load balancing, failure recovery, consistency** and **synchronization**

- **maintenance phase** → upgrades and **patching, data rebalancing, observability**
- **decommissioning phase** → **graceful node shutdown**, resource **cleanup, documentation** updates

Talking about **performance and scaling**, an enterprise application must process data as fast as possible. The 2 most important metrics to evaluate its performances are **latency** (overall **time** needed to complete a transaction [**delay** between when a request is sent and when the response is received]) and **throughput** (**rate** of completing incoming load [**# of transactions** in a given time interval]):



To evaluate the **response time**, a statistical approach is used: **on a single client**, *count* transactions are invoked in a loop and the overall time t is computed; then:

$$avg_{t_{exec}} = \frac{t}{count} \quad \text{and} \quad throughput_{singleClient} = X(1) = \frac{count}{t}$$

So in a distributed system, there can be concurrent transactions coming from different clients at same time, so $X(N) = X(1) * C(N)$ (with $C(N)$ = relative throughput gain due to concurrent client support).

If the system is **perfectly scalable**, **increasing # of clients** would yield a proportional **increase in throughput**. **Universal Scalability Law (USL *)** says that the system throughput can be approximated by:

$$X(N) = \frac{N * \gamma}{1 + \alpha * (N - 1) + \beta * N * (N - 1)} \quad \text{with}$$

$$\left\{ \begin{array}{l} \alpha = \text{internal resource contention (due to waiting or queueing for shared resources)} \\ \beta = \text{coherency cost (due to the delay for data to become consistent)} \\ \gamma = \text{scalable part of the system} \end{array} \right.$$

The **need for system coherency** cause the β parameter to be not null (but in turn, sets a limit beyond which the throughput will not increase any more). Also, in order **to reduce contention**, a lot of resources need to be poured in the system (e.g. reason why, in a blocking system, hundreds of threads are allocated and a corresponding amount of memory need to be provided).

⚠ You can use tools like Apache bench (**ab**) to provide measurements of behavior of a web system; or you can use **us14j** library to estimated USL parameters of a system ↓


```
import com.codahale.us14j.*
fun main(args: Array<String>) { //put here your measurements with ab (thread# to rps)
    val values = listOf( (1 to 2228.54), (2 to 4650.24), (3 to 5947.23),
                        (4 to 6037.80), (5 to 7393.00), (6 to 7687.61) )
    val model = Model.build(values.asIterable()
        .map { Measurement.ofConcurrency().andThroughput(
            it.first.toDouble(), it.second.toDouble()) }
        .toMutableList())
    val f = {i:Int -> println("At $i concurrent clients, expect ${
        model.throughputAtConcurrency(i.toDouble()) } req/sec")}
}
f(10)           // log10(10) = 1
f(32)           // log10(32) = 1.5
f(100)          // log10(100) = 2
f(316)          // log10(316) = 2.5
f(1000)         // log10(1000) = 3
}
```

⚠ **Fallacies of DS:** **network** is not reliable (API calls in fact can also not return or be timed out), not secure and heterogenous, **latency** is not zero, **bandwidth** is not infinite, **topology** changes (🔄), there are **many administrators**, **transport** cost is not zero. The DS must be designed to know that something can go wrong; also, it's difficult to deal with these problems manually, so we use **orchestrators** (e.g. *Kubernetes*) and **architectural patterns** (e.g. *Circuit Breaker*)

⚠ **Reliable app properties** = **idempotence** (e.g. POST in REST API to address network problems [maybe retransmissions]), **immutability**, **location independence** (🔄 e.g. DMBS auto-increment IDs tend to break this rule), **versioning** (since systems evolve across time, it is possible that **multiple versions** of the application exists **at different locations** → so need to be versioned)

5) Kubernetes



As system grows, manual coordination becomes fragile, so automation works better against failure. **Kubernetes** is a control system that stores **desired state**, observes **current state** and runs **continuous reconciliation between them** (replacing failed components automatically [don't prevent failure, but makes failure survivable]). Cloud-native systems are built around failure:

What will fail

- Pods
- Nodes
- Networks
- Deployments
- Humans

Failure is assumed, not exceptional

How the system responds

- Desired state is declared
- Controllers enforce invariants
- Components are replaced
- Recovery is automated

The system is always slightly wrong - and always correcting itself

What this means

Applications must be:

- Restartable
- Stateless or externally stateful
- Idempotent
- Observable
- Honest about health

Apps that assume stability will fail in Kubernetes

If a Kubernetes cluster were a large construction project:

- **Control Plane** = site office
- **API Server** (*kube-apiserver*) = front desk where all permits are filed; **central gateway** that authenticates requests, enforces policy, persists desired state and coordinates all control-plane

activity in K8s ([only endpoint](#) for all cluster operations, exposing a RESTful interface). It's stateless and delegates data storage to etcd. It supports [multiple authN](#) mechanisms and enforces [authZ](#)

After a request has been authorized, it is intercepted by [admission controllers](#) (that enforce policy, safety and automation before anything becomes persistent in the cluster) which can be of 2 types:

- [mutating](#) admission controllers = can **change** incoming object, adding resource requests/limits, inject sidecar containers for service meshes, add labels/annotations, normalize fields. *Run first*
- [validating](#) admission controllers = can **accept or reject**, but not modify incoming objects

If a request passes the admission controller step, its schema is [validated](#) (invalid YAML never enters the system, protecting the cluster from corruption) and after passing all checks, the incoming object is [stored](#) in etcd.

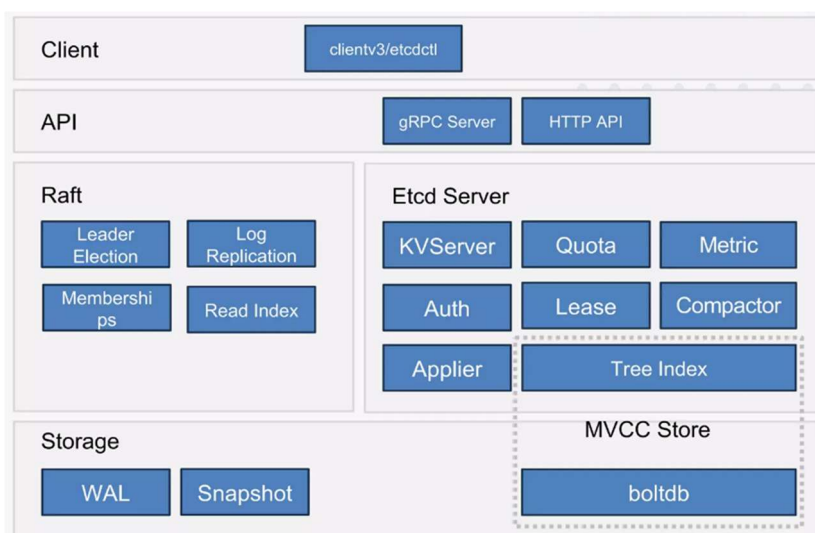
⚠ Controllers do not poll randomly, they **watch** the apiserver (when a relevant fact happens, the apiserver *streams changes* efficiently to controllers)

API server is the [convergence point](#) (all intent flows in, all state flows out) and obviously if it's down, workload keep running but no changes can be made (controllers can't reconcile new intent).

Since it's stateless by design, [multiple API servers](#) can run *concurrently* (horizontally scalable)

- [etcd](#) = blueprint vault. It's a consistent [distributed key-value store](#) that holds the [entire desired](#) and [observed state](#) of a Kubernetes cluster. *Direct interaction* with etcd is restricted solely to API server.

Architecture:



Principles are strong consistency, single source of truth and key-value model (everything is stored as `/path/to/resource` → `serialized_object`).

etcd runs as a [cluster](#) (typically 3 or 5 members [odd number for quorum] → [fault tolerance](#): 3 nodes tolerate 1 failure, 5 nodes tolerate 2 failures) [if quorum is lost, cluster becomes *read-only* or *unavailable* (K8s can't accept changes)].

A [watch](#) is a [continuous ordered stream of changes from the API server, backed by etcd's revision history](#) (that lets client observe state transitions as they happen → every write to etcd is assigned a monotonically increasing [version number](#) [totally ordered across the cluster; etcd periodically compacts history removing older revisions]). API server internally maintains a watch cache. Steps:

1. controllers start making a [LIST](#) request for a specific kind of object (e.g. `LIST /api/v1/pods`)
2. API server returns full current state including a [resourceVersion](#) field
3. controller opens a [WATCH](#) (e.g. `WATCH /api/v1/pods?resourceVersion=107`) and API server keeps the connection open and streams events as they occur
4. each change produces an [event](#) `{"type": "MODIFIED", "object": {...}, "resourceVersion": "108"}`

5. controllers don't trust events alone → periodically they **re-LIST** all objects (reconcile desired vs actual state) [to protect against missed events, transient errors, cache inconsistencies] → in fact, controllers are level-based and not event-based

⚠ **Watches may break** due to network issues, API server restarts and resourceVersion compaction → when a watch connection breaks, controllers retry (if revision too old, a full LIST request is issued)

- **Scheduler** = **manager** deciding which crane (*node*) handles which load. It assigns newly created *Pods* to healthy nodes based on resource availability, hardware constraints, and affinity rules. 2 phases: **filtering** (finding capable nodes) and **scoring** (ranking best fit). The scheduler watches the API server for Pods with "spec.nodeName: null" and these are **queued** for scheduling.

Each node is then evaluated against hard constraints (enough CPU, memory budget...) and each feasible node is scored using weighted plugins (least/most resource-usage, pod affinity, topology...): the node with the highest score wins → the scheduler updates the Pod object via the API server, setting the **spec.nodeName** field to the chosen node (if no nodes are available, scheduler may evict lower-priority Pods)

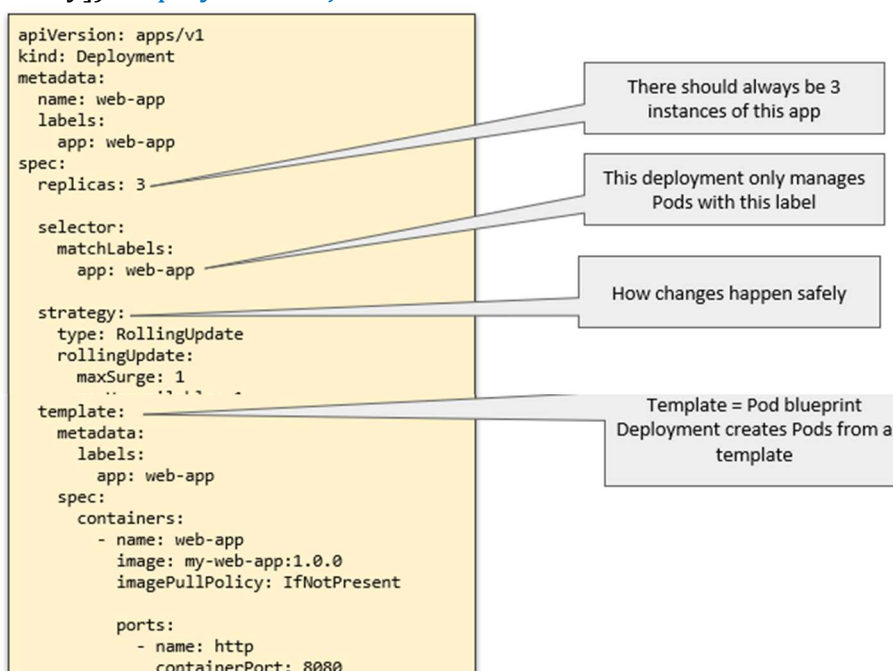
⚠ Scheduler is **optimistic** (assumes that nodes will accept the Pod and that resources will not disappear mid-decision [if reality changes, Pod fails to start and controllers will reschedule]) and **stateless** (multiple scheduler can run → leader election ensures that only 1 binds Pods)

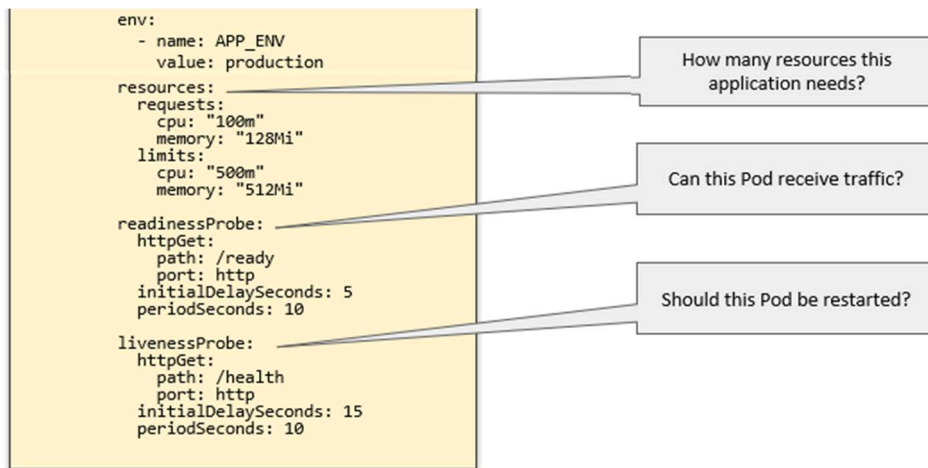
⚠ Nodes can be decorated with **taints** (they let nodes say "I repel certain workloads unless they explicitly opt in"); also Pods can declare their **tolerations** ("This Pod is allowed to run on nodes with that taint") [in fact, without taints, any Pod can land on any node → special-purpose nodes get polluted]

- **Controller** = **long-running specialized control loop** that watches the Kubernetes API and takes actions to move the cluster from its current state toward the desired state (always running, level-based, idempotent, narrow responsibility, acts only via API) [each controller is responsible for **enforcing a single invariant** that keeps the cluster correct despite constant failure and change].

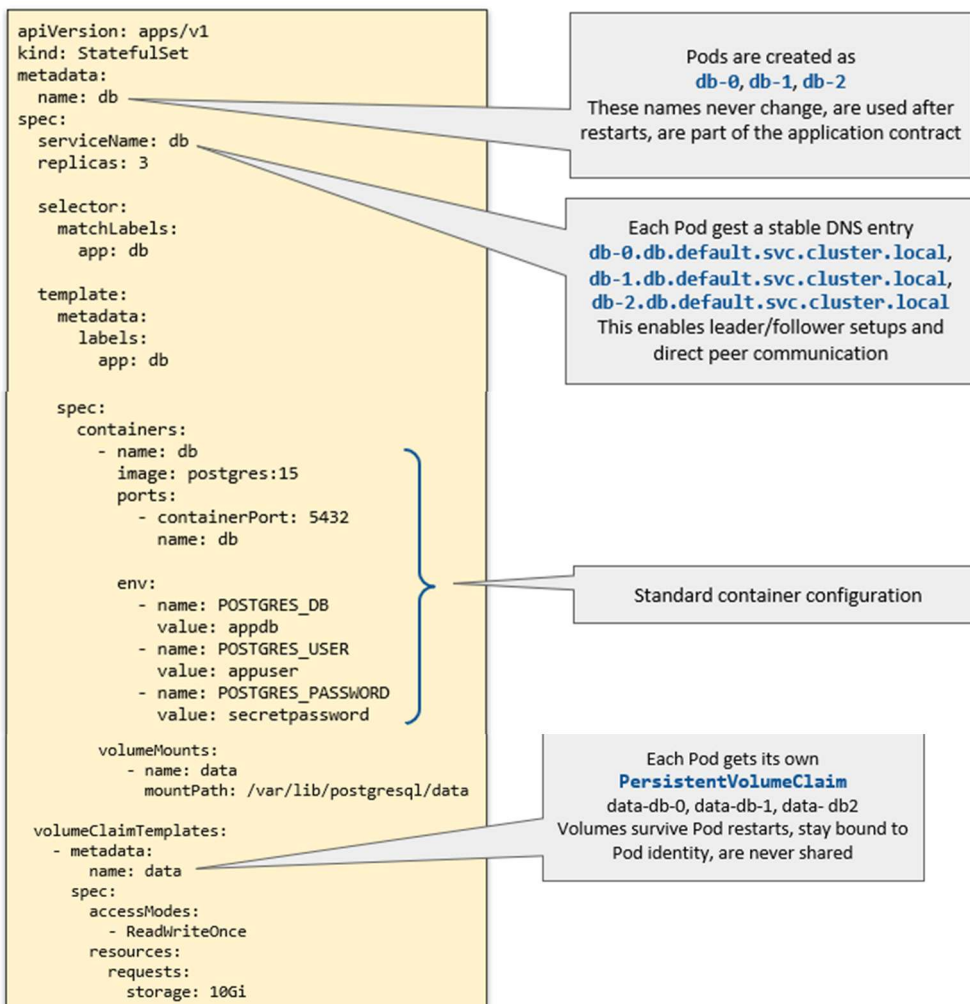
Types of controllers:

- o **Deployment controller** = watches **Deployment objects** enforcing that the app matches the desired version and replica count, respecting rollout strategy (ensures that correct ReplicaSet objects exists [old ReplicaSet are scaled down, new ReplicaSet are scaled up → rollouts converge safely]). **Deployment object**:



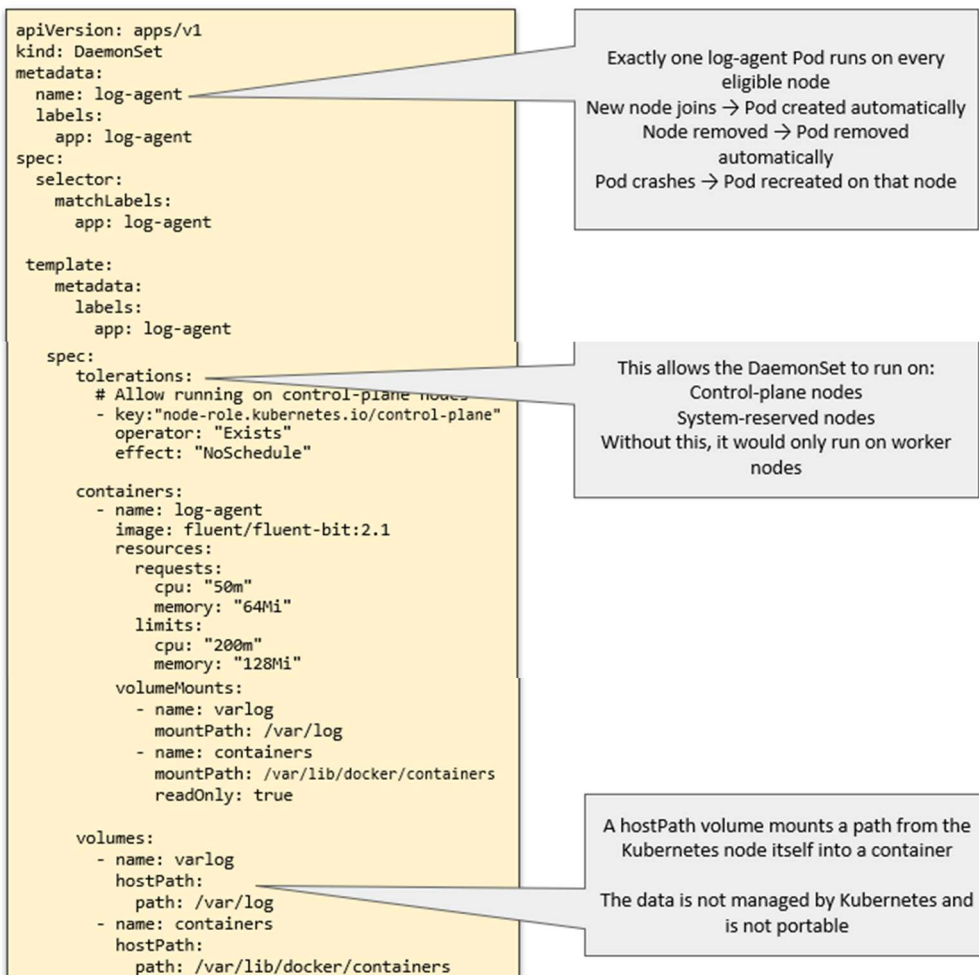


- **ReplicaSet controller** = enforces that the number of owned active Pods equals the value of the `spec.replicas` field in each **ReplicaSet** object (creates Pods if <, deletes Pods if >, ignores why Pods disappeared)
- **StatefulSet controller** = guarantees that all Pods owned by StatefulSet objects have a stable identity, stable storage and ordered lifecycle (assign deterministic Pod names [app-0, app-1...], start and stop Pods in order, keep volumes bound to Pod identities). **Stateful object**:



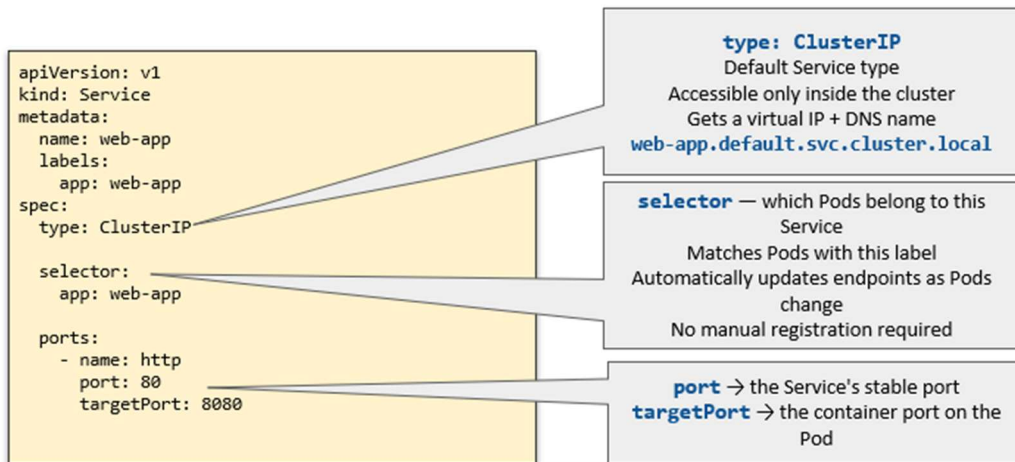
⚠ A StatefulSet requires a **headless Service companion** to provide stable network identity (this Service is what enables stable DNS entries per Pod [without StatefulSet DNS identity don't exist])

- **DaemonSet controller** = guarantees that exactly 1 Pod runs on each eligible node (new node implies new Pod; node removed implies Pod removed). **DaemonSet object**:



- **Job controller** = guarantees that a specified number of Pods complete successfully (retry failed Pods, track completions, stop when success criteria is met)
- **CronJob controller** = ensures that jobs are created according to a schedule (timely job creation, concurrency policy, cleanup of old Jobs)
- **Node controller** = takes care that the cluster's view of nodes reflects their actual health (mark nodes Ready/NotReady, applies taints on failure, triggers Pod eviction)
- **Namespace controller** = guarantees that a deleted namespace contains no remaining resources (cascading deletion, finalizer cleanup)
- **Garbage Collector controller** = takes care that no orphaned resources exist (honors Owner-References, deletes dependent objects when owner is removed, ensures that cascading cleanup is correct)
- **ServiceAccount controller** = invariant enforced (each namespace has default **ServiceAccount** [auto-creation + token management])
- **ResourceQuota controller** = ensures that namespaces can't exceed declared resource limits (tracks aggregate usage, block excessive creation)
- **Horizontal Pod Autoscaler controller** = guarantees that replica count reflects observed metrics within bounds (scales up/down, honoring min/max constraints, stabilizes fluctuations)
- **Endpoints controller** = guarantees that services route traffic only to Pods in the **ready** state (adds/removes Pod IPs from load-balancer list, reflects readiness changes)
- **Service controller** = invariant enforced: service status matches underlying infrastructure (provisions load balancer, assigns external IP).

⚠ Since Pods are ephemeral and their IPs change upon restart, a Service provides a stable IP and DNS name (type of **exposure**: **ClusterIP** [internal communication within the cluster], **NodePort** [binds a port on every worker node to the service], **LoadBalancer**). **Service object**:



⚠ An **Ingress** resource manages L7 traffic (providing features like SSL termination and path-based routing) [the *newer gateway API* offers a more extensible way to manage complex multi-tenant routing scenarios]

- **Controller Manager** (`kube-controller-manager`) = inspector ensuring the building matches the blueprints. It's a **long-running control-plane** process that hosts and coordinates many independent controllers (a *container* for control loops that manages the *lifecycle* of controllers).

⚠ In fact, all controllers share the same concerns (API client config, authN credentials, leader election, rate-limited work queues, shared caches...), so **grouping** them in a single process reduce duplication, simplifies deployment and improves performance

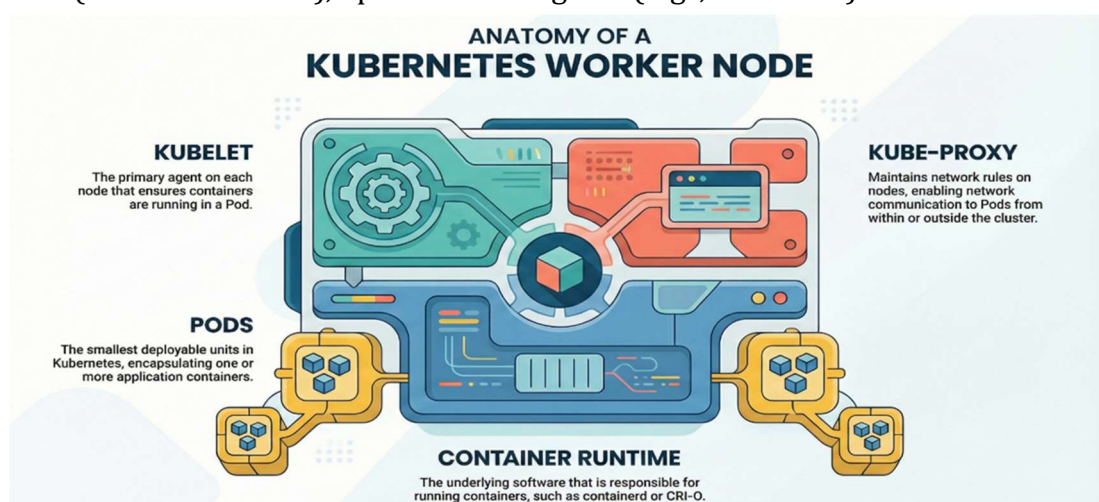
Several concurrent tasks run inside Controller Manager:

- **core** workload controllers (Deployment, ReplicaSet, StatefulSet...)
- **cluster** level controllers (Node, Namespace, ServiceAccount, GarbageCollector...)
- **custom** ones

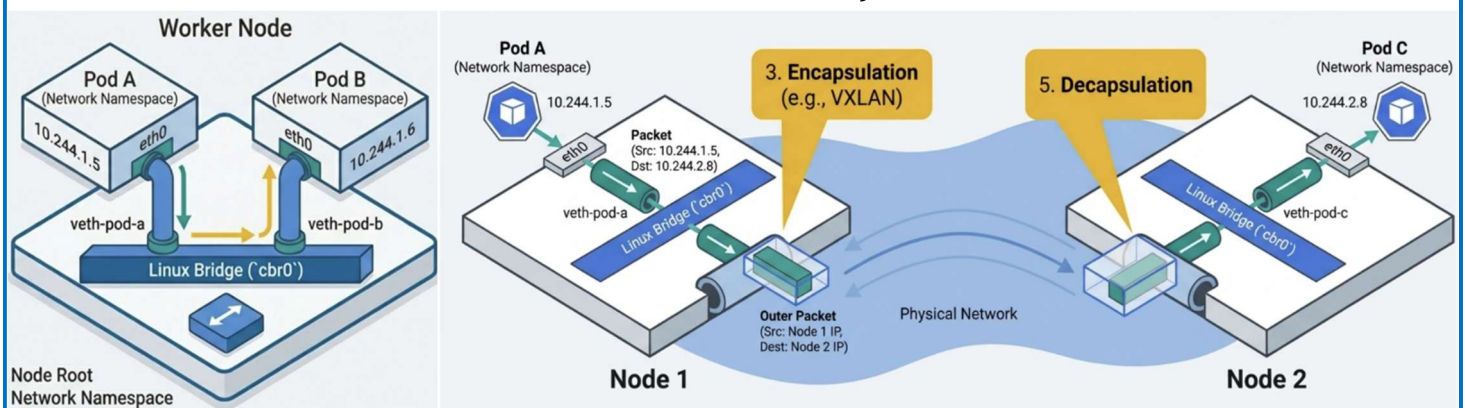
Each controller watch different resources, owns a specific invariant, has its own queue and workers and can fail independently (although they share a binary, controllers are **logically isolated**). Multiple controller managers can run simultaneously, but only 1 is active at a time (a **leader election** based on K8s *leases* [stored in etcd] guarantees that 1 instance holds leadership lock, while the others stay passive [*hot standby*])

⚠ If controller manager is unavailable, running workloads continue, but no new reconciliations!

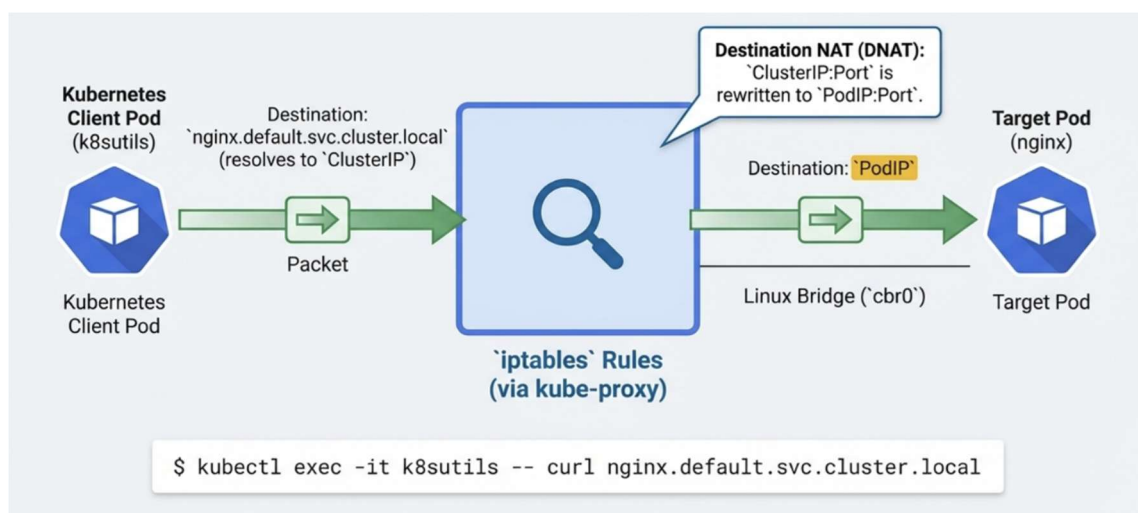
- **Worker Node** = worker node is responsible for **executing Pods and reporting reality back to the control plane** (control plane decides, worker executes: **nodes never make cluster-level decisions**). A typical worker node runs kubelet (node agent), container runtime, kube-proxy (network rules), OS + kernel (the real executor), optional node agents (logs, monitor...):



- **Kubelet**: only component on a node that talks to Kubernetes API server; he watches for Pods assigned to this node, creates and manages containers via container runtime, mounts volumes, runs health probes and reports Pod and node status.
Kubelet runs a local reconciliation loop (watch API server for assigned Pods, compare desired Pod spec vs running containers [+ manages containers]):
- **Container runtime** = this component is what actually pull images, create containers and starts/stops containers (implemented by **containerd** or **CRI-O**) [K8s talks to runtime via Container Runtime Interface (**CRI**)]
- **Pod execution model** = a Pod on a node consists of 1 or more containers + a shared network namespace + shared volumes. **When a Pod starts**:
 - kubelet creates a **Pod sandbox**
 - **networking** is setup via CNI → every Pod gets its own IP and any Pod can talk to any other Pod without NAT (main promise made by K8s); every Pod runs inside its own Linux network namespace. **When a Pod is scheduled** to a node, kubelet asks the container runtime to create a Pod sandbox. On the node side, a **veth pair** is created (this create a fast kernel-level plumbing that move packets on the node into the Pod namespace and v/v). Images (sx = network on the node; dx = network across nodes):



- **Service networking**: **kube-proxy** watches the API server for Services and EndpointSlices → whenever a change is reported, it programs node-local forwarding rules (**don't proxy traffic itself**: when a Pod sends packets to a Service, kube-proxy rewrites the destination address, replacing it with the address of one of the Pods associated with the Service) [**services are not real endpoints** (no interface, no socket, no process), but they are **kernel-level forwarding rules**] [when a ready probe fails for a given Pod associated with a Service, kube-proxy removes it from the *eligible list* → control plane is not in the data path]



⚠ **External HTTP/S traffic** can't reach internal Services, unless a suitable routing mechanism is setup → **Ingress** is a K8s API object that declares how external HTTP/S traffic should be routed to internal Services (ingress controller is a real component that runs as a Pod)

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-example
spec:
  rules:
  - host: "store.example.com"
    http:
      paths:
      - path: /products
        pathType: Prefix
        backend:
          service:
            name: products-service
            port:
              number: 80
      - path: /checkout
        pathType: prefix
        backend:
          service:
            name: checkout-service
            port:
              number: 80

```

Host-based routing
Directs traffic based on the domain name

Path-based routing
Further route traffic based on the URL path

Backend definition
Each path maps directly to a backend Service and port

- **volumes** are mounted
- **containers** are started
- **probes** begin (kubelet periodically probes containers to decide whether they are alive [liveness probe], ready to receive traffic [readiness probe] or still starting [startup probe]). Each probe can use 1 of 3 mechanisms:

- **HTTP** probe = most common, language agnostic, return HTTP status code 200-399 if success, 400-599 if fail
- **TCP** probe = checks if port is open, no app semantics, useful for simple services
- **Exec** probe = run inside container, tightly coupled to container internals

```

httpGet:
  path: /ready
  port: 8080

```

```

tcpSocket:
  port: 8080

```

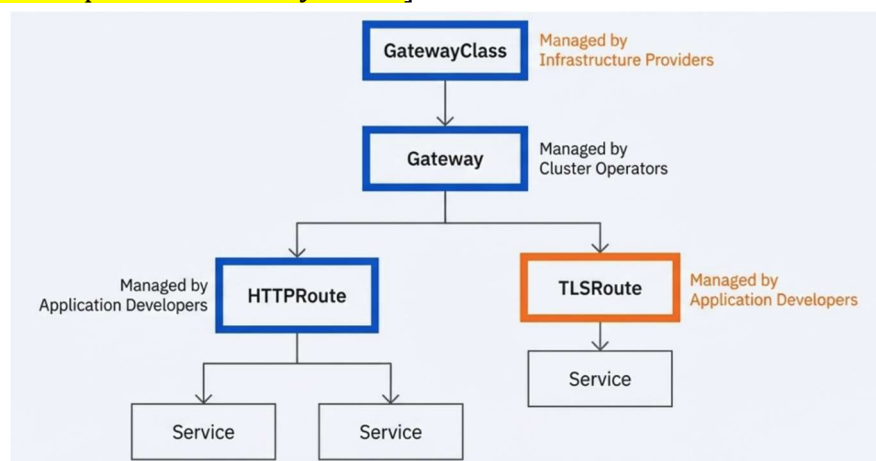
```

exec:
  command: ["cat", "/tmp/healthy"]

```

- **status** is reported

- **API Gateway evolves K8s networking by separating concerns**, providing a highly extensible and role-based model for traffic management (authN, authZ, rate limiting, request validation, schema enforcement, API versioning, transformations, observability, developer experience) [Service Mesh is part of this ecosystem ↓]



- **Service Mesh** = private security and radio network ensuring only authorized workers can speak to each other in a language the public can't hear

Talking about **VOLUMES**:

- **Volume handling in Pods** → containers are ephemeral, apps not; volumes exist to persist data across container restarts, share data between containers in same Pod, inject config and secrets, bridge Pods to external storage systems. In K8s, **volumes belong to Pods** (containers mount volumes, but volumes outlive containers [but not Pods]) [restarting a container doesn't lose volume data]. When a Pod is bound to a node, kubelet interprets **volumes** and **volumeMounts** (specifying what container will mount them). The **Container Runtime** mounts volumes into containers and **control plan** only declares what volumes are needed. There are **5 major volume types**:
 - o **emptyDir** = ephemeral shared storage (created when Pod starts, deleted when Pod is deleted), shared across containers in the Pod
 - o **ConfigMap & Secrets** = config injection (files generate from API objects, automatically updated, read-only by default, used for app config, certificates, feature flags)
 - o **PersistentVolumeClaim** (PVC) = durable storage (survives pod deletion, backed by real storage, used for persistent data)
 - o **hostPath** = node-local filesystem (direct access to node filesystem, node-specific, no portability guarantees [used for node-level agents, logging, monitoring])
 - o **CSI-backed volumes** = extensibility point (cloud disks, network filesystem, distributed storage systems)

```
volumes:  
- name: cache  
  emptyDir: {}
```

```
volumes:  
- name: config  
  configMap:  
    name: app-config
```

```
volumes:  
- name: data  
  persistentVolumeClaim:  
    claimName: db-data
```

```
volumes:  
- name: logs  
  hostPath:  
    path: /var/log
```

- **Volume provisioning** → to satisfy a **PersistentVolumeClaim** (PVC), 1 of more **PersistentVolume** (PV) should be available to the cluster (a disk, NFS share or cloud volume with fixed characteristics managed at cluster/platform-level). **PVs** are created manually by an admin or dynamically by a **StorageClass** (they exist independently of Pods, have a given capacity, access modes, policies) [they are backed by real storage via CSI]

```
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: pv-10g  
  
spec:  
  capacity:  
    storage: 10Gi  
  
accessModes:  
  - ReadWriteOnce  
  
persistentVolumeReclaimPolicy: Retain  
csi:  
  driver: example.csi.io  
  volumeHandle: vol-123
```

When a **PersistentVolumeClaim** object is created, control plane searches for a matching **PV** (enough capacity, compatible access mode, matching **StorageClass**). If found, **PV** ⇔ **PVC** are bound, otherwise **PVC** stays **Pending** (or a **PV** is dynamically provisioned)

⚠ **Operators** extend K8s with *app-specific control loops* (they allow to deploy, config, operate and recover a specific application) [while *built-in controllers* handle generic-invariants, operators allow to manage specific ones]. Operators are based on 2 building blocks:

- **CRD** (Custom Resource Definition) = express the intent via a custom object
- **Operator controller** = defines the logic to be implemented

Level of operator sophistication → lvl 1 = basic automation; lvl 2 = full lifecycle; level 3 = autonomous operations

Deploying large applications involves a lot of yaml files: to solve this problem, **Helm** was introduced (it packages all K8s resources in a single versioned chart that can be easily customized acting on a single file [**values.yaml**]; the entire app stack can be installed and upgraded with a single command).



Helm is the **package manager for K8s**: a **Helm chart** is a pre-configured package of K8s resources (YAML manifest [**values.yaml**]) that define how to run an app/service (instead of writing a bunch of YAML separated files).

A configured chart can be deployed to a K8s cluster becoming a **Helm release** (you can install the same chart multiple times, each as a separate release). Helm templates can be stored and fetched from a **Helm repository**. A Helm chart contains YAML templates with placeholders (variables):

```
# deployment.yaml (template)
spec:
  replicas: {{ .Values.replicaCount }}
```

```
# values.yaml (defaults)
replicaCount: 3
```

Charts are packaged and stored in **chart repo** (like Docker images in registries) [you can install charts also from public repos → **helm repo add bitnami https://charts.bitnami.com/bitnami; helm install mydb bitnami/mysql**]. When you run **helm install**, Helm renders the templates with your values and submits them to Kubernetes API; you can also **upgrade**, **rollback** or **uninstall** releases. **Helm stores release metadata as K8s Secrets** (kept in the namespace where the release is installed → the secret contains the chart version used, values provided at install/upgrade, K8s manifests applied, revision number [release history because **every operation creates a new revision** and revisions are numbered progressively so that you can rollback to any previous rev]). **Lifecycle operations** are:

- **Install** = creates the release (rev 1)
- **Upgrade** = applies changes, increments rev count
- **Rollback** = restores manifests from a prior rev
- **Uninstall** = deletes all K8s resources for the release and its metadata

⚠ Helm ensures that the **desired state defined by the chart + value is applied consistently** (in fact, like K8s itself, Helm operations are **idempotent**). **Typical structure**:

```
mychart/
├── Chart.yaml    # Chart metadata (name, version, description)
├── values.yaml   # Default configuration values
├── templates/    # YAML templates for K8s manifests
├── charts/       # Subcharts (dependencies)
└── README.md     # Optional, documents the intended usage
```

K8s Security Model: K8s assumes the cluster is **hostile by default** and security must be enforced at multiple independent layers:

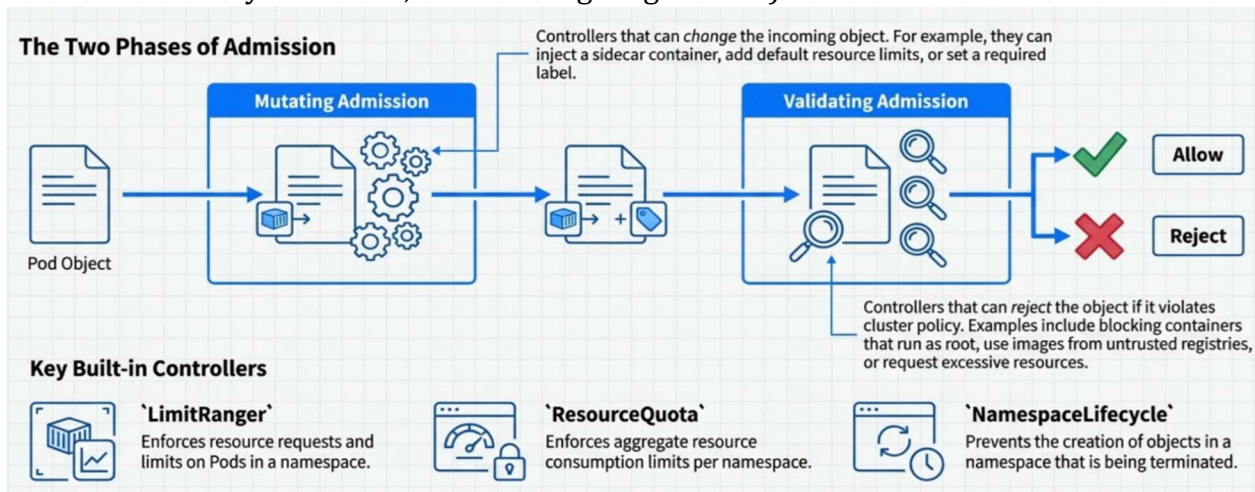
- **Authentication** → before any action is considered, **kube-apiserver** must verify identity of requestor. AuthN is externalized and pluggable (X.509 certificates); Pods authenticate using **ServiceAccount** tokens (automatically mounted into the container filesystem to allow secure API access)
- **Authorization** → once requestor is authenticated, **RBAC** (Role-Based Access Control) determines if their identity has permission to perform the requested action on the target resource [RBAC is made of **Role/ClusterRole** (permissions) and **RoleBinding/ClusterRoleBinding** (attach permissions to identities)]. RBAC is **explicit, deny-by-default, evaluated on every request**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: dev-developer
  namespace: dev      # restrict to
                     # this namespace
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]

- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "create",
           "update", "patch"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-developer-binding
  namespace: dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-developer
  apiGroup: rbac.authorization.k8s.io
```


- **Admission Control** → after a request is authenticated and authorized, it passes through a series of admission controllers (can inspect, modify, reject the request before it's persisted to [etcd](#)). Possible policies enforced by admission controllers are: require resource limits, block privileged containers, enforce Pod security standards, restrict image registries...)



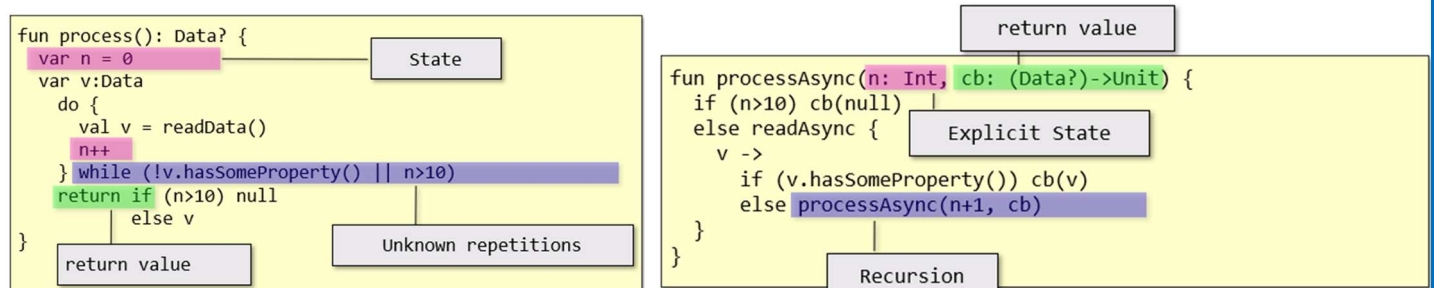
- **Runtime Security** → even after passing admission, **threats exist** (pods may be compromised, code can be malicious...). Containers provide process and resource isolation. K8s defines security profiles (**restricted**, **baseline**, **privileged**), but follows the **least privilege** (most prod workloads should be **restricted**)
- **Network Security** → by default, all Pods can communicate with all other Pods, regardless of namespace: **NetworkPolicy** resources allow you to specify how groups of Pods are allowed to communicate with each other and other network endpoints

⚠ A **Secret** is a K8s API object used to store and distribute sensitive data to Pods in a controlled way (secrets decouple sensitive data from container images, pod specs, source code, git repos). Secrets are stored in etcd as **base64-encoded strings** (whoever has access to namespace with the get, list and watch permission can decode secrets)

6) Asynchronous Execution Models for Cloud-Native Systems

Asynchronous Programming (calling different threads to execute things in parallel) needs memory (also *pre-allocated* threads to support many concurrent requests), more pods (and more nodes) and blocking I/O → **more cost!**

We can evolve this paradigm if the underlying system provides **async API**, which divides computation in 2 parts (what happens before the blocking call, what happens after the blocking call). We can see a **synchronous [sx] vs asynchronous [dx]** call:



Callbacks reduce # of threads requested, but makes it difficult to handle most async corner cases (exceptions, cancellation, multiple async computations...). To deal with these problems, there are **2 async models**:

- **Reactive Streams libraries** (like **RxJava** or **Reactor**) = require the programmer to describe the computation to be performed in terms of a **chain of functional operators** to be applied to an observable stream of events.

Package **org.reactivestreams** defines java/Kotlin abstractions for **Reactive Extensions** (tools that allows imperative languages to work with both synchronous and asynchronous data streams). It offers a combination of **Observer** pattern and **Iterator** pattern, and functional programming

```
interface Subscriber<T> {
    // Invoked after calling
    // Publisher.subscribe(Subscriber)
    // No data will start flowing until
    // Subscription.request(long) is invoked
    fun onSubscribe(s: Subscription);

    // Invoked by publisher if subscription is active
    // and a request for more data has been issued
    fun onNext(t: T);

    // Fail terminal state.
    // No more invocations will be received
    fun onError(t: Throwable);

    // Successful terminal state.
    // No more invocations will be received
    fun onComplete();
}
```

```
interface Subscription {
    // No events will be sent by a Publisher
    // until demand is signalled via this method
    fun request(n: Long);

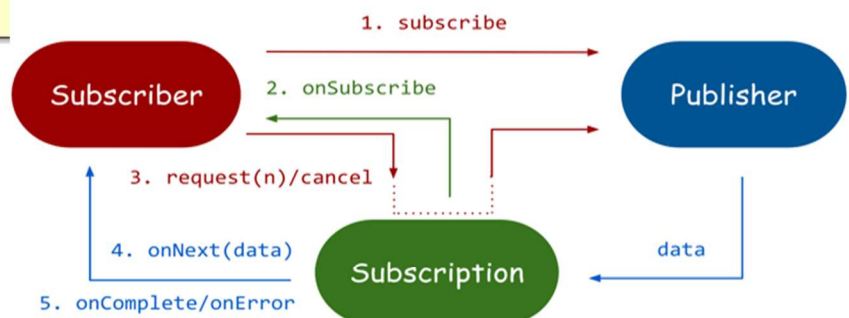
    // Request the Publisher to stop sending data
    // and clean up resources
    // Data may still be sent to meet previously
    // signalled demand after calling cancel
    fun cancel();
}
```

```
interface Publisher<T> {
    // Request Publisher to start streaming data
    // This can be called multiple times,
    // each time starting a new Subscription
    // Each Subscription will work for only a single Subscriber
    // A Subscriber should only subscribe once to a single
    // Publisher
    // If the Publisher rejects the subscription attempt
    // or otherwise fails, it will signal the error via
    // Subscriber.onError(...)
    fun subscribe(s: Subscriber<in T>?)
}
```

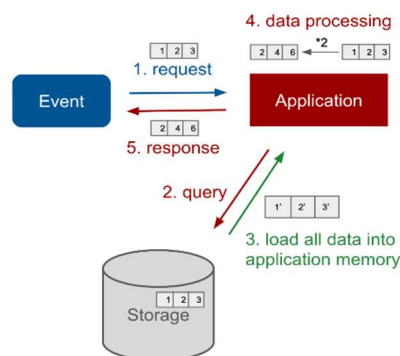
```
// A Processor represents a processing stage
// which is both a Subscriber and a Publisher
// and obeys the contracts of both
// T is the type of element signalled to the Subscriber
// R is the type of element signalled by the Publisher

interface Processor<T, R> :
    Subscriber<T>,
    Publisher<R> {
}
```

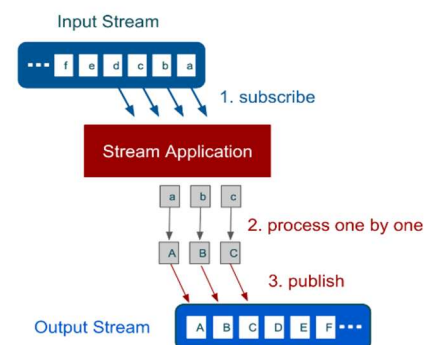
The **interaction diagram** is this →



The **approach** is this:



Traditional Data Processing

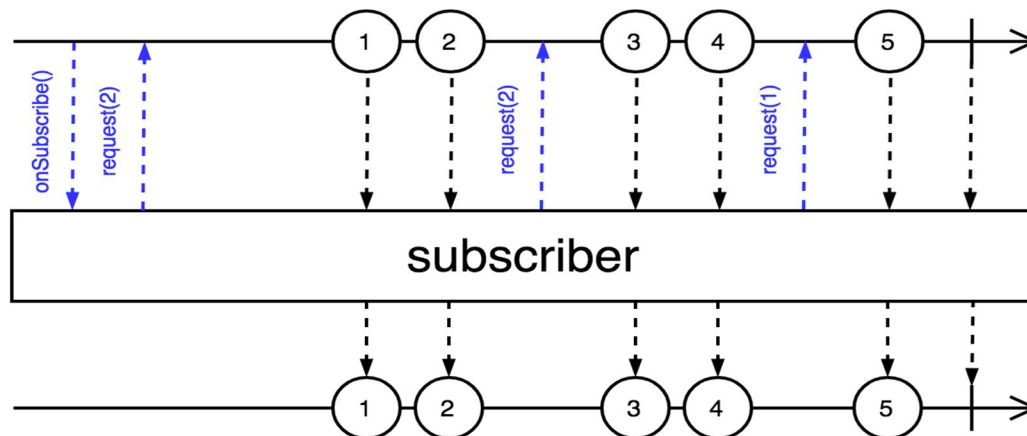


Stream Processing

Base concepts (similar to how K8s controllers consume events from etcd)

- When a **Subscriber** connect to a **Publisher** (via `subscribe(...)` method) an **async process** of event generation is triggered
- **No data** is actually transmitted between the 2 parties, until the receiver signals its will to accept some data via `subscription.request(...)` (to avoid introducing queues between publisher and subscriber)

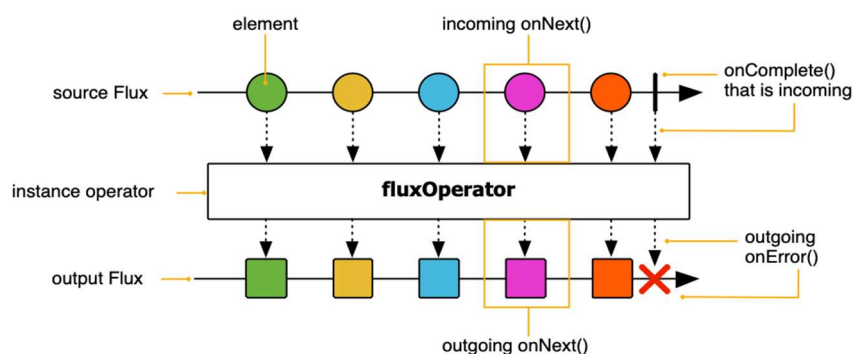
Push-based vs Pull-based architectures: a pure pull-based architecture is slow → by letting the remote source push items freely, a large speedup can be achieved. Backpressure via subscription:



As said before, 2 alternatives implementation exists: **RxJava** (and **RxKotlin**) and **Project Reactor**: both schedule the events to be sent according to a well defined state machine linking **Publisher** to **Subscriber**, but they differ in terms of classes that implements the reactive abstractions:

- **Project Reactor** provides 2 implementations of the **Publisher<T>** interface:
 - **abstract class Flux<T> {...}** = defines a reactive stream that can produce zero, 1 or many elements
 - **abstract class Mono<T> {...}** = defines a stream that can produce at most 1 element

These classes have a lot of operators (this graphic depicts the time behavior of an operator):



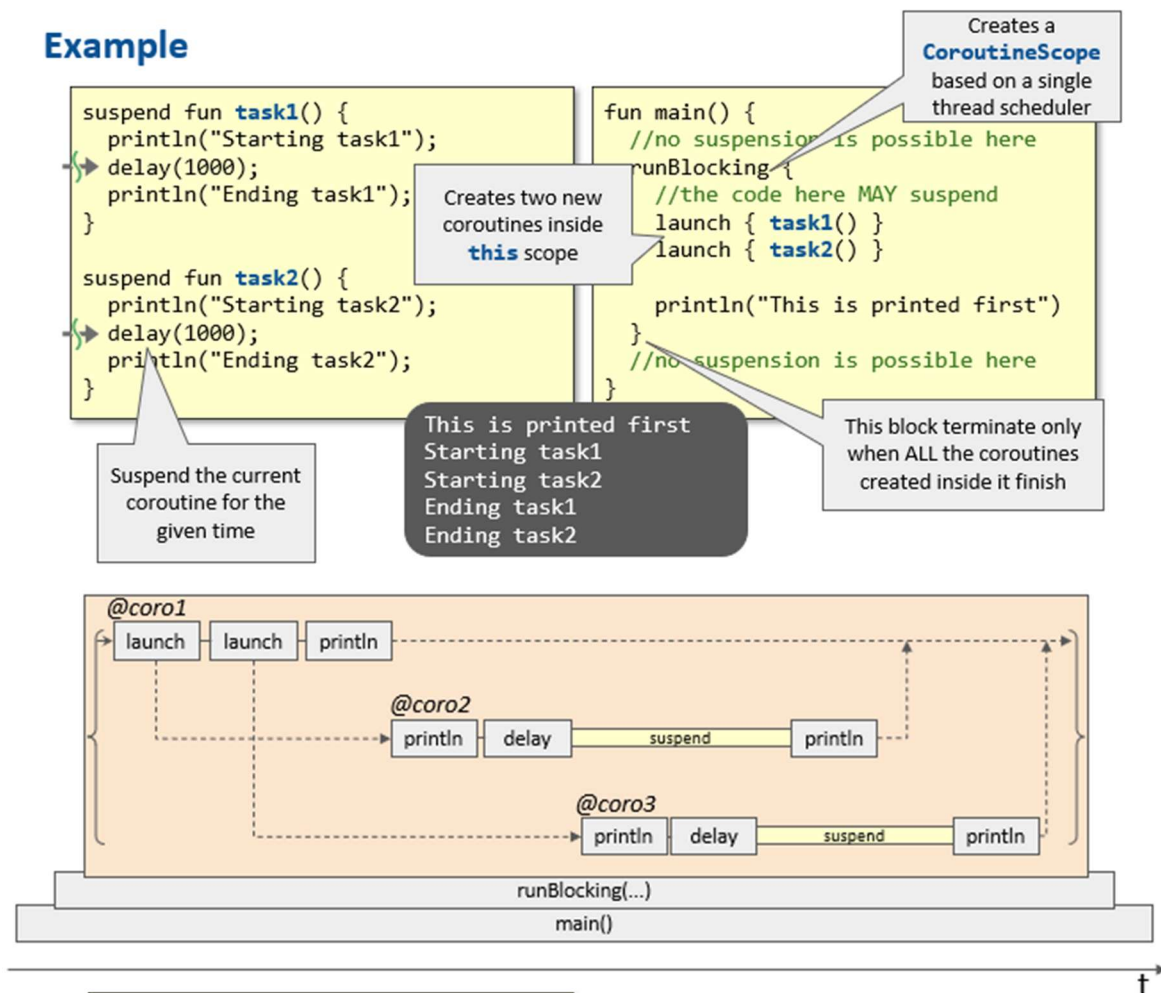
- **RxJava3** has similar concepts with different names (**Observable**, **Flowable**, **Single**, **Maybe**, **Completable**)
- **Coroutines** = based on the idea of **suspendable computations** and leverage on the compiler capability to transform a mostly-imperative code in a callback-based state machine. Coroutines create an **abstraction layer** on top of system threading (more consistent, scalable and maintainable abstraction for writing async code) using suspending functions. In fact, a **COROUTINE** is an instance of a **suspendable computation** (a computation that can suspend at a given point, freeze its state and later can be resumed, possibly in a different thread from the previous one). This allows to express sequential logic but with asynchronous computation (with coroutines, a computation that can't proceed may **suspend itself**).

Together with the idea of coroutines, Kotlin has introduced the **suspend fun** (a new kind of functions that can be suspended and can be only invoked in a coroutine). Coroutines can only be created inside a **CoroutineScope** which encompasses their lifetime and offers a set of builder functions to create new coroutines (each scope encapsulates a **CoroutineContext** that controls the run-time behavior of the coroutines spawned from it). When a suspending function is invoked, the computation will proceed until it reaches a **suspension point**, where it may decide to suspend (suspension points correspond to the invocation of other **suspend fun**) [coroutines are part of Kotlin standard library].

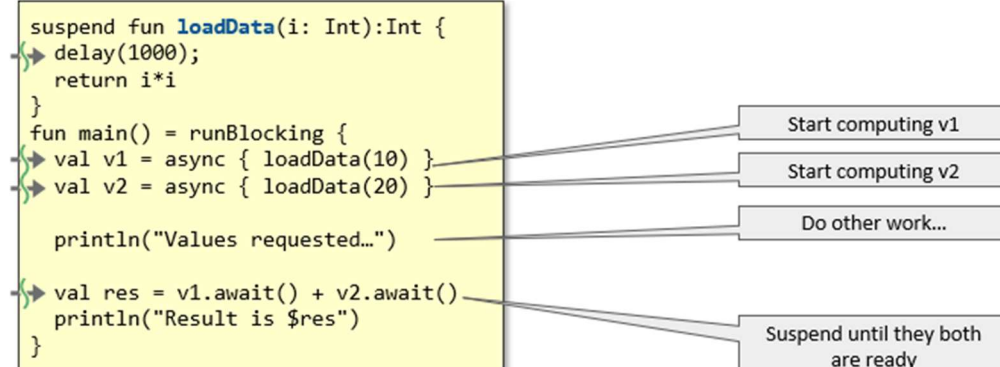
A coroutine is **created** using 1 of the following methods of a **CoroutineScope** instance:

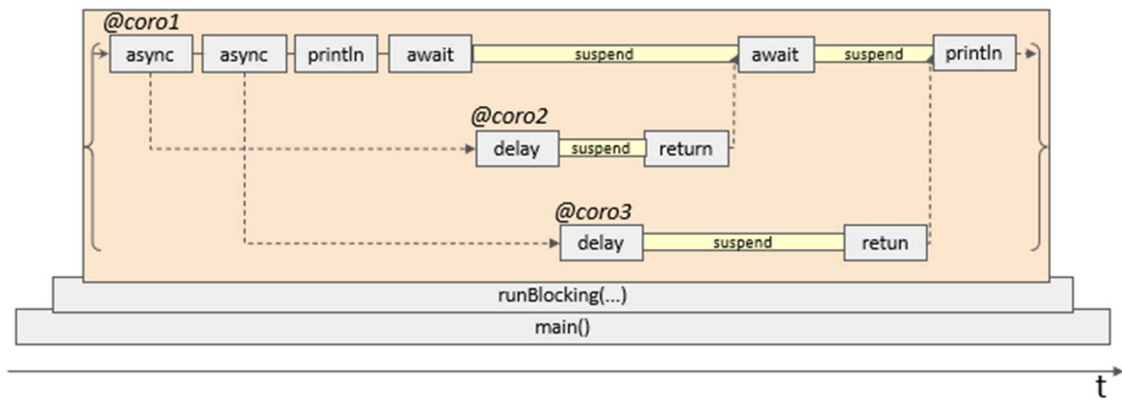
- **launch(...)** → launches a new coroutine without blocking the current computation and return a **Job** that represents the launched computation (*fire-and-forget*)
- **async(...)** → creates a coroutine and returns its future result as a **Deferred<T>** object (this can be later awaited for, in order to *access the returned value*) [if requested computation has failed with an exception, the **await()** method **re-throws** if, instead of returning a value (*)]
- **runBlocking(...)** → launches a new coroutine, blocking the current thread until it and all its sub-coroutines finish

Example

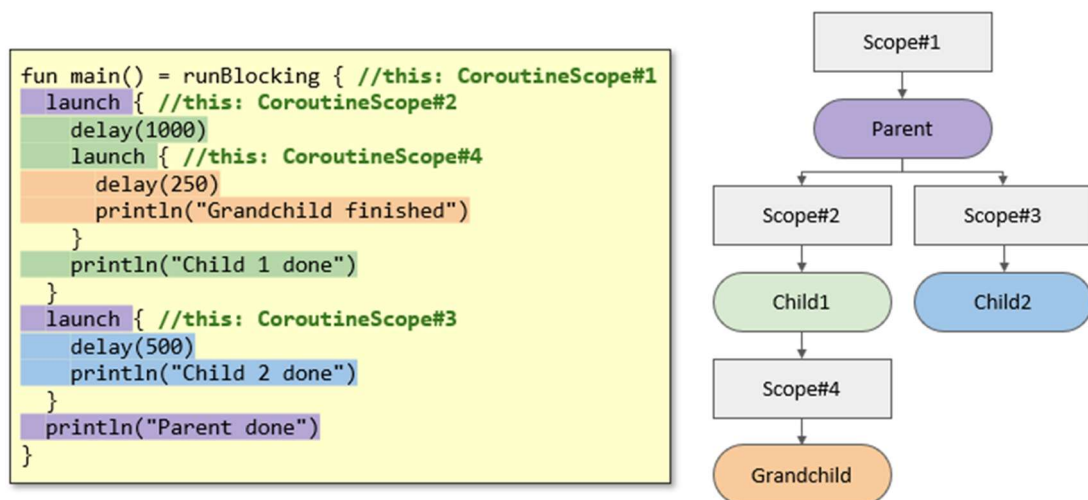


(*)

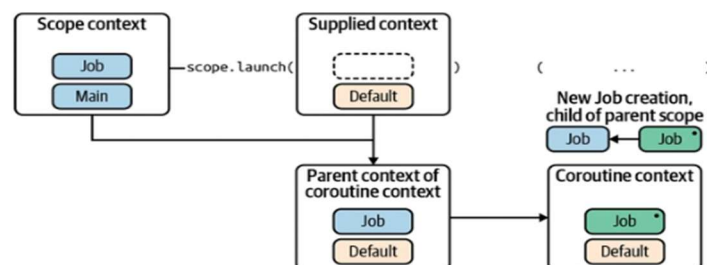




Structured concurrency: OS scheduler manages its threads as independent computations, instead Kotlin coroutines maintain their parent/child relationships → this happens because the **CoroutineScope** not only provides methods for creating new coroutines, but it also tracks their relationships (when a new coroutine is created using **launch(...)** or **async(...)** in the body of another coroutine builder, it automatically becomes a child of the one in which it was create). A **CoroutineScope** defines a **failure domain** (just like a Pod or a K8s controller).



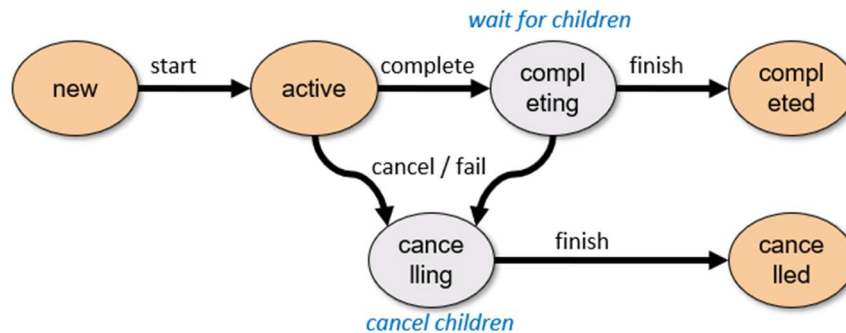
A **CoroutineScope** is an interface that encapsulates a **CoroutineContext** and defines a set of extension functions useful for managing (creating, destroying) a group of related coroutines. To create a new scope there is a function **CoroutineScope(...)** which create a new scope based on the given coroutine context. Every scope encapsulates a **CoroutineContext** (when a coroutine is created inside a scope, it inherits its context):



Context defines run-time behavior of a coroutine (in terms of a set of properties) and are typically created adding the elements they are made of (e.g. `val ctx: CoroutineContext = Dispatchers.IO + SupervisorJob()`):

- **CoroutineDispatcher** → defines set of threads used to execute the coroutine (determines what thread or threads the corresponding coroutine uses for its execution [class `kotlinx.coroutines.Dispatchers` provide ready-made dispatchers: `Dispatchers.IO`, `Dispatchers.Default`, `Dispatchers.Unconfined`])

- **Job** → represents the state of the computation of the coroutine (state of background tasks carried out by coroutines); a job may have 6 states (combination of `isActive`, `isCompleted`, `isCancelled`) and jobs are in hierarchies. **SupervisorJob** is a Job subclass that allows children to fail independently of each other.



Jobs offer several methods to control their state (`fun start()`, `fun cancel(cause)`, `suspend fun join()`).

- **CoroutineExceptionHandler** → defines how exceptions thrown inside coroutine are handled (coroutines created with the `async` builder function always catches all their exceptions and throw them when `await` is invoked).

All suspending functions from `kotlinx.coroutines` package are cancellable (a periodic check on `isActive` property or an invocation of the `yield()`, to quit in case of cancellation [if context contains an instance of **Job**, cancellation of a single coroutine propagates to siblings; if instance of **SupervisorJob**, not propagates])

⚠ **supervisorScope(...)** creates a `CoroutineScope` with a `SupervisorJob` and calls the specified and calls the specified suspend block with this scope

⚠ **launch(...)** spawns cancellable tasks (*fire and forget*)

- **CoroutineName** → specifies the name of coroutine (debug)

Synchronization: since coroutines introduce concurrency in programs, they have all the problems that usual concurrent programs are subject to; to avoid these, the library provides a couple of classes which support synchronization leveraging on suspension and not on blocking:

- `kotlinx.coroutines.sync.Mutex` protects all modifications of the shared state with a critical section that is never executed concurrently. A **Mutex** has 2 states: **locked** and **unlocked** (`withLock {...}` is used to suspend until it's possible to grab it):

```

fun main() = runBlocking<Unit> {
    var mutex = Mutex()
    var n = 0
    runBlocking(Dispatchers.Default) {
        launch {
            for ( i in 1..1_000_000 )
                mutex.withLock { n++ }
        }
        launch {
            for ( i in 1..1_000_000 )
                mutex.withLock { n++ }
        }
    }
    println("Final value: $n")
}

```

- `kotlinx.coroutines.sync.Semaphore` limits number of coroutines that can execute specified code at the same time. When it's created, it's assigned a number of permits (`withPermit {...}` allows to control the degree of concurrency) [1 permit = similar to `Mutex`]:

```

suspend fun loadData(name: String) {
    println("Begin $name")
    delay((Math.random()*300+500).toLong())
    println("End $name")
}

fun main() = runBlocking<Unit> {
    var semaphore = Semaphore(2)
    runBlocking(Dispatchers.Default) {
        for (i in 0..3)
            launch {
                semaphore.withPermit {
                    loadData("t$i")
                }
            }
    }
}

```

```

Begin t0
Begin t1
End t1
Begin t3
End t0
Begin t2
End t2
End t3

```

To avoid interference, an approach is to **confine shared state modifications** to blocks manipulated by a single-thread dispatcher (e.g. the way Android manage UI-related code).

FLOWS = coroutine-native answer to stream data over time (because coroutines are restricted to return a single value). Use **Flow** when streaming stays inside a service boundary, use **Reactive Streams** when crossing service or process boundaries. Kotlin library introduce a new interface named **Flow<T>** that is an **async data stream that sequentially emits values** (can complete normally or with an exception). And there are a lot of **extension functions** (**map**, **filter**, **take**, **zip**...) that provide intermediate operators implementation that transform the original flow in a different one. **Terminal operators** are either suspending functions (**collect**, **single**, **reduce**, **toList**...) or operators that trigger execution of further processing steps. Most flow operations are executed sequentially in same coroutine. Flows can be:

1. **cold** → don't produce any value unless it's collected
2. **hot** → can emit values even if no one is listening to it

```

interface Flow<out T> {
    //Internal method not to be used directly
    suspend fun collect(collector: FlowCollector<T>)
}

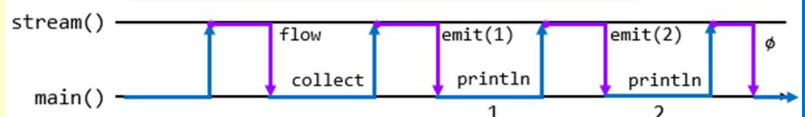
interface FlowCollector<in T> {
    //Not thread safe - MUST BE INVOKED SEQUENTIALLY
    suspend fun emit(value: T)
}

```

```

fun stream(): Flow<Int> = flow {
    emit(1)
    emit(2)
}

```



```

fun main() = runBlocking {
    stream().collect { value ->
        println(value)
    }
}

```

Helper functions are used to **create a flow**:

- o fun <T> **flowOf**(vararg elements: T): Flow<T>
- o fun <T> Iterator<T>.asFlow(): Flow<T>
- o fun <T> Iterable<T>.asFlow(): Flow<T>
- o fun <T> Array<T>.asFlow(): Flow<T>
- o fun <T> Sequence<T>.asFlow(): Flow<T>
- o fun <T> IntRange.asFlow(): Flow<T>
- o fun <T> (() -> T).asFlow(): Flow<T>
- o fun <T> **emptyFlow**(): Flow<T>
- o fun <T> **flow**(
 block: suspend FlowCollector<T>().->Unit
): Flow<T>

Operators

- **Intermediate**:

- o **map(...)** / **mapNotNull(...)** → transform Flow<T> in Flow<R>
- o **mapLatest(...)** → when the upstream flow emits a new value, computation of the previous transform is cancelled
- o **debounce(...)** / **sample(...)** → drop elements emitted too frequently

- `filter(...)` / `filterIsInstance(...)` / `filterNot(...)` / `filterNotNull(...)` → selectively emit elements according to the given predicate
- `flatMapConcat(...)` / `flatMapLatest(...)` / `flatMapMerge(...)` → apply a transformation that produces a new flow per each element and flatten it using different strategies
- `onStart(...)` / `onEach(...)` / `onEmpty(...)` / `onCompletion(...)` → perform a given action when the corresponding event happens
- `zip(...)` → zip value of the current flow with another one, applying a transformation
- ...
- **Terminal:**
 - `collect(...)` → collects the given flow with a provided action (if exception, it's rethrown)
 - `single()` / `first()` → await for the only/first element of the flow
 - `toList()` / `toSet()` / `toCollection()` → collect the flow elements in a structure
 - `fold()` / `reduce()` → accumulates values in the flow

⚠ **Flows and sequences** are similar, but flow is powered by a coroutine, while sequence is synchronous (operations in flows can be **suspended**, while sequences can only **block**; flows can be canceled, while sequences can't). Flows can add more items along the pipeline (using **transform** operator), while sequences need to produce collections that should be flatMapped.

All implementations of **Flow** interface must adhere to **2 key properties**:

1. **Context preservation** → flows preserve execution context: a flow should only emit values from the same coroutine (can't start a new one or switch thread or coroutine) [`flowOn(d: Dispatcher)` causes the upstream flow to be executed on the given dispatcher] [**ChannelFlow** encapsulates all the context preservation work, allowing collection and emission to happen in different coroutines]

```
fun main() = runBlocking {
    flow {
        emit("Emitted in ${
            Thread.currentThread().name
        }")
    }
    .flowOn(Dispatchers.IO)
    .map { it -> "$it\nProcessed in ${
        Thread.currentThread().name
    }" }
    .collect {
        println(it)
    }
}
```



```
Emitted in DefaultDispatcher-worker-1
Processed in main
```

2. **Exception transparency** → flows are transparent to exceptions (a downstream exception must always be propagated to the collector); flows can use the `catch(...)` operator to handle upstream exceptions (`onCompletion(...)` operator replaces **finally** block)

```
flow {
    emit(1); emit(2); emit(3)
}
.map {
    computeOne(it)
}
.catch { ... } // catches exceptions
                // in emitting data and computeOne
.map {
    computeTwo(it)
}
.collect {
    process(it)
} // throws exceptions from process and computeTwo
```


JavaRx **Observables** can be transformed into flows via the **.asFlow()** extension function (viceversa flows can be transformed into **Observable** via the **.asObservable()** extension function):

```
fun aFlow():  
    Flow<String> {  
    return Mono  
        .just("a")  
        .asFlow()  
    }
```

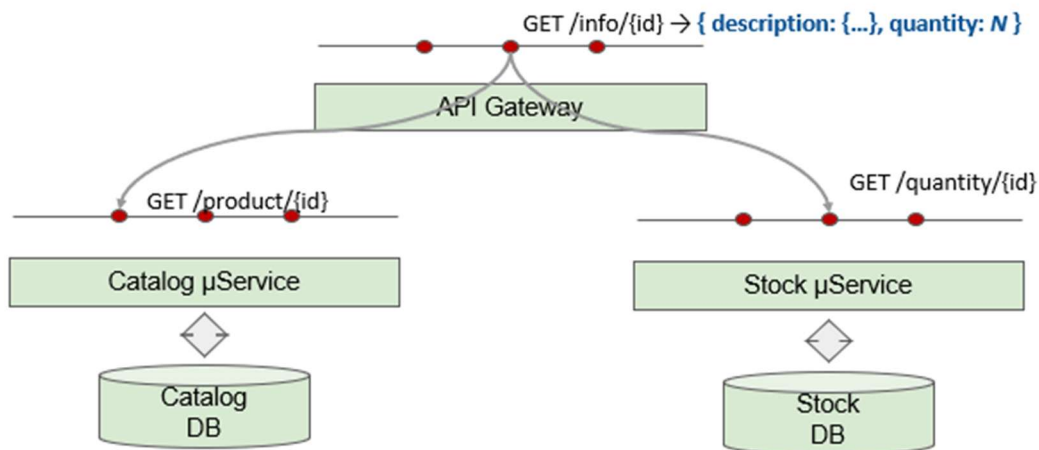
```
fun anObservable():  
    Observable<String> {  
    return flow {  
        emit("a")  
    }  
    .asObservable()  
    }
```

Kotlin Flow **backpressure** is based on suspension: when downstream is suspended, no collection is happening, thus suspending upstream as well. When **onflow(...)** is applied to a flow, upstream and downstream computations are separated by a channel.

To **integrate async** apps in K8s there are methods annotated with **@RequestMapping** and derived can be implemented as suspending functions:

- `fun f(): Mono<Void> → suspend fun f() // no results`
- `fun f(): Mono<T> → suspend fun f(): T // exactly one result`
- `fun f(): Mono<T> → suspend fun f(): T? // at most one result`
- `fun f(): Flux<T> → fun f(): Flow<T> // many results`

Querying microservices



```
@GetMapping("/info/{id}")  
suspend fun getAvailability(@PathVariable id: Int): InfoDTO = coroutineScope {  
    val product = async {  
        webClient.get().uri("/catalog-service/product/${id}")  
            .accept(APPLICATION_JSON)  
            .retrieve()  
            .awaitBody<ProductDTO>()  
    }  
    val quantity = async {  
        webClient.get().uri("/stock-service/quantity/${id}")  
            .accept(APPLICATION_JSON)  
            .retrieve()  
            .awaitBody<Int>()  
    }  
    return InfoDTO(product.await(), quantity.await())  
}
```

⚠ Coroutines don't make requests faster: they make **waiting cheaper**! They also reduce Pod count because effect of **async + coroutines**:

- Fewer threads ⇒ **smaller memory footprint**
- Smaller footprint ⇒ **lower pod memory requests**
- Lower requests ⇒ **more pods per node**
- More work per pod ⇒ **fewer replicas needed**

Async makes **readiness** meaningful and **liveness** quieter!

⚠ What happens on **shutdown/cancellation**?

1. Pod receives **SIGTERM**
2. Readiness flips to **false**
3. Grace period starts
4. Process must terminate cleanly

During shutdown, coroutines scopes tied to the app lifecycle are cancelled and cancellation propagates automatically → this means that in-flight requests complete quickly or are cancelled cooperatively (no orphan background tasks, no thread leaks, no partial writes...). So, cancellation isn't an error (it's a normal operational signal).

SSE (Server Sent Events) = the **Flow** abstraction makes it easy to produce server sent events (a server push technology enabling a client to receive automatic updates from a server via an HTTP connection). A controller may pack streams of data of type **T** in **ServerSentEvent<T>** objects and return it (these will be automatically transferred using the **ChunkedTransferEncoding**, labelled with **text/event-stream** mime type):

```
@RestController
class SSEController(val sseService: SSEService) {
    @GetMapping("/sse")
    fun sse(): Flow<ServerSentEvent<String>> = sseService.emitter.map {
        ServerSentEvent.builder<String>().event("sse").data(it).build()
    }
}
```

```
@Service
class SSEService {
    private val scope = CoroutineScope(Dispatchers.Default)

    val emitter: StateFlow<String> = flow {
        var count = 0L;
        while (true) { delay(1000); emit("e${++count}") }
    }.stateIn(scope = scope, started = SharingStarted.Eagerly, initialValue = "e0")
}
```

Spring Data R2DBC repositories: by using the *R2DBC Spring starter pack* and a non-blocking database driver, it is possible to use the Repository abstractions and **access relational data in a non-blocking way** (domain classes need to have a field labelled with **@Id** to identify the primary key, while **@Table** and **@Column** annotations can be used to instruct the object mapper when fetching and storing data). Repositories can be created extending a coroutine based interface (either **CoroutineCrudRepository** or **CoroutineSortingRepository**) [custom method can be added preceding them with **@Query(sql_string)** annotation].

```
@Table("users")
data class User(
    @Id val id: Long,
    val name: String,
    val email: String,
    val lastAccess: LocalDateTime
)
```

```
@Repository
interface UserRepository : CoroutineCrudRepository<User, Long> {
    @Query("""
        SELECT * FROM users
        WHERE last_access > :p1
        ORDER BY "last_access"
        """)
    fun findLatest(@Param("p1") p1: LocalDateTime): Flow<User>
}
```

Spring Data R2DBC transactions → a transactional operator was introduced to define transaction boundaries using a reactive stack (to overcome limitations of traditional transactions):

```
@Component
class LogRepository {
    private val client: DatabaseClient,
    private val operator: TransactionalOperator } {
    suspend fun updateUser(user: User, value: String) =
        operator.executeAndAwait {
            client
                .execute("""INSERT INTO log(user_id, activity)
                    VALUES (:user_id, :activity)""")
                .bind("activity", user.activity)
                .bind("user_id", user.id)
                .await()
            client
                .execute("""UPDATE users SET value = :value
                    WHERE id = :id""")
                .bind("value", value)
                .bind("id", user.id)
                .await()
        }
}
```

```
class PersonRepository {
    private val operator: TransactionalOperator
} {

    fun updatePeople() =
        findPeople()
            .map(::updatePerson)
            .transactional(operator)

    private fun findPeople(): Flow<Person> {
        // SELECT SQL statement
    }

    private suspend
    fun updatePerson(person: Person): Person {
        // UPDATE SQL statement
    }
}
```

⚠ **Common async anti-patterns** in the cloud: blocking inside suspend functions, mixing blocking JDBC with [WebFlux](#), [GlobalScope](#) misuse...

Cloud systems spend most of their time [waiting](#) and suspension makes waiting cheap; 2 async models:

- [Reactive streams](#) = a coordination protocol (how do async components talk safely)
- [Coroutines](#) = a programming model (how do I write async code I can reason about)

7) Anatomy of a K8s-native system

A **Kubernetes-native application** assumes ephemeral infrastructure, uses declarative intent, relies on continuous reconciliation and is designed for replacement (not repair). K8s isn't the platform, but the [coordination fabric](#). Recap:

Expect process death at any time

Assume ephemerality of instances

Kubernetes may...	Application must...	Kubernetes may...	Application must...
- Kill Pods during rescheduling - Evict Pods under resource pressure - Restart containers on liveness failure	- Handle SIGTERM gracefully - Shut down quickly and cleanly - Persist in-flight work or make it retryable	- Replace Pods instead of fixing them - Move workloads between nodes	- Treat instances as disposable - Externalize all important state - Avoid local disk dependence
Pattern	Anti-pattern	Pattern	Anti-pattern
Fast, idempotent cleanup	Long running shutdowns	External DB, object storage, queues	Writing critical state to the local filesystem

Do not assume stable identity

Be resilient to retries and duplicates

Kubernetes may...	Application must...	Kubernetes may...	Application must...
- Change Pod IPs - Replace Pod names - Recreate workloads during updates	- Use services for discovery - Avoid IP-based logic - Use logical identities, not physical ones	- Retry requests - Restart Jobs - Re-run controllers	- Make operations idempotent - Accept duplicate requests - Design safe retries
Pattern	Anti-pattern	Pattern	Anti-pattern
DNS-based service discovery	Hardcoded hostnames	At-least-once with deduplication	Assume exactly-once with side-effects

Tolerate partial availability

Declare health honestly

Kubernetes may...	Application must...	Kubernetes may...	Application must...
- Route traffic away from Pods - Delay scheduling - Lose nodes temporarily	- Provide graceful degradation - Implement retries, timeouts, and circuit breakers - Manage backpressure	- Restart broken Pods when liveness probes fail - Stop routing traffic when readiness probes fail	- Expose meaningful health endpoints - Fail readiness before failing completely - Separate startup, liveness, readiness probes

Pattern

- Bounded retries + fallbacks

Anti-pattern

- Blocking forever

Pattern

- Readiness reflex real availability

Anti-pattern

- Always return 200 from probe endpoints

Support horizontal scaling

Kubernetes may...

- Horizontally scale a heavy-loaded application
- Scale-down instances when traffic diminishes

Application must...

- Avoid in-memory shared state
- Make concurrency safe
- Support multiple active instances

Kubernetes may...

- Temporarily diverge from expressed intent
- Asynchronously perform state corrections

Application must...

- Not assume immediate propagation
- Avoid tight coupling across services
- Use asynchronous patterns

Pattern

- Stateless workers + coordination store

Anti-pattern

- Singleton logic

Pattern

- Event-driven or async messages

Anti-pattern

- Tight synchronous dependencies

Treat restarts as normal

Kubernetes may...

- Restart Pods automatically
- Roll Pods during upgrades

Application must...

- Provide a fast startup
- Guarantee idempotent initialization
- Safe re-entry

Kubernetes may...

- Expose events, status, metrics, but has no access to internal app state

Application must...

- Emit structured logs
- Export metrics
- Trace requests across restarts

Pattern

- "Run-on-every-start" initialization

Anti-pattern

- One-time init logic without guards

Pattern

- Guarantee observability in presence of process churn

Anti-pattern

- Assume process stability

Align observability with failure reality

There is a **separation of responsibilities**:

Concern	Owned by
Business logic	Application
Scaling & placement	Kubernetes
Recovery	Controllers
Configuration	Declarative APIs
Coordination	Control plane

Spring Boot support developing *K8s-native apps*, but it's **not magic**, it gives the right primitives to build apps that fit naturally into K8s. It provides production-grade **lifecycle hooks** via **Spring Boot Actuator**:

- Liveness checks (via [GET /actuator/health/liveness](#))
- Readiness checks (via [GET /actuator/health/readiness](#))
- Startup checks (based on liveness checks)
- Spring Boot will automatically report DOWN on readiness if key dependencies (e.g., DB) are down, and report UP when everything required is reachable
- The **ApplicationAvailability** API can be used to customize this behavior

Spring Boot does **not** enforce **statelessness**, but it encourages it: session state can be easily externalized via the **spring-session-*** dependency, and databases, caches and queues are supported. So, Spring Boot focuses only on app concerns, but doesn't intentionally manage pods, control deployments, replace K8s APIs and hide K8s concepts. You can customize app availability and invoke remote services with a **CircuitBreaker**.

⚠ There are **frameworks built explicitly for K8s** (e.g. **Quarkus**, **Micronaut**...) that are more tuned, but Spring Boot currently is more mature!

K8s-native Spring Boot checklist:

- Container and runtime fundamentals
- Health, readiness, liveness and startup behavior
- Graceful shutdown and lifecycle
- Configuration → externalize everything
- Stateless and scaling
- Networking → make it cloud-native
- Observability → metrics, logs, traces
- Security posture
- Build, release and deployment discipline
- Data layer and migrations
- Performance and reliability tuning

If all the the previous design guidelines have been followed, the following facts should be **true**:

- o **Pods can be killed** at any moment without breaking correctness
- o **Service scales horizontally** without coordination drama
- o **Readiness/liveness** probes reflect the true health
- o **Config** is externalized and environment-agnostic
- o **Metrics/logs/traces** tell you what's happening without SSH
- o **Rollouts** and **rollbacks** are routine, not events

A **K8s manifest template** for Spring Boot:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-service
  namespace: default
  labels:
    app: my-service
spec:
  replicas: 2
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: my-service
  template:
    metadata:
      labels:
        app: my-service
    annotations:
      # Prometheus scrape
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
      prometheus.io/path: "/actuator/prometheus"
    spec:
      serviceAccountName: my-service
      terminationGracePeriodSeconds: 45
      securityContext:
        seccompProfile:
          type: RuntimeDefault
      containers:
```

```

- name: my-service
  image: registry:32000/my-service:1.0.0
  imagePullPolicy: IfNotPresent
  ports:
    - name: http
      containerPort: 8080
  env:
    # Spring profile (optional)
    - name: SPRING_PROFILES_ACTIVE
      value: "prod"
    # Example: set timezone if needed
    - name: TZ
      value: "UTC"
  envFrom:
    - configMapRef:
        name: my-service-config
    - secretRef:
        name: my-service-secrets
  # Probes
  startupProbe:
    httpGet:
      path: /actuator/health/startup
      port: http
    failureThreshold: 30
    periodSeconds: 2
  readinessProbe:
    httpGet:
      path: /actuator/health/readiness
      port: http
    periodSeconds: 5
    timeoutSeconds: 2
    failureThreshold: 3
  livenessProbe:
    httpGet:
      path: /actuator/health/liveness
      port: http
    periodSeconds: 10
    timeoutSeconds: 2
    failureThreshold: 3
  resources:
    requests:
      cpu: "200m"
      memory: "512Mi"
    limits:
      cpu: "1"
      memory: "1024Mi"
  # Security hardening
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
  # Writable temp space if you need it
  # (some JVM/libs write temp files)
  volumeMounts:
    - name: tmp
      mountPath: /tmp
volumes:
  - name: tmp
    emptyDir: {}

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: default
  labels:
    app: my-service
spec:
  selector:
    app: my-service
  ports:
    - name: http
      port: 80
      targetPort: http
  type: ClusterIP
---
# Optional Ingress (adjust for your environment)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-service
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "60"
spec:
  rules:
    - host: my-service.example.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-service
                port:
                  name: http

```

Here a [Spring Boot application.yaml](#) template:

```

server:
  port: 8080
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,loggers
  endpoint:
    health:
      probes:
        enabled: true
        show-details: "never"
health:
  livenessstate:
    enabled: true
  readinessstate:
    enabled: true

```

```

metrics:
  tags:
    application: my-service
# If you want structured logging, prefer
# JSON encoder in logback configuration.
logging:
  level:
    root: INFO
    org.springframework.web: INFO

```

An **InitContainer** is a special type of container in a K8s Pod (init containers establish preconditions for the app):

- run before any app containers
- run to completion
- run sequentially
- must succeed before the main container starts
- if any **InitContainer** fails, the Pod is restarted until all of them succeed

Example of init container (populating the app config):

```

spec:
  volumes:
    - name: config
      emptyDir: {}

  initContainers:
    - name: generate-config
      image: alpine:3.19
      command:
        - sh
        - -c
        - |
          echo "server.port=8080" > /config/application.properties
      volumeMounts:
        - name: config
          mountPath: /config

```

```

containers:
  - name: app
    image: my-spring-app:1.0.0
    volumeMounts:
      - name: config
        mountPath: /config
    env:
      - name: SPRING_CONFIG_ADDITIONAL_LOCATION
        value: file:/config/

```

A **Sidecar** is a secondary container that runs in the same Pod as the main app container (sharing the same network namespace, volumes and lifecycle), used to extend the main container's functionality without changing its code (sidecars allow reducing thigh coupling between app and infrastructure) [common use cases are proxies, loggers, metrics adapters...]. But sometimes sidecars can be replaced by libraries:

Sidecar advantages	Library advantages
○ Language-agnostic ○ Independent upgrades ○ Centralized policy enforcement ○ No code changes required	○ Lower resource overhead ○ Less operation complexity

Example of sidecar (log forwarding):

```

apiVersion: v1
kind: Pod
metadata:
  name: app-with-logging
spec:
  containers:
    - name: app
      image: my-app:1.0.0
      volumeMounts:
        - name: logs
          mountPath: /var/log/app
    - name: log-forwarder
      image: fluent/fluent-bit:2.2
      volumeMounts:

```



```

- name: logs
  mountPath: /var/log/app
volumes:
- name: logs
  emptyDir: {}
# The application writes logs
# the sidecar ships them elsewhere

```

The **Ambassador** pattern is a **K8s design pattern** used to **externalize communication** concerns (such as connectivity, protocol handling, security and retries) from an app, by delegating them to a local proxy that runs alongside the app: here an **app container doesn't talk directly to remote services**, instead it talks to a local ambassador proxy (usually on **localhost**). It's a *special sidecar*, specialized for communication and performs also protocol translation (HTTP ⇌ gRPC, REST ⇌ legacy TCP, JSON ⇌ SOAP) and security (authN and authZ). Obviously pattern useful only when communication complexity outweighs these costs. **Example** of handling retries, timeouts, circuit breaking:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-service
spec:
  replicas: 2
  selector:
    matchLabels: { app: my-service }
  template:
    metadata:
      labels: { app: my-service }
    spec:
      containers:
        - name: app
          image: localhost:32000/my-service:1.0.0
          env:
            - name: USERS_BASE_URL
              value: "http://127.0.0.1:15001"
              # app calls envoy locally
          ports:
            - name: http
              containerPort: 8080

```

```

- name: envoy-ambassador
  image: envoyproxy/envoy:v1.30-latest
  args: ["-c", "/etc/envoy/envoy.yaml"]
  ports:
    - name: envoy
      containerPort: 15001
    - name: envoy-admin
      containerPort: 15000
  volumeMounts:
    - name: envoy-config
      mountPath: /etc/envoy
  resources:
    requests:
      cpu: "50m"
      memory: "64Mi"
    limits:
      cpu: "300m"
      memory: "256Mi"
  volumes:
    - name: envoy-config
      configMap:
        name: envoy-config

```

Adapter standardizes the output of an application (like logs or metrics) in a format expected by external systems before forwarding it (without changing the app itself). Like Ambassador, Adapter is a **special kind of sidecar** focuses on transformation of the output of the app (*normalization*). Adapter is used when integrating the app/platform in another, instead in domain behavior context it's better to **modify the app code**.

Then we have **ConfigMap** (that stores non-sensitive data) and **Secret** (that stores sensitive data); we can inject these setting *env variables*, mounting a read-only file (*volume mount*) or a combination of those. Secrets are **base64-encoded** (not encrypted), are **namespace-scoped** (not pod-scoped) and are **long-lived** (*auto-rotation*).

⚠ The mechanisms offered by Kubernetes align well with the way Spring Boot is configured: **env variables** (changes require pod to restart to take effect), **property files** (mounted files can update live) and **external config locations**.

Example (injecting config and secret in Spring Boot):

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-service-config
data:
  SPRING_APPLICATION_NAME: orders-service
  SPRING_PROFILES_ACTIVE: prod
  EXTERNAL_API_BASE_URL: https://api.example.com
  LOGGING_LEVEL_ROOT: INFO

```

```

apiVersion: v1
kind: Secret
metadata:
  name: orders-service-secrets
type: Opaque
stringData:
  DB_USERNAME: orders_user
  DB_PASSWORD: super-secret-password

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: orders-service
  template:
    metadata:
      labels:
        app: orders-service
    spec:
      containers:
        - name: orders-service
          image: reg:32000/orders-service:1.0.0
          envFrom:
            - configMapRef:
                name: orders-service-config
            - secretRef:
                name: orders-service-secrets

```

Spring Boot resolves values automatically from env variables:

```

spring:
  application:
    name: ${SPRING_APPLICATION_NAME}

external:
  api:
    base-url: ${EXTERNAL_API_BASE_URL}

spring:
  datasource:
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

```

application.yaml

The **metrics scraping and dashboard pattern** is a foundational observability pattern in K8s (standard way to expose metrics, store metrics centrally and visualize/alert on them via dashboards): this pattern is most commonly implemented as **Prometheus** (scrape + storage + query) + **Grafana** (dashboards + visualization) [+ optional AlertManager (alert routing)]. This works better because K8s is *distributed* (so only logs don't work).

Scraping is a **pull-based approach**: a metrics system (**Prometheus**) periodically pulls metrics from targets over HTTP (targets expose a **/metrics** endpoint in Prometheus format; Prometheus can discover targets dynamically). Spring Boot Actuator provides auto-configuration for Micrometer that supports numerous monitoring systems (including Prometheus) [common endpoint **/actuator/prometheus**].

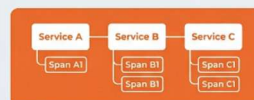
Distributed tracing is an observability practice that lets you follow a single request as it flows through multiple services, processes and network hops (in microservices and K8s envs, distributed tracing is essential for understanding runtime behavior). A **trace** is the full lifecycle of 1 request (e.g. Browser → Gateway → Orders → Payments → Inventory → DB), while a **span** is a timed operation within a trace (e.g. HTTP request handling, database query...). Spans form a tree and every request carries a **trace context** (that includes Trace ID and Span ID) that is *propagated automatically* between services.

Anatomy of a Distributed Trace



1. A Trace is Born

At the entry point (e.g., gateway), a new trace with a unique ID is created to track the request.



2. Services Add Spans

Each service involved in the request adds a "child span" for every significant operation it performs.



3. Context is Passed On

The trace's unique ID is propagated between services using HTTP headers or messaging metadata.



4. Spans Sent to Collector

All individual spans are exported from their respective services to a central collector.



5. Traces Visualized

The collector assembles spans into complete traces, which are stored and displayed in a tracing backend.

Signal

Metrics

Logs

Traces

Best for

Health, trends, alerting

Local details, debugging

End-to-end behaviour

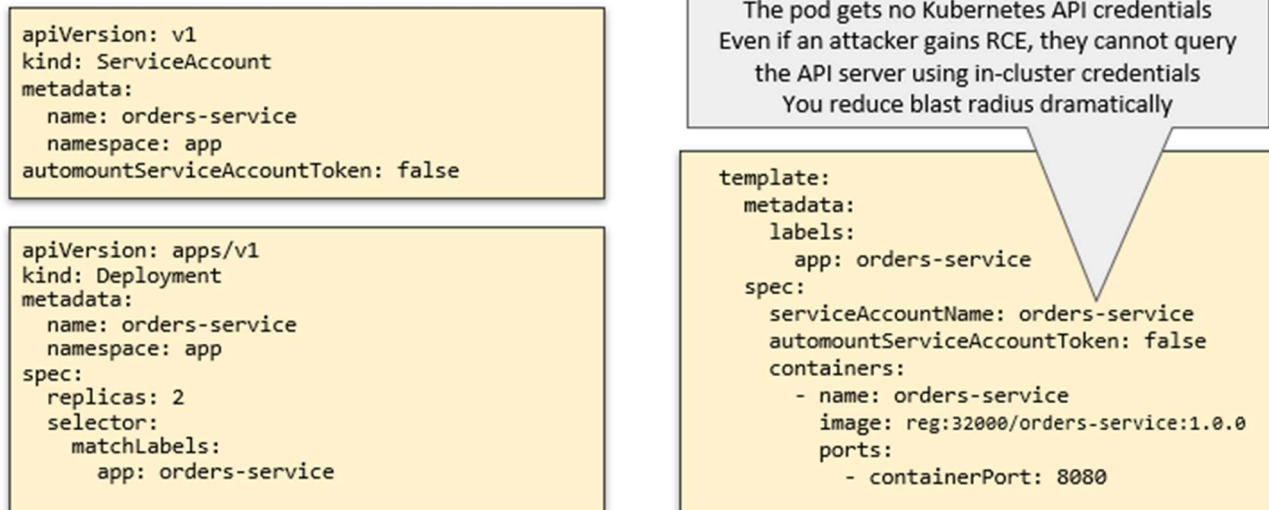
⚠ Metrics + Logs + Traces is the best (so *combination* of them, not single)!

Least Privilege pattern is foundational security principle stating that a system component should be granted only the minimum permissions it needs to perform its function (to limit possible damage). It's applied at multiple layers simultaneously (**identity** layer, **authZ** layer, **resource** layer, **runtime** layer).

This pattern doesn't need perfection on day one, but it's implemented incrementally; a possible [sequence of goals](#) are:

- separate ServiceAccounts
- remove cluster-wide roles where unnecessary
- lock down secrets access
- harden pod security contexts
- reduce cloud IAM scopes
- automate enforcement and review

So an app that **only perform standard actions** generally doesn't need to access Kubernetes API (as a consequence it's possible to provide it no privileges). **Example:**



Policy as Code (PaC) is the practice of defining, enforcing and auditing policies using [versioned](#), [testable](#) and [automated](#) code, rather than manual reviews. In this way, organizations scale governance without slowing delivery. A **policy** is a rule about what is allowed, required or forbidden (constraints on system behavior, but not business logic). PaC is enforced in *CI* (pre-merge), *admission* (pre-runtime) and *runtime* (continuously).

Batch and **periodic jobs** address a class of workloads that are finite, non-interactive and outcome-oriented, [rather than long-running request/response services](#). K8s provides Job and CronJobs to model these workloads correctly (jobs provide a structural pattern able to offload heavy or risky work from services, to enable clean operational boundaries and to reduce pressure on request path):

- **Job** = a task that runs on 1+ Pods, retries on failure and completes successfully or fails definitively;
- **CronJobs** = for recurring batch work → run **Jobs** on a **schedule** (Unix cron syntax)

Aspect	Service/Deployment	Job
Runtime	Long-lived	Finite
Traffic	Handles requests	No inbound traffic
Success	"Is it running?"	"Did it complete?"
Restart semantics	Always restart	Retry until success/failure
Scaling	Horizontal replicas	Parallel workers

So a **K8s Job** defines:

- what to run (container image + command)
- how many times it should succeed

- how to retry failures (`backoffLimit` = retry attempts before failing)
- when to give up
- `completions` = # of successful Pods required; `parallelism` = how many Pods may run concurrently

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-backfill
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: backfill
          image: my-app:1.0.0
          command: ["java", "-jar",
                    "app.jar", "backfill"]
```

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-reconciliation
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 5
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: reconcile
              image: my-app:1.0.0
              command: ["java", "-jar",
                        "app.jar", "reconcile"]
```

⚠ **concurrencyPolicy** = **Allow** (run overlapping jobs), **Forbid** (skip if previous is running), **Replace** (terminate old job and start the new one)

⚠ Jobs require observability just like services, but with different signals: important metrics are job duration, success/failure count, retry count, queue/backlog size. Logs are **primary** (centralized logging is essential)

Operator pattern is a K8s design pattern for **automating operations of complex, stateful or domain-specific systems by encoding human operational knowledge in code that runs inside the cluster**. An operator consists of **3 main elements**:

1. **CRD (Custom Resource Definition)** = defines a new API type in K8s that represents the domain concept, letting users express intent, not steps
2. **controller** = watches the custom resource, compares desired state to actual state and reconcile the difference (*reconciliation pattern* = observe → decide → act → repeat) [so operators are reactive, self-healing, resilient to restarts and idempotent by design]
3. **domain logic** = operational knowledge embedded in controller code

So operators manage systems **continuously**, not just at install time:

Approach	What is does well	Limitations
Helm	Install static resources	No runtime logic
Scripts	Automate tasks	Not continuously reconciled
Operator	Continuous, domain driven management	Complex to build

Let's see some operators:

- **CloudNativePG (CNPG)** = open-source operator that covers the full lifecycle of a highly available **PostgreSQL db cluster** with a primary/standby architecture, using native streaming replication. Steps to install the CNPG operator:
 1. install CNPG operator via Helm:

- a. `helm repo add cnpg https://cloudnative-pg.github.io/charts`
- b. `helm upgrade --install cnpg --namespace cnpg-system --create-namespace cnpg/cloudnative-pg`
2. create a namespace for the db: `kubectl create namespace data`
3. create and apply a credential Secret: `kubectl apply -f db-secret.yaml`
4. create a cluster CRD and apply it: `kubectl apply -f db-pg.yaml` (this creates stable Services and Secrets for the app [Secrets can be injected into the Spring Boot app])

```
# file db-secret.yaml

apiVersion: v1
kind: Secret

metadata:
  name: orders-db-app-secret
  namespace: data

type: kubernetes.io/basic-auth

stringData:
  username: orders_app
  password: "strong-password"
```

```
# file db-pg.yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: orders-pg
  namespace: data
spec:
  instances: 3

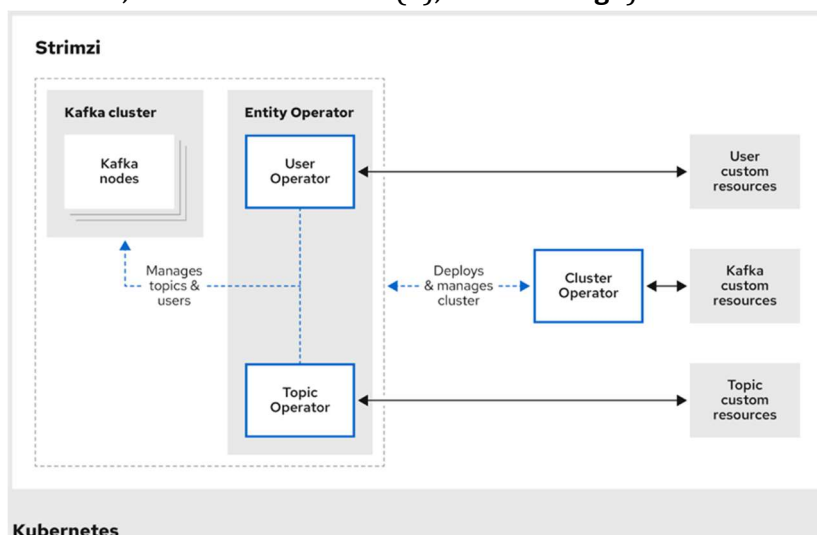
  storage:
    size: 20Gi

  bootstrap:
    initdb:
      database: orders
      owner: orders_app
      secret:
        name: orders-db-app-secret
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
  namespace: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: orders-service
  template:
    metadata:
      labels:
        app: orders-service
    spec:
      containers:
        - name: orders-service
          image: reg:32000/orders-service:1.0.0
```

```
env:
  - name: DATASOURCE_URL
    valueFrom:
      secretKeyRef:
        name: orders-pg-app
        key: jdbc-uri
  - name: DATASOURCE_USERNAME
    valueFrom:
      secretKeyRef:
        name: orders-pg-app
        key: username
  - name: DATASOURCE_PASSWORD
    valueFrom:
      secretKeyRef:
        name: orders-pg-app
        key: password
```

- **Strimzi** = allow the implementation and management of **Kafka clusters** (difficult without this operator because Kafka is stateful and operationally complex). Strimzi defines a set of CRDs that extend K8s making Kafka components appear as native objects (**Kafka**, **KafkaNodePool**, **KafkaTopic**, **KafkaUser**, **KafkaConnect**, **KafkaMirrorMaker**(2), **KafkaBridge**):



```

apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: my-cluster
  namespace: kafka
  annotations:
    strimzi.io/kraft: "enabled"
    strimzi.io/node-pools: "enabled"
spec:
  kafka:
    # Global Kafka settings for the whole
    cluster
    version: 3.7.0
    config:
      auto.create.topics.enable: "false"
      offsets.topic.replication.factor: 3
      transaction.state.log
      .replication.factor: 3
      transaction.state.log.min.isr: 2

```

```

listeners:
  - name: plain
    port: 9092
    type: internal
    tls: false
  - name: tls
    port: 9093
    type: internal
    tls: true
storage:
  # Default storage template
  type: persistent-claim
  size: 100Gi
  class: standard
entityOperator:
  topicOperator: {}
  userOperator: {}

```

A **Kafka cluster** consists of 1 or more node pools, all connected to a single parent Kafka resource (**KafkaNodePool** defines how many nodes it contains, which roles they serve, which storage they use, what config overrides apply). For each **KafkaNodePool**, Strimzi will generate a **StatefulSet** for the pool, a **PodDisruptionBudget**, a headless **Service** and **PVCs** for each replica [+ derived configs adapted to controller/broker roles].

⚠ The **KafkaTopic CRD** is managed by the Topic operator (which automatically creates, updates and deletes the topics in Kafka [name of the topic = metadata.name property]) [when CRD is deleted, also Topic is deleted]

7b) Frontend security

While backend runs inside a controlled server, frontend runs inside the user's browser, so a single XSS can undo everything the backend enforces (*frontend is first and last line of defence for the user*):

Backend concern	Frontend equivalent
SQL injection	XSS / DOM injection
CSRF tokens (Spring Security)	SameSite cookies / double-submit pattern
CORS filter on the server	Properly configured Access-Control-Allow-* in response
JWT validation middleware	Token verification + protected routes
Spring Security filter chain	AuthContext + loader guards
Secret in server env vars	Never: env vars are bundled into the JS build
Dependency CVE scanning	npm audit / dependabot

XSS (Cross-Site Scripting) → an attacker injects malicious JS into a page other users see and victim's browser executes it with full trust of the legitimate page (it can do session hijacking, credentials theft and keylogging). It can be:

- **stored XSS** → payload saved to db and re-served to every visitor
- **reflected XSS** → payload embedded in URL, reflected in response
- **DOM-based XSS** → client-side JS writes untrusted data to the DOM

React escapes all JSX expressions by default ({ **userInput** } is safe), but **when does React's protection break?**

```
// UNSAFE – renders raw HTML
<div dangerouslySetInnerHTML={{ __html: comment.body }} />

// SAFE – sanitize before rendering
import DOMPurify from "dompurify";
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(comment.body) }} />

// SAFE – validate URL scheme before using as href
const safeUrl = url.startsWith("javascript:") || url.startsWith("data:") ? "#" : url;
```

⚠ Common mistakes XSS:

- using **dangerouslySetInnerHTML** without sanitizing
- passing user-controlled strings to **eval()**, **new Function()**, **setTimeout()**
- trusting **href** / **src** from user input or URL params without validation
- including third-party scripts without Subresource Integrity (SRI) → **<script src="..." integrity="sha256-..." crossorigin="anonymous" />**

CSRF (**Cross-Site Request Forgery**) → a malicious site tricks the browser into making an authenticated request (only affects cookie-based authN). **SameSite** cookie attribute is the most effective modern defence:

- **SameSite=Strict**: never sent cross-origin (may break OAuth redirects)
- **SameSite=Lax** (*default* for browser): sent on top-level navigation, not sub-resources; dangerous if the target server accepts GET-mutating endpoints (which should not)
- **SameSite=None; Secure**: explicitly opts in to cross-site sending

SOP (**Same-Origin Policy**) blocks JS from reading cross-origin responses (origin = scheme + host + port) [e.g. <https://api.acme.com:443>] because the browser blocks the response (the request is still sent).

CORS lets servers opt in to cross-origin reads **via response headers** (CORS doesn't protect the server → it protects users from data leaks, the server already received + processed the request before CORS fires):

Header	Purpose	Watch out for
Access-Control-Allow-Origin	Which origin(s) may read the response	Never use * for authenticated APIs
Access-Control-Allow-Credentials	Allow cookies/auth header cross-origin	Requires specific origin, not*
Access-Control-Allow-Methods	Allowed HTTP methods (preflight reply)	Must include all methods used by SPA
Access-Control-Allow-Headers	Allowed custom headers (preflight reply)	Include Content-type, Authorization
Access-Control-Max-Age	How long to cache the preflight result	Low value = many OPTIONS requests

OAuth2/OIDC is used when a **SPA** (a public client which can't keep a secret) authenticates with a third-party IdP. **PKCE** (**Proof Key for Code Exchange**) proves the entity requesting the token is the same that started the flow (SPA plays the role of OAuth2 Client) [**PKCE is now required for all OAuth2 clients**].

Steps:

- before the login process starts, the SPA generates a random unique string called **code verifier** (it then creates a code challenge by applying hash to verifier; both are stored only in memory, never sent together);
- then SPA redirects the browser to the **IdP** (Identity Provider) for login;
- the user authenticates with their credentials (IdP saves the code challenge and associates it with the resulting authZ code);
- once user is redirected back, client sends a **request** to IdP's token endpoint (which must include both authZ code and the original unhashed code verifier);

- IdP hashes the provided code verifier and **compares** it to the code challenge received before → if match, IdP confirms the client is the same that initiated request and issues the access token.

AuthN state in React must be **global**: it contains the current user, token, any loading or error flags; protected routes check authN in the React Router loader → **AuthContext** pattern:

```
// auth/AuthContext.tsx
type AuthState = {
  user: User | null;
  token: string | null
};
const AuthContext = createContext<AuthState>({ user: null, token: null });

export const AuthProvider = ({ children }: PropsWithChildren) => {
  const [state, dispatch] = useReducer(authReducer, initialState);
  useEffect(() => {
    dispatch(restoreSession());
  }, []);
  return (
    <AuthContext.Provider
      value={state}>
      {children}
    </AuthContext.Provider>
  );
};
```

⚠ Store access tokens in memory, refresh tokens in httpOnly cookies → **never JWTs in localStorage!**
Short-lived access tokens limit blast radius: even if stolen, they expire quickly!

CSP (Content Security Policy) is the 2nd layer of XSS defence (injected scripts can't execute if origin not listed) [**Content-Security-Policy: default-src 'self'; script-src 'self' cdn.example.com**]:

Directive	Controls	Typical value
default-src	Fallback for all resource types	'self'
script-src	Which origin may serve JavaScript	'self' 'nonce-...'
style-src	CSS origins	'self' fonts.googleapis.com
connect-src	fetch / XHR / WebSocket destinations	'self' https://api.example.com
img-src	Image origins	'self' data: blob:
frame-ancestors	Who may embed this page in an <iframe>	'none'

GET / HTTP 1.1
Host: some.host.com
Connection: Close

↓

HTTP/1.1 200 OK
Content-Type: text/html
Content-Security-Policy: script-src 'nonce-3b7f546d'

```
<html>
<head>
  <script nonce="3b7f546d">dohatever()</script>
</head>
<body>
  ...
</body>
</html>
```

Trusted Types turns “remember to sanitize” into a compile-time guarantee enforced by browser: it’s a structural fix for DOM-based XSS (so when JS writes attacker-controlled strings into injection sinks), because **makes the browser refuse strings at sinks** (sinks accept only **TrustedHTML**, **TrustedScript**, **TrustedScriptURL**). It’s enabled via CSP (browser enforces the rule, not the app code) [**Content-Security-Policy: require-trusted-types-for 'script'; trusted-types app-policy**].

⚠ **React supports Trusted Types natively** (**dangerouslySetInnerHTML** accepts **TrustedHTML** directly):

```
// 1. Declare the policy (only allowed names from CSP)
const policy = trustedTypes.createPolicy("app-policy", {
  createHTML: (input) => DOMPurify.sanitize(input),
});

// 2. Use it at the sink
el.innerHTML = policy.createHTML(userInput); // ✓ accepted

// 3. CSP header (set by backend)
// Content-Security-Policy:
//   require-trusted-types-for 'script';
//   trusted-types app-policy;
```


⚠ Trusted Types is "defence in depth" layer that turns a single XSS bug from a breach into a **TypeError**!

⚠ **Additional threats** (treat **JS bundle** as published source code: **anything in it is visible to the world**):

- Secrets leakage via env variables (never store API keys, passwords or signing secrets in frontend env vars) [**VITE_*** and **REACT_APP_*** variables are inlined into the JS build at compile time]
- Supply chain attacks via **npm** packages (a compromised package can exfiltrate tokens, inject scripts, or steal credentials) → run **npm audit** regularly
- Client-side validation is UX, not security: **all validation must be repeated on server** (always!!!) → validate twice, enforce once → **client is UX, server is truth**

⚠ **HSTS enforce HTTPS at browser level!** (HSTS tells browsers: *This domain is HTTPS-only for the next N seconds*) [**max-age=0** accidentally disables HSTS and tells the browser to forget the policy]

Production security checklist

Check	Passing condition
Token storage	Access tokens in memory; refresh tokens in httpOnly cookie
Dangerous HTML	Always wrap into DOMPurify.sanitize() calls
CORS origin	Access-Control-Allow-Origin lists specific origins, not *
Session cookie	SameSite=Lax (or Strict) and Secure attributes set
CSP header	Content-Security-Policy present; no unsafe-inline in script-src
Frontend secrets	No API keys or credentials in VITE_*/REACT_APP_* vars
Input validations	All client-side rules duplicated on server
Dependencies	npm audit clean; lockfile committed; Dependabot enabled

8) Platform engineering, CI/CD and automation

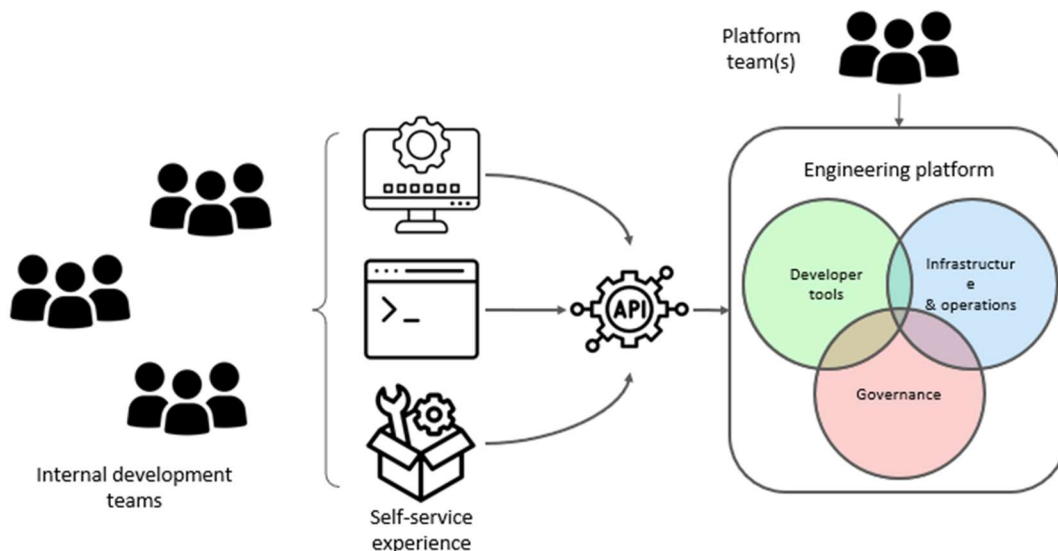
Increasing scale of a service (e.g. European-wide digital payments):

- **Infrastructure complexity** grows faster than team capacity
- **Operational knowledge** becomes fragmented
- **Toolchains** diverge across teams
- **Cognitive load** exceeds what individual can manage safely

Platform Engineering is the discipline of building and operating internal platforms that **enable application teams to deliver, run and evolve sw safely at scale**. Platform Engineering is a response to coordination limits, not tech limits. A **platform** is an abstraction layer between teams and infrastructure, providing:

- standardized ways to build and deploy software
- pre-configured operational capabilities
- guardrails for security, reliability and cost
- self-service access to common infrastructure needs

A platform doesn't: own business logic, replace app design or eliminate need for operational thinking:



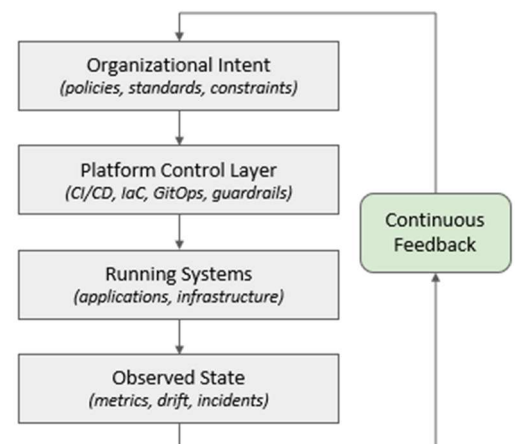
Platforms abstracts **how**, not **what**:

- **abstract** → infrastructure provisioning, deployment mechanics, security baselines, observability integration, operational guardrails
- **don't abstract** → app semantics, business logic, failure consequences

⚠ **DevOps** defines how teams should work, **platforms** define how systems should behave!

A platform turns organizational intent into operational behavior through continuous feedback and automation. Key properties of the control loop are:

- **Declarative**: platform encodes what should be true, not step-by-step procedures
- **Continuous**: enforcement does not stop after deployment
- **Observable**: failures, drift and violations aren't hidden
- **Risk-oriented**: goal is safe change and fast recovery, not max deployment speed



CI/CD pipelines are control loops (not scripts) that implement a control loop managing change risk.

Declarative pipeline specifies (in a **YAML** [e.g. GitHub Actions]):

- **stages** and **required outcomes** (e.g. "unit tests must pass")
- **artifacts** that flow between stages (images, manifests...)
- **policies/constraints** (branch protections, required approvals...)
- **triggers** (on PR, on main merge...)
- **environments** (dev → staging → prod)

Declarative pipeline use a set of **reusable templates** that prevent reinventing the wheel (*platform engineering*). Policy-as-code says what must be true (required test coverage thresholds, container image scanning thresholds...). Delivery controller does the **reconciliation loop** (CI produces artifacts, CD reconciles env to desired state).

⚠ **CI/CD optimizes** for safe change, not fast change (**reduce**: undetected defects, config drift, security regressions, human errors; **increase**: confidence in change, predictability, mean time to recovery). **Bad platforms** retry blindly, hide failures and push risk downstream into production! Goal is not to fix the single pipeline, but **fix the risk model**!

⚠ **Good pipelines** have some properties: idempotency, observability, policy-driven, separation of concerns, designed for failure

GitOps is the delivery specialization of the same control-loop model that platforms and CI/CD implement; it's a practice where:

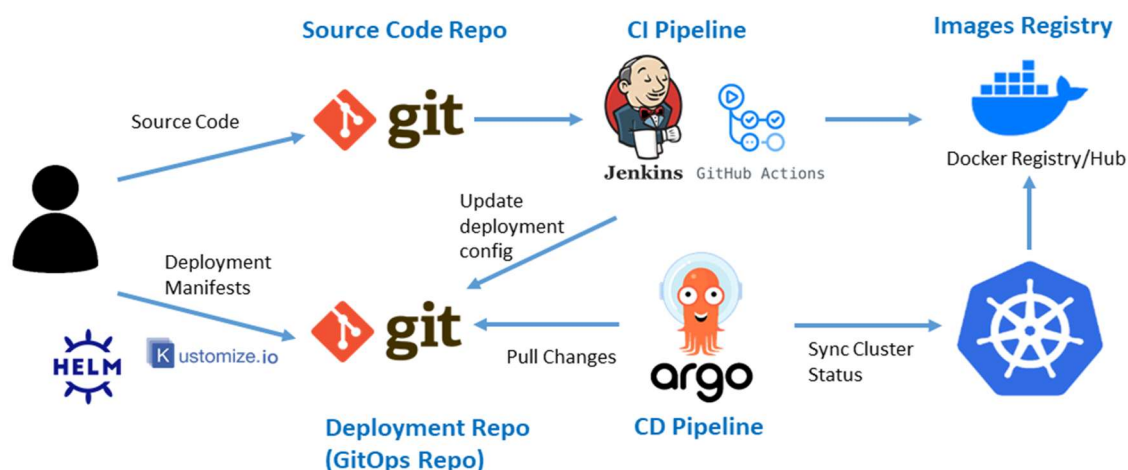
- desired state of systems (app, infrastructure, config) is **declared in Git**
- changes are made only by **modifying Git**
- automated agents continuously **reconcile** the actual state **to match Git**
- **drift** between desired and actual state is detected and corrected automatically

GitOps mirrors the same control-loop pattern of K8s controllers, but it's versionable. But GitOps **trades flexibility for predictability**!

ArgoCD is an **example of GitOps continuous delivery controller for K8s**: it continuously compares the desired state stored in Git with the actual state of a K8s cluster and reconciles differences automatically.

Architecture:

- **Kubernetes-native controller** (runs inside the cluster)
- **Git repository access** (read-only by default)
- **Application CRDs** describe what to reconcile
- **Optional UI/API** for visibility and audits



It does:

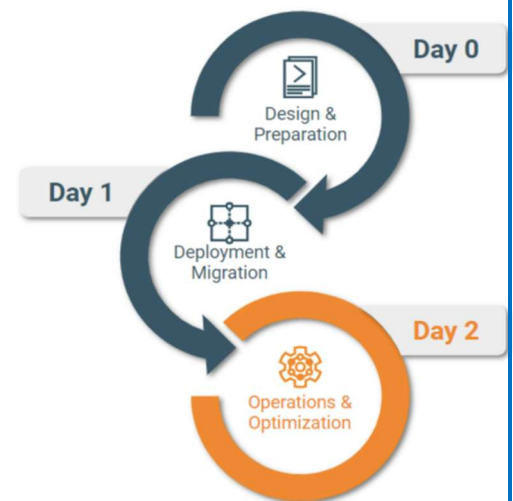
- **Continuous reconciliation**: drift is detected and corrected automatically
- **Declarative delivery**: you describe what should run, not how to deploy it
- **Clear visibility**: live diff, sync status, health per application
- **Deterministic rollback**: revert a Git commit → system converges back
- **Separation of concerns**: CI builds artifacts; Argo CD deploys declared state

9) Operations & Lifecycle management

Most software education stops at deployment: in the cloud software is **continuously** changing, running and failing (partially); so a cloud system is never “done”. Software is no longer a product, but it’s a **process executing over time**.

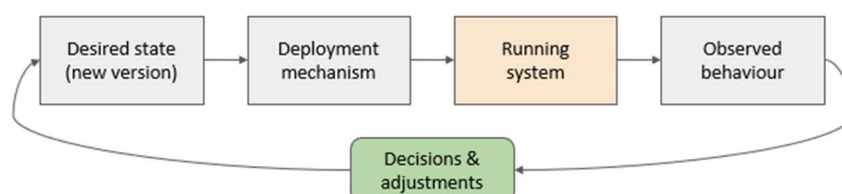
So operational concerns begin before the system exists and continue for its entire lifetime → the **Day 0 / Day 1 / Day 2 model** exists to enforce this:

- **Day 0** → **design for operability**: happen before the system runs:
 - architectural decisions
 - technology choices
 - deployment models
 - failure assumptions
 - automation strategies
- **Day 1** → **bring system to life**: initial delivery:
 - first deployments
 - env provisioning
 - config and secrets setup
 - initial monitoring and alerting
 - validation that the system works as intended
- **Day 2** → **living with the system**: dominate system’s lifetime (if system fail or succeed):
 - continuous deployments
 - scaling and performance tuning
 - incident response
 - debug and recovery
 - security patching
 - cost management
 - compliance and audits



But these days don't finish: Day 0 decisions are revisited during incidents, Day 1 activities happen with every new service, Day 2 feedback reshapes architecture and automation. So, they form a **continuous feedback loop**. **Operations shape architecture!**

So in cloud systems, **deployment never truly finishes**: it's a process with a duration, a risk window and a feedback loop; deployment success is measured by observed behavior (not by pipeline completion):

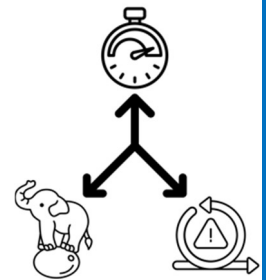


Every **rollout** is an *experiment* (rollout exist to limit blast radius, gather feedback early, preserve recovery options).

Rollout strategy	Risk profile	Feedback speed	Recovery complexity	When it tends to fail
All-at-once	High - entire system exposed simultaneously	Fast - issues appear immediately	High - rollback affects whole system	Tight coupling, irreversible changes, slow rollback paths
Rolling update	Medium - failures limited to subsets	Medium - issues emerge progressively	Medium - requires version compatibility	Incompatible versions, stateful components, uneven traffic
Blue/green	Low (initially) - one environment isolated	Fast - full traffic switch reveals issues quickly	Low/Medium - switch back is simple if data allow	Data divergence, hidden external dependencies
Canary	Lowest - very small blast radius	Slow/Medium - depends on signal quality	Low - limited scope to revert	Poor observability, noisy metrics, rare failure modes

A **rollback** is only possible if it was designed in advance. **Drift** is **not a mistake** (it's a property of running system) and its sources are manual interventions, partial failures, hotfixes, env-specific differences... In fact, **declarative systems** (IaC, K8s, GitOps) **don't prevent drift**: they make drift visible and correct it continuously. **Incidents are also architectural feedback** (lifecycle closes when incidents change how systems evolve).

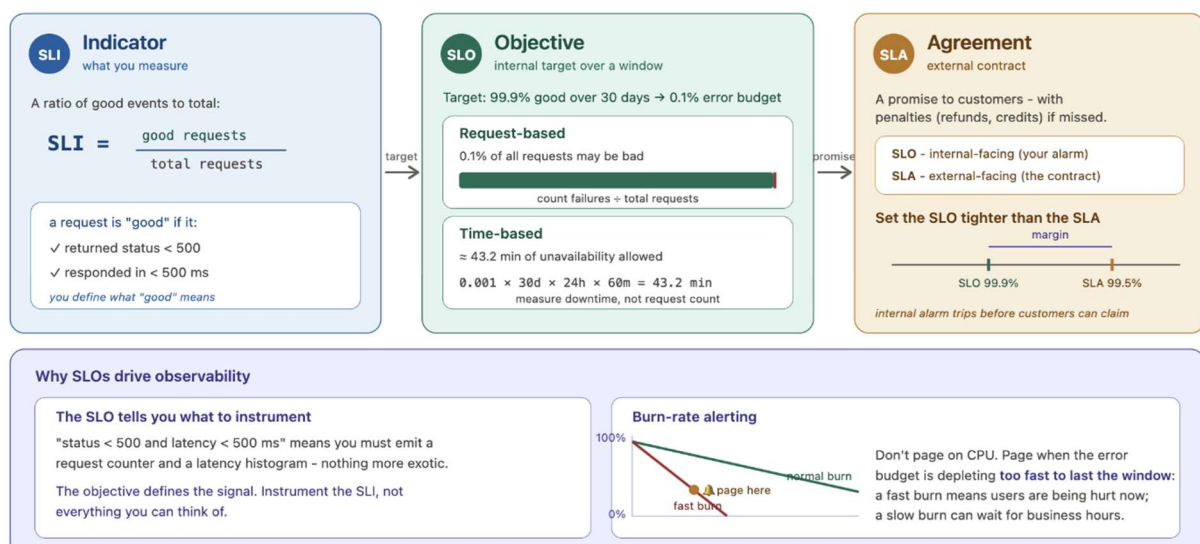
Every system chooses a point in the space of **velocity** (speed of change), **stability** (predictability) and **risk tolerance** (blast radius).



10) Observability, Monitoring and Optimization

Monitoring tells you whether the system is working, **observability** lets you ask why if it's not working. We define:

- **SLI** (Service Level Indicator) = $\text{goodEvents} / \text{totalEvents}$
- **SLO** (Service Level Objective) = target value of an SLI over a window → this matter for observability (define which signals matter, measure SLI)
- **SLA** (Service Level Agreement) = contractual commitment with penalties (SLA is external-facing, SLO is internal-facing)



5 failure modes that matter in distributed systems:

1. **Crash failure** = component stops responding entirely (easy to detect)
2. **Omission failure** = component drops messages silently (hard to detect)
3. **Timing failure** = response arrives, but too late (SLO breach without error)
4. **Byzantine failure** = component sends incorrect data (hardest to detect)
5. **Partial failure** = some replicas fail, others continue (**most common in cloud**)

The **3 common telemetry signals** are:

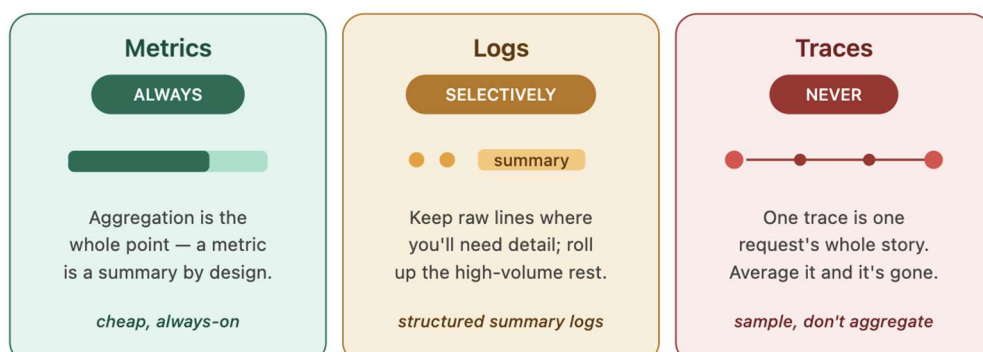
- **Metrics** → **what is happening**: a metric is a named time-stamped numerical value with labels (e.g. `http_requests_total { method="GET", status="200" }`). Types: *counter* (monotonically increase), *gauge* (current value), *histogram* (distribution across buckets). Metrics are important because are the cheapest signals to store and query (natural fit for SLO burn-rate calculations) and are the best signals for alerting and autoscaling decisions (but poor for reconstructing individual requests). **Prometheus** is the *standard* in cloud-native stacks (pull-based scraping, PromQL query language, stores data in local time-series database)
- **Logs** → **what happened exactly**: timestamped records of events (structured [JSON] or unstructured [plain text]). Best practice is using structured logs with consistent fields (`trace_id`, `service`, `level`, `message`, `timestamp`) and enabling log-trace correlation (e.g. **Loki** + **Tempo**)
- **Traces** → **how did the request flow**: **causal chain of spans** (**span** = **each operation within a trace**) **across services**; a trace follows a single request through all services it touches. Every service must propagate trace context. **OpenTelemetry** is standard for instrumentation, context propagation and telemetry export. **Grafana Tempo** stores traces in object storage (cheap and unindexed)

Grafana observability stack:

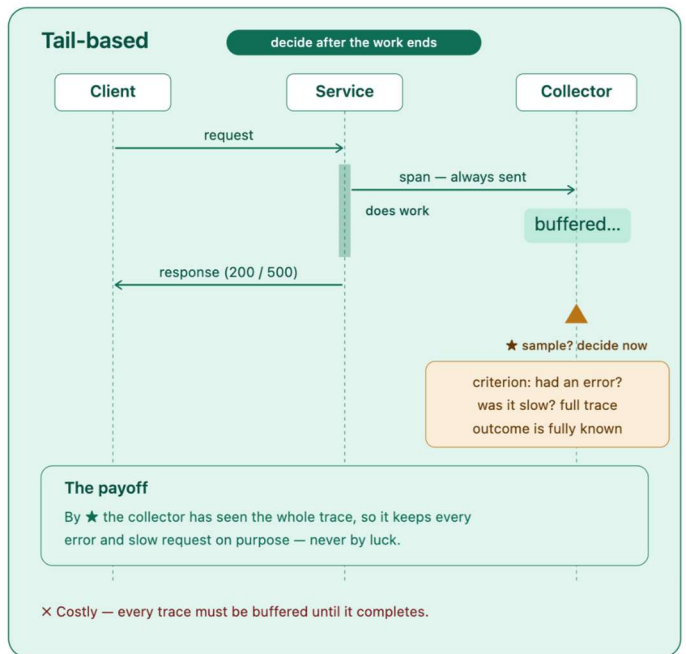
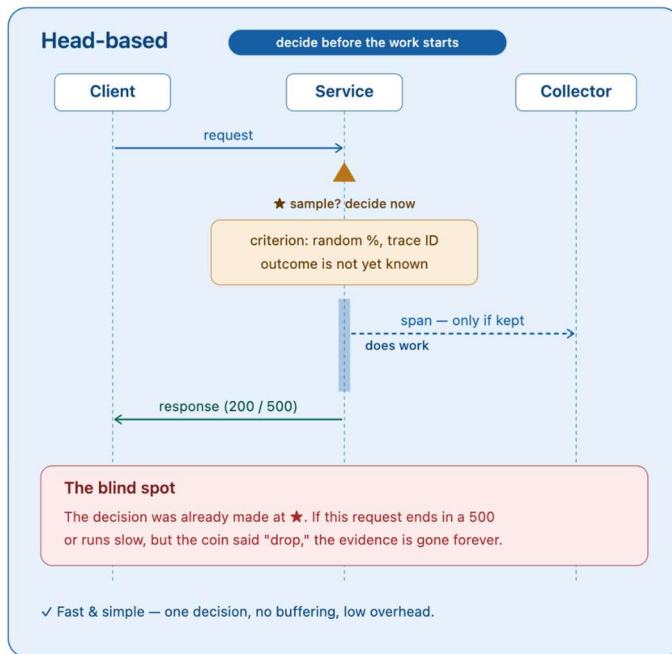
Feature	Tempo	Loki	Prometheus
Purpose	Distributed tracing	Log aggregation	Metrics monitoring
Data type	Distributed traces (spans)	Structured or unstructured logs	Structured, numerical metrics
Query language	TraceQL	LogQL	PromQL
Use case	Request flow analysis, debugging	Log search and correlation	Performance monitoring, alerting
Storage	Object storage (unindexed)	Object storage (label-based indexing)	Local/remote time series database

Cardinality is **# of unique label combinations** for a metric: each unique label combination is a separate time-series; storage, ingestion and query cost scale with cardinality (high cardinality labels can destroy a metrics system).

But **aggregating** makes you lose single events (because it's summarizing). **When to aggregate:**



Sampling → what you lose when dropping events:



Designing observability:

- **USE method (for infrastructure)**

- Utilization (% of time in use)
- Saturation (work queue)
- Errors

- **RED method (for services)**

- Rate (*requests/s*)
- Errors (*failed requests/s*)
- Duration (distribution of request latency)

Failure mode	Metric	Log	Trace
Payment API slow	dependency latency histogram	payment timeout event	checkout trace span
DB connection pool exhausted	pool utilization / wait-queue depth	connection failed event / acquire-timeout error	span blocked on connection acquire
Slow N+1 query pattern	DB query count per request	repeated near-identical query log lines	traces with dozens of sequential small DB spans
TLS / cert expired	handshake-failure rate / connection-error count	certificate expired / verification failed	client span failing during connection establishment

⚠ Autoscalers react to metrics (so if metric is wrong, so is the reaction): **autoscaling** should protect SLOs, not only resource utilization

Metric

aggregate behavior — "is something wrong?"

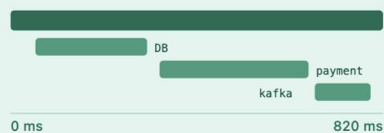


Cheap, always-on, pre-aggregated.
Tells you the shape and scale of a problem across all requests.

```
checkout_failures_total
checkout_request_duration_seconds
```

Trace span

request path & timing — "where is it slow?"



One trace = one request's full journey.
Nested spans show which dependency ate the time.

```
span: payment.authorize
duration: 540 ms · error=true
```

Structured log

precise context — "what exactly failed?"

```
{
  "level": "ERROR",
  "order_id": "A-7741",
  "reason": "PAYMENT_DECLINED",
  "trace_id": "4bf92f..."
}
```

The exact event, with the fields you need to understand it — error reason, order ID, stack trace.

Queryable fields, not prose — filter, group, and pivot on them.

Correlation ID — the thread through all three

```
trace_id = 4bf92f...
```

A failure in the **metric** → filter **logs** by reason → each log carries the id → open the exact **trace** and see which span failed. One ID turns three silos into one story.

Without correlation, you have three disconnected datasets. With it, you have an investigation.

Telemetry design example:

Service: checkout API

- SLO: 99.9% successful checkout under 800 ms

Metrics

- `checkout_requests_total`
- `checkout_request_duration_seconds`
- `payment_api_duration_seconds`
- `checkout_failures_total`

Logs

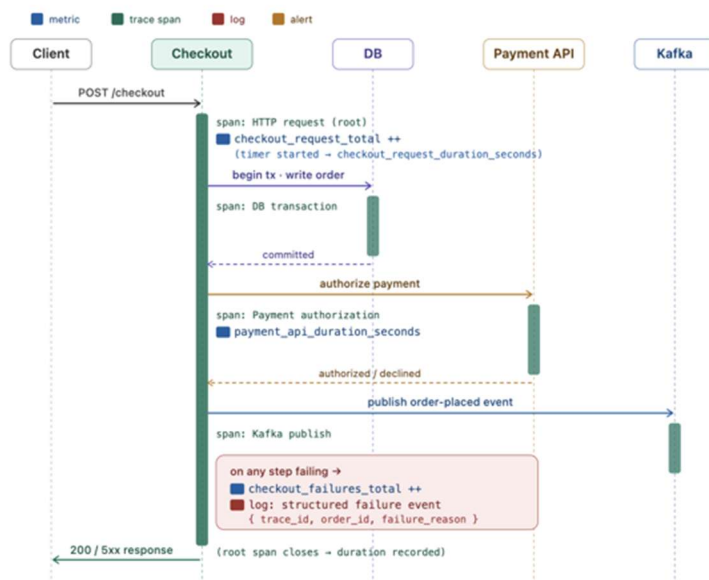
- Structured failure event with `trace_id, order_id, failure_reason`

Traces

- HTTP request span
- DB transaction span
- Payment authorization span
- Kafka publish span

Alert

- Page on checkout SLO burn rate, not CPU usage



```
@Service class CheckoutService {
    private val registry: MeterRegistry,
    private val orderRepository: OrderRepository,
    private val orderEventPublisher: OrderEventPublisher,
    private val tracer: Tracer,
    private val paymentClient: PaymentClient,
} {
    private val log = LoggerFactory.getLogger(javaClass)

    private val requestCounter: Counter = Counter.builder("checkout_requests_total")
        .description("Total checkout requests")
        .register(registry)
    private val failureCounter: Counter = Counter.builder("checkout_failures_total")
        .description("Failed checkouts")
        .register(registry)
    private val requestTimer: Timer = Timer.builder("checkout_request_duration_seconds")
        .description("End-to-end checkout latency")
        .publishPercentiles(0.5, 0.95, 0.99)
        .register(registry)
    private val paymentTimer: Timer = Timer.builder("payment_api_duration_seconds")
        .description("Payment authorization latency")
        .publishPercentiles(0.5, 0.95, 0.99)
        .register(registry)

    //continues...
```

```
fun checkout(command: CheckoutCommand): CheckoutResult {
    requestCounter.increment()
    val sample = Timer.start(registry) // checkout_request_duration_seconds starts
    val order = Order.from(command)
    try {
        persistOrder(order)
        val auth = authorizePayment(order)
        publishOrderPlaced(order, auth)
        return CheckoutResult.success(order.id, auth.authId)
    } catch (e: CheckoutException) {
        // failure metric + structured log carrying the correlation IDs
        failureCounter.increment()
        logFailure(order, e.reason, e)
        throw e
    } catch (e: Exception) {
        failureCounter.increment()
        logFailure(order, FailureReason.UNKNOWN, e)
        throw e
    } finally {
        sample.stop(requestTimer) // root duration recorded on every path
    }
}
// continues...
```



```
// ---- DB transaction span ----
private fun persistOrder(order: Order) {
    val span = tracer.nextSpan().name("db.transaction").start() //create a child span from current context
    try {
        tracer.withSpan(span).use {
            span.tag("db.operation", "insert_order")
            span.tag("order.type", order.type)
            orderRepository.save(order) // @Transactional on the repository/method
        }
    } catch (e: Exception) {
        span.error("persistence.exception")
        throw CheckoutException(FailureReason.DB_WRITE_FAILED, e)
    } finally {
        span.end()
    }
}

//continues...
```

```
// ---- Payment authorization span (+ payment_api_duration_seconds) ----
private fun authorizePayment(order: Order): AuthResult {
    val span = tracer.nextSpan().name("payment.authorize").start()
    return try {
        tracer.withSpan(span).use { // record() captures latency for the payment dependency specifically
            paymentTimer.record<AuthResult> {
                paymentClient.authorize(order)
            }
        }.also { auth -> if (!auth.approved) {
            span.tag("payment.declined", "true")
            throw CheckoutException(FailureReason.PAYMENT_DECLINED)
        }
    } catch (e: CheckoutException) {
        span.error(e)
        throw e
    } catch (e: Exception) {
        span.error(e)
        throw CheckoutException(FailureReason.PAYMENT_ERROR, e)
    } finally {
        span.end()
    }
}
```

```
// ---- Kafka publish span ----
private fun publishOrderPlaced(order: Order, auth: AuthResult) {
    val span = tracer.nextSpan().name("kafka.publish").start()
    try {
        tracer.withSpan(span).use {
            span.tag("messaging.destination", "order-placed")
            orderEventPublisher.publish(OrderPlacedEvent(order.id, auth.authId))
        }
    } catch (e: Exception) {
        span.error("kafka.error")
        throw CheckoutException(FailureReason.EVENT_PUBLISH_FAILED, e)
    } finally {
        span.end()
    }
}

// ---- Structured failure log with trace_id / order_id / failure_reason ----
private fun logFailure(order: Order, reason: FailureReason, e: Exception) {
    val traceId = tracer.currentSpan()?.context()?.traceId() ?: "none"
    log.atError().setMessage("checkout failed")
        .addKeyValue("trace_id", traceId).addKeyValue("order_id", order.id)
        .addKeyValue("failure_reason", reason.name).setCause(e).log()
}

}
```

Optimization → **cost, reliability and feedback loop you should fear**; cost drivers in cloud-native systems are:

- *Compute* (CPU and memory allocated)
- *Network* (egress, cross-zone, cross-region traffic)
- *Storage* (object storage, time-series, logs retention)

- **Observability** (high-cardinality metrics, verbose logs, trace storage)

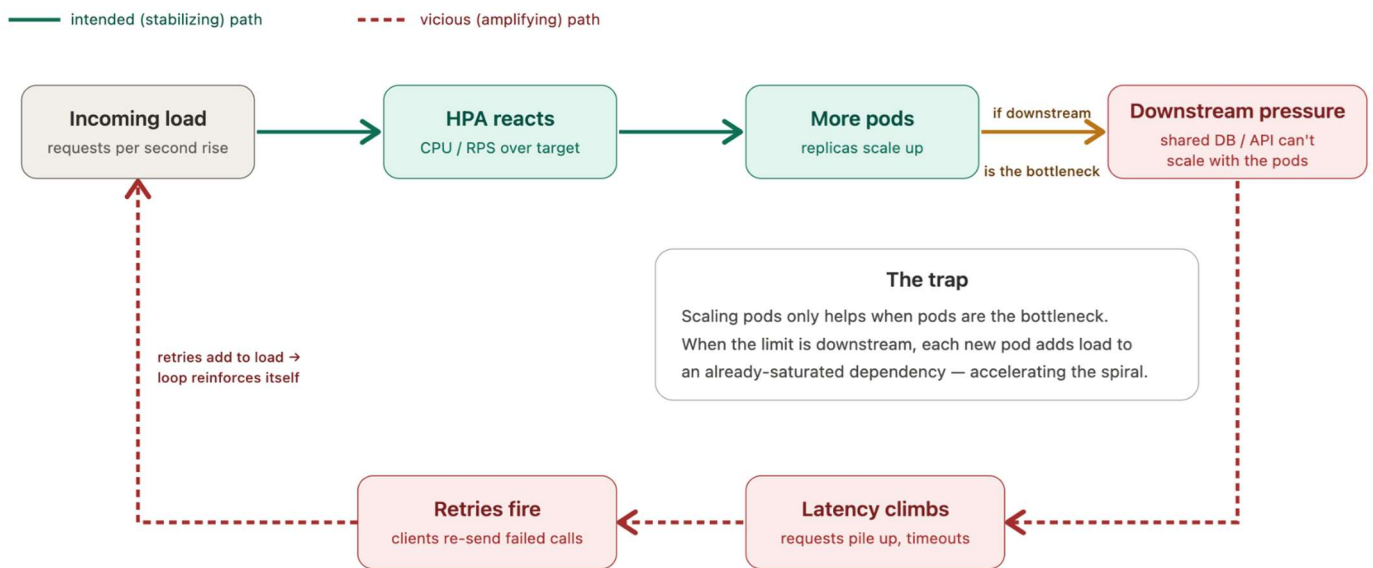
⚠ **Cost optimization without observability** is guessing! **Latency optimization** can shift cost elsewhere!

⚠ **K8s provides autoscalers** that operate on different axes:

1. **HPA** (**Horizontal Pod Autoscaler**) = **adds/removes pod** replicas based on a metric (CPU, custom): **reacts to load** (more traffic → more pods)
2. **VPA** (**Vertical Pod Autoscaler**) = adjust CPU/memory requests per pod on observed usage (requires pod restart to apply new resource limits)

Dangerous patterns:

- a. **Scale storm**: rapid scale-out causes cold-start latency → HPA scales further
- b. **Oscillation**: HPA over-corrects → idle pods → scale-in → spike → repeat
- c. **Cascade**: one service scales out → floods a downstream service



11) Resilience Engineering: High Availability & Disaster Recovery

RELIABILITY isn't really addable to a system after it's built: reliability emerges from many interacting design choices (redundancy, isolation, recovery, observability). A system that has **never been recovered** from failure has never been proven to be recoverable. **4 resilience properties at a glance:**

Property	Question	Typical measure	Main drivers
Availability <i>reachable now</i>	Is it reachable now?	Uptime % SLO	Redundancy Failover Health checks
Reliability <i>correct over time</i>	Is it doing the right thing?	Error rate Correctness SLI	Testing Consistency Error handling
Durability <i>data is not lost</i>	Will data survive?	Probability of data loss	Replication Backups Storage design
Recoverability <i>restoration after failure</i>	How well can it recover?	RTO RPO	DR plans Automation Drills

How a failure propagates? One component slows down under load, callers wait longer and start timing out; adjacent services are starved of resources and the failure cascades → but **cascade failures** aren't random, they are structural (tight coupling, shared resource pools, sync call chains...). Cascade happens in cloud-native systems because:

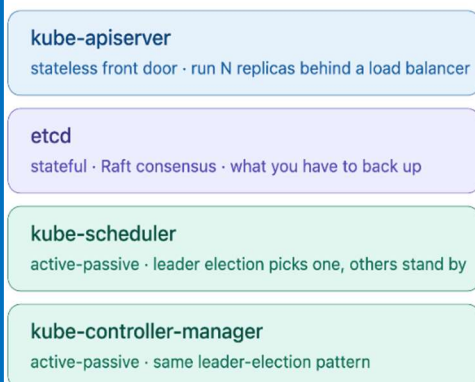
- microservices increase # of *call hops*
- each hop adds *latency variance* and a failure point
- *retries* are often implemented without backoff or jitter

⚠ We **design against cascades** using [*]:

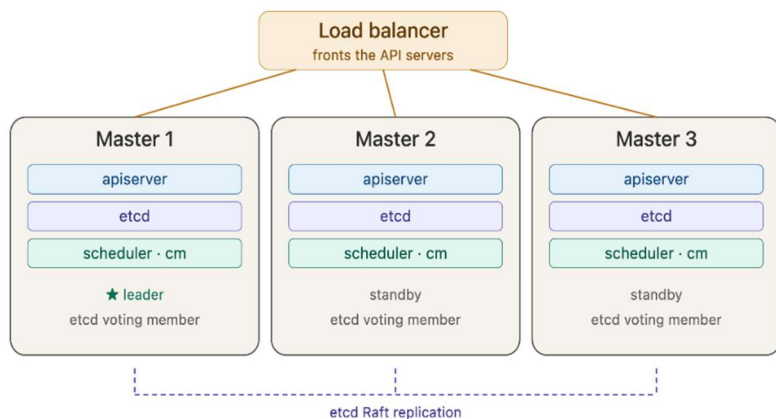
- *bulkheads* = isolated thread/connection pools per downstream
- *circuit breakers* = stop sending to a failing downstream
- *backpressure* = reject work rather than queue indefinitely
- *timeouts* = every call must have a deadline
- *load shedding* = reject non-critical work before the system collapses

Kubernetes control-plane HA (High Availability):

What runs in the control plane



Three control-plane nodes — the minimum for HA



Quorum: the rule that decides outages

etcd needs a majority to accept writes:

- 3 nodes → tolerates 1 failure
- 5 nodes → tolerates 2 failures
- even counts give no extra tolerance

Lose quorum → cluster goes read-only.
No scheduling, no scaling, no deploys.

What people get wrong

- Putting all masters in one AZ
→ AZ outage = quorum loss
- Skipping etcd backups
→ no recovery from data corruption
- Running etcd on shared, slow disk
→ fsync latency stalls the whole cluster

Workloads keep running

Pods on worker nodes stay up even if the control plane is fully down.

What you lose is the ability to **change** the cluster — deploy, scale, recover from node failure, reconcile.

Static workloads survive. Operations don't.

Principles of K8s' HA: replicas are necessary but not sufficient; replacement ≠ HA; startup time matters (slow startup widens every outage); readiness must remove unhealthy pods from traffic; true HA requires no single pod to be critical (no pod can irreplaceable).

PodDisruptionBudget (PDB) = limits simultaneous voluntary disruptions (node drains, rolling update, cluster upgrade) to a workload, but doesn't protect against involuntary failures (node crash). **minAvailable** means at least N pods must be running during disruption, while **maxUnavailable** means at most N pods may be down at once (these are mutually exclusive and expressed as count or percentage) [**PDB does not guarantee availability if replicas are badly placed**]

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: myapp-pdb
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: my-deployment
```

topologySpreadConstraints = distributes pods across topology zones (zone, node, rack) provided they are available; **maxSkew** = max allowed imbalance between topology buckets; **whenUnsatisfiable** → **DoNotSchedule** or **ScheduleAnyway** [without it, scheduler may pack all replicas into 1 zone]. Combining it with PDB is the best!

Health checks are **liveness** probe, **readiness** probe and **startup** probe (*probes* = control signals; bad probes cause bad automation) as said before and are the **feedback signal** that drives K8s self-healing.

Common mistakes are:

- liveness checks a downstream DB
- no readiness probe (so traffic sent to pods that aren't ready yet)
- readiness probe that never fails (so overloaded pods)

⚠ **Design** principles for probes → never use same endpoint for liveness and readiness (readiness can fail during overload, while liveness usually not)

Load Balancing distributes requests across healthy replicas (**L4** and **L7** load balancing here):



Traffic shaping for resilience:

- **canary** release → send 5% of traffic to new version, monitor SLO impact
- traffic **mirroring** → copy live traffic to a shadow service for dark testing
- **rate limiting** → reject excess requests before they saturate backends

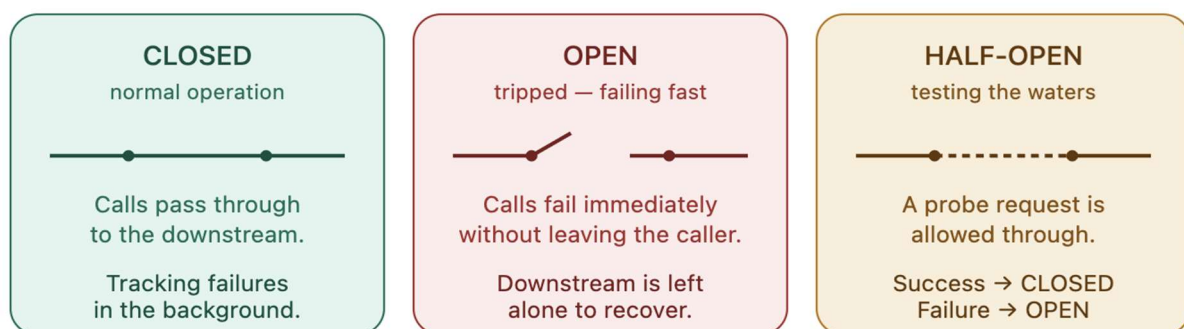
Connection draining on scale-down: **terminationGracePeriodSeconds** (allow in-flight requests to complete); **preStop** hook (sleep briefly so endpoint removal propagates first). **Graceful degradation** = disable optional features before core flows fail.

[*]

- **Circuit breaker pattern** = wraps calls to a downstream service; useful because **they prevent cascade** (a slow downstream no longer blocks callers) **but only** works when the caller has a **safe failure behavior** (cached response, degraded response, queued work or explicit error):



Three states



When to open the circuit

Error rate threshold

e.g. more than 50% of calls fail over the last 10 seconds.

Captures clear breakage.

Latency threshold

e.g. P99 latency exceeds 2 seconds.

Catches "slow death" — calls succeed but tie up threads and cascade upstream.

Why it matters

Prevent cascade

A slow downstream stops starving your threads and dragging callers down too.

Let it recover

An open circuit shields the downstream from retries while it gets back on its feet.

Fast fail

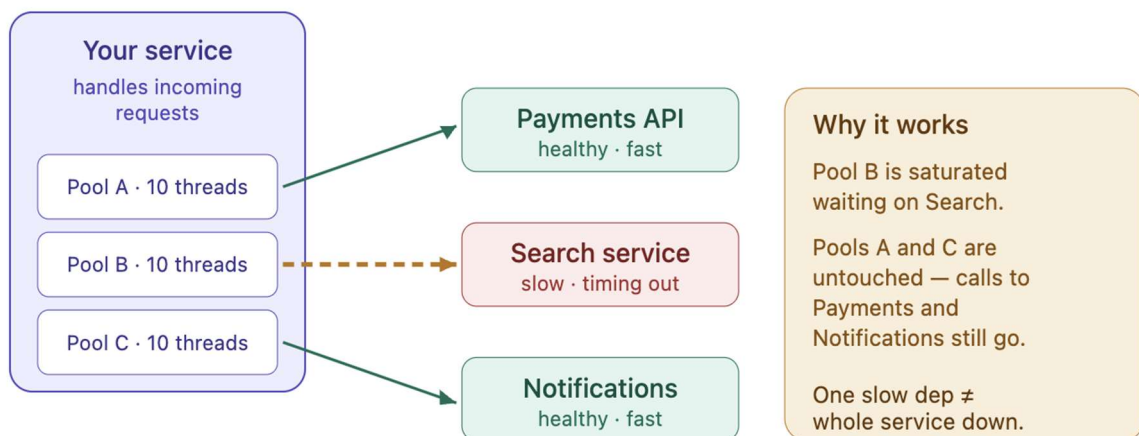
Callers immediately return an error or cached response instead of hanging on timeouts.

So, **circuit breaker** protects the caller and the downstream service at the same time!

➤ **Bulkhead pattern** = isolate resource pools per downstream service (*as watertight compartments in ships*) so that a slow service can't consume all threads/connections. **Implementation:**

- separate thread pools per HTTP client
- separate db connection pools per service
- separate queue consumers per event type

Service mesh approach: envoy sidecar implements circuit breaking and connection limits; no code changes required in the app (policy applied via **DestinationRule**)



Retries aren't free. **Exponential backoff** with jitter. **Timeout budget** for every call:



Exponential growth

Wait doubles each retry:
1s → 2s → 4s → 8s ...

Eases pressure on a struggling service.

Random jitter

Adds a random offset so many clients don't retry in lock-step.

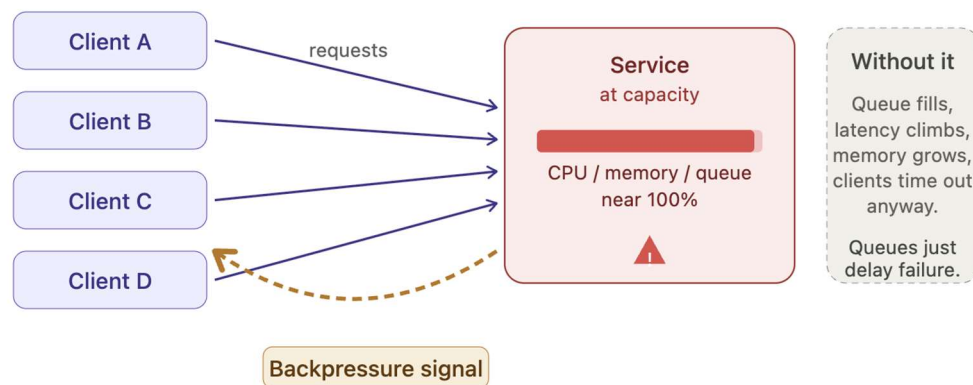
Breaks thundering herds.

Two hard caps

Max wait: e.g. 30s
(prevents endless growth)

Max retries: e.g. 3
(prevents endless trying)

Backpressure = mechanism for a system to signal that it can't accept more work (*better than queue indefinitely*); **backpressure signals** are **HTTP 429** (Too many requests), **HTTP 503** (service unavailable), **gRPC** (flow control on streams) and **Kafka** (consumer lag as a back-pressure signal). Backpressure is not failure: it's **controlled refusal**:



Common backpressure signals

HTTP 429 Too Many Requests Server tells client: retry after N seconds Retry-After header	HTTP 503 Service Unavailable Service refuses work while overloaded or recovering. Clients back off and retry.	gRPC streams Flow control Receiver advertises window size; sender pauses when the window is full.	Kafka Consumer lag Lag growing = consumers can't keep up. Slow producers down.
---	---	---	--

In **sync** approach, **primary waits for all replicas**; in **async** approach, **primary acks immediately and replicates later!** **Data** is the hardest thing to make resilient; in fact:

- **CAP** theorem revisited → during a network partition, you must choose:
 - o **CP** = refuse writes to preserve consistency (etcd, HBase)
 - o **AP** = allow writes and reconcile later (Cassandra, CouchDB)
- **PACELC**: even without a partition (E) you choose:
 - o **L** (lower *latency*) = accept eventual consistency
 - o **C** (stronger *consistency*) = accept higher latency

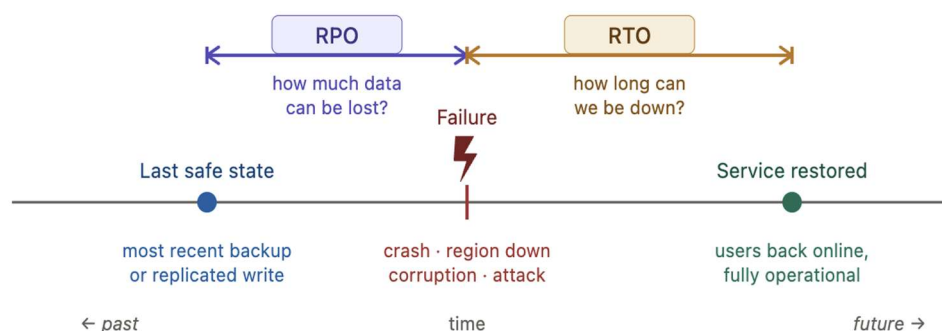
In cloud-native system, we want to **BACKUP**: persistent volume data (db, file stores), K8s cluster state (etcd snapshots), app config and secrets (store references, not values) and container registry (images pinned to digests). Backup **types**:

- *full* = complete copy (> cost, simple restore)
- *incremental* = changes since last backup (< cost, slow restore)
- *differential* = changes since last full backup (balance cost and restore)

Backup should be **encrypted**, **access-controlled** and **immutable** (3 copies of data, 2 different storage media, 1 copy offsite [3-2-1]): a backup you have never restored is a backup you don't have.

We define directly **from business** (not from engineering):

- **RTO** (**Recovery Time Objective**) = max acceptable time from failure to full service restoration (*DR strategy choice, automation level, runbook complexity*)
- **RPO** (**Recovery Point Objective**) = max acceptable data loss expressed as time (*replication strategy, backup frequency, consistency model*)



Lower RTO/RPO usually means **sharply higher cost and operational complexity**. Here the **4 disaster recovery strategies**:

Strategy	RTO	RPO	Cost	How it works	Choose when
Backup & Restore	Hours	Hours	Lowest	Restore from backup into a freshly provisioned env	Long outage and data-loss windows are acceptable
Pilot Light	30 min	Minutes	Low	Min core infrastructure always; scale out on DR event	Core services must restart quickly, but full capacity can wait
Warm Standby	Min	Seconds	Medium	Scaled-down full copy always; scale to full on DR event	Service must resume in minutes
Multi-site Active-Active	s	Near-zero	Highest	Full duplicate env serving live traffic at all times	Near-continuous availability is business-critical

⚠ **DR** is an organizational decision as much as a technical, but common *DR failures* are organizational!

Resilience is **multi-region** (reduce exposure to regional outages, reduce latency and meet data-residency regulations [GDPR]; *active-active* multi-region and global databases) and **multi-cloud** (protect against provider-wide events and avoid vendor lock-in for negotiation leverage). **Observability** is a prerequisite for resilience!

Chaos engineering = deliberately **injecting failure to verify resilience** (start small → begin in stage, not in prod; never inject a failure without a rollback or abort condition):

Tool	Scope	Failure types	Notes
Chaos Mesh	Kubernetes	Pod kill, network delay/loss, DNS failure, IO fault, time skew	CNCF project; declarative CRD; dashboard UI
Litmus Chaos	Kubernetes + cloud	Pod, node, network, storage, cloud-provider specific	CNCF project; workflow-based experiments
Chaos Monkey (Simian Army)	AWS / cloud VM	Random instance termination	Original Netflix tool; inspired the discipline
Toxiproxy	Network (any)	Latency, bandwidth limit, packet loss, TCP reset	Open source; ideal for integration tests

⚠ **Resilience anti-patterns to avoid:**

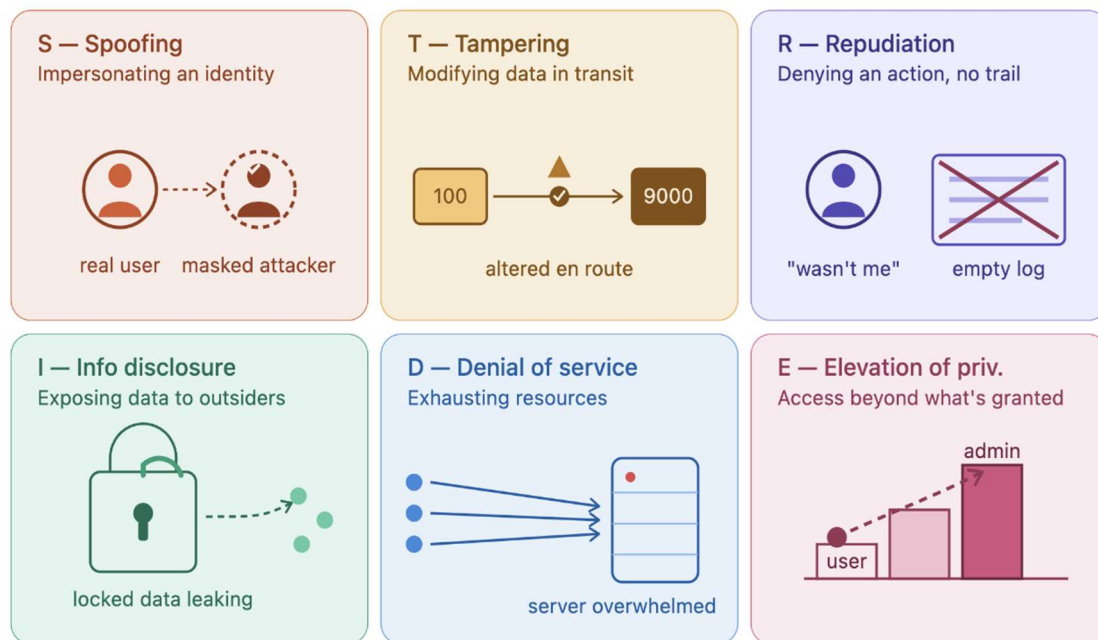
- ✗ assuming replicas = HA
- ✗ retry without backoff or circuit breaking
- ✗ DR plan that has never been tested
- ✗ RTO/RPO set without business input
- ✗ no observability

12) Cloud Security & DevSecOps

Security is about reducing blast radius, not preventing all attacks: traditional security model is “defend the boundary” (*firewall*), while in cloud-native **IP-based trust is meaningless** → “**assume breach, so design for containment, not for prevention!**” → **reduce at minimum Blast radius** (max damage a successful attack can cause) using design strategies (*day 0 decision*):

- least privilege
- isolation
- short-lived credentials
- immutable infrastructure (no ssh, no shell, no pivot)

So you must **think of threat modelling** (identify threats and mitigations at design time) **before the build** (at design - *Day 0*), using **STRIDE framework**:

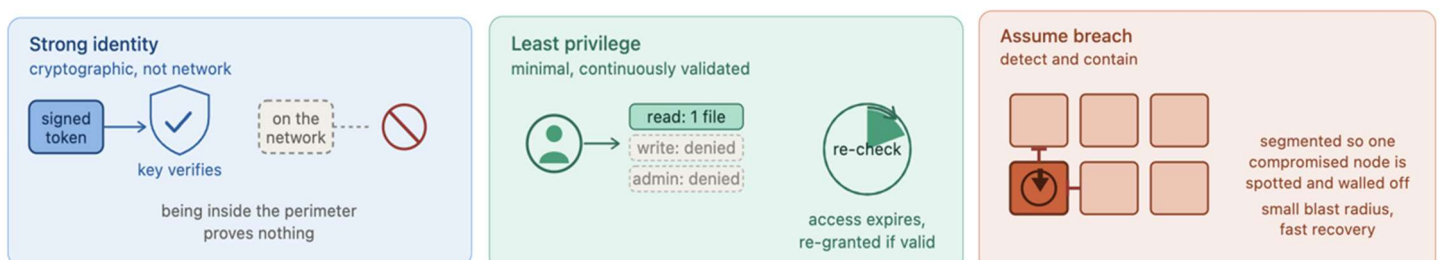


So, in cloud-native, IPs don't matter (as in network security), but it's **identity-based security** (*service accounts [K8s, IAM], mTLS between services, OIDC tokens, RBAC...*) [works **regardless of net topology**; every workload has a **cryptographic identity**; scales with **ephemeral infrastructure**].

Every Pod runs as a K8s **ServiceAccount** (*default ServiceAccount has broad permissions [so don't use it for prod]* → so **create a dedicated ServiceAccount per workload** using as always **least privilege**). As said before, RBAC is Role-Based Access Control (**Role** = what verbs on what resources [get, create...]; **RoleBinding** = which Service Account get which Role [**ClusterRole** / **ClusterRoleBinding** are used for *cluster-wide* permissions]).

⚠ **Workload Identity Federation** = bind a K8s ServiceAccount to a **cloud IAM role** (to have no more long-lived credentials stored in cluster)

As said before, the core principle in cloud is **never trust, always verify**, basing on:



These 3 principles are enforced using **service mesh** (Istio/Linkerd)!

As in traditional cybersecurity, there is a **Supply Chain risk**, but is **software-based** and there are **3 attack vectors in cloud**:

- **container images** → the image is the unit of deployment and so the unit of risk; we need to enforce:
 - o **base image hygiene** (use minimal images, fewer packages)
 - o **vulnerability scanning** (scan every image in CI before pushing to registry [Trivy] and rescan regularly)
 - o **image signing** (Cosign/Sigstore) and **attestation** (admission controllers and Software BOM)
- **open-source dependencies** → **problems**: known CVEs in dependencies, dependency confusion; **mitigations**: lock files [package-lock.json], software composition analysis (SCA) tools, artifact repo, SBOM generation and attestation
- **CI/CD pipelines** → these have privileged access to entire stack, so compromising them = compromising everything. **Problems**: secrets as env vars visible always, third-party GitHub Actions runs arbitrary code with repo access; **mitigations**: pin actions to a commit SHA (not a tag), separate credentials per environment (build ≠ stage ≠ prod), audit logs, run PR pipelines in isolated low-privilege context

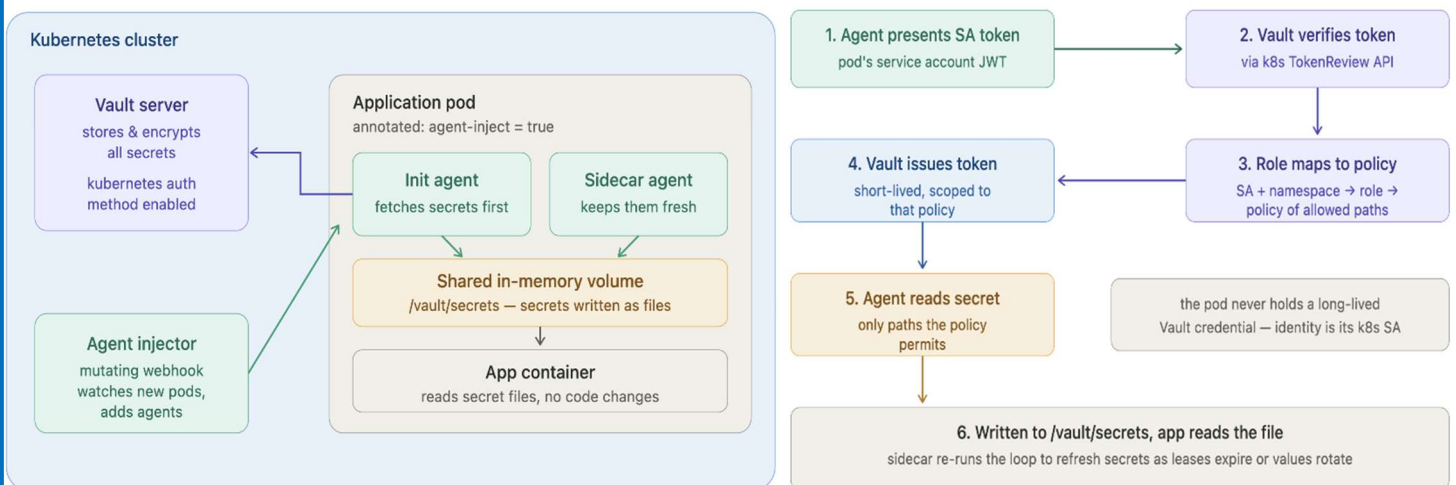
SECRETS MANAGEMENT: a secret is any value that grants access (API keys, pwds, TLS certs, tokens...).

There are 4 failure modes:

1. **Creation**: secret generated with insufficient entropy or reused
2. **Storage**: secret committed to git, stored in a config file...
3. **Distribution**: secret passed as an env var, visible in logs and in pod spec YAML
4. **Rotation**: secret never rotated (**lifetime problem**)

Patterns usable:

- **K8s secrets** (base64-encoded, stored in etcd [which must be encrypted at rest]; RBAC controls which pods can read which secrets) [**problems**: **secrets visible in pod spec YAML**]
- **External secret operators** (sync from AWS, GCP, Vault; moving source of truth outside cluster; rotation propagates automatically)
- **HashiCorp Vault** (**centralized secrets store** with fine-grained policies; also *dynamic secrets* on demand; every secret has a TTL; audit log of every secret access)



TLS inside the cluster encrypts traffic between services (even on private network), but **mTLS** add also **mutual authN** (both sides verify identity). **cert-manager** is K8s' certificate lifecycle manager (requests and rotates TLS certificates automatic) [**ClusterIssuer** = 1 CA for whole cluster; **Issuer** = namespace-scoped CA for multi-tenant isolation] [**certificates are secrets with an expiry, so manage them same way**]

⚠ **Shift-left security** = find issues earlier (pre-commit, pull request, CI build, CD deploy, runtime) make **fixing cheaper**; shift-left contract says: security team owns policies, while dev team get fast feedback from them and fix

Integrating security in CI/CD without blocking delivery:

- what must **block** pipeline:
 - o critical CVE in app image
 - o secret committed
 - o image not signed or fails attestation
 - o deployment manifest violates policy
- what should **warn** but not block:
 - o high CVE in a base OS package
 - o dependency with a low-severity issue
 - o licence compatibility warning

By default, **every pod can reach every other pod in cluster**: so, as said before, we can use **NetworkPolicy** resource to define ingress and egress rules per pod selector (**segmentation** [e.g. frontend pods may reach the API tier, but not db]). For **L7** policies, use a **service mesh**!

Admission controllers intercept every K8s API request before it's stored. **Policy-as-Code** (with OPA/Kyverno) is **writing security policies in code** (reviewed, versioned, tested like any other code), closing gap between what teams intend and what is running.

Pod Security Standards (PSS) are built-in K8s security profiles (replaced by PodSecurityPolicy in newer version of K8s) and have 3 levels:

- **Privileged** → no restrictions
- **Baseline** → prevents known privilege escalations (start from this)
- **Restricted** → hardened profile following current best practices:
 - o must not run as root
 - o must drop ALL Linux capabilities
 - o must use a non-root user ID
 - o must set seccompProfile to RuntimeDefault or Localhost
 - o hostPath volumes, hostNetwork, hostPID all prohibited

⚠ Prevention controls stop known threats (**runtime security detects unknown ones**) [use **Falco**]

Concern	Control	Where it lives
Identity & access	RBAC, ServiceAccounts, mTLS, Workload identity	Kubernetes, service mesh, cloud IAM
Secret management	External Secrets Operator, Vault, short-lived tokens	CI/CD, cluster, external store
Supply chain	Image scanning, signing (Cosign), SBOM, lock files	CI pipeline, registry, admission
Network segmentation	NetworkPolicy, service mesh AuthorizationPolicy	CNI, Istio/Linkerd
Policy enforcement	OPA, Kyverno, admission webhooks	Kubernetes API server
Runtime detection	Falco (eBPF), immutable containers, SIEM integration	Node kernel, alerting pipeline

⚠ **DevSecOps anti-patterns** to avoid:

- security as a gate at the end of pipeline
- treating CVEs all equal
- long-lived credentials
- security theatre
- flat cluster permissions